

공학박사 학위논문

적응적 서열 정렬 기법을 이용한 프로그램  
유사도 분석 프레임워크

2010년 8월

부산대학교 대학원

컴퓨터공학과

지정훈

공학박사 학위논문

적응적 서열 정렬 기법을 이용한 프로그램  
유사도 분석 프레임워크

지도교수 우 균

2010년 8월

부산대학교 대학원

컴퓨터공학과

지정훈

# 지정훈의 공학박사 학위논문을 인준함

2010년 6월 4일

주 심 류 광 렬 (인)

부 심 채 홍 석 (인)

위 원 양 태 천 (인)

위 원 조 장 우 (인)

위 원 우     균 (인)

# 차 례

|          |                                |          |
|----------|--------------------------------|----------|
| <b>1</b> | <b>서론</b>                      | <b>1</b> |
| 1.1      | 프로그램 유사도 분석 연구구의 필요성 . . . . . | 1        |
| 1.2      | 프로그램 표절 탐색 연구의 문제점 . . . . .   | 3        |
| 1.3      | 논문의 접근방법 . . . . .             | 5        |
| <b>2</b> | <b>관련연구</b>                    | <b>7</b> |
| 2.1      | 프로그램 유사도 분석 기법 연구 . . . . .    | 7        |
| 2.1.1    | 메트릭 기반 유사도 분석 . . . . .        | 8        |
| 2.1.2    | 텍스트 기반 유사도 분석 . . . . .        | 10       |
| 2.1.3    | 토큰 기반 유사도 분석 . . . . .         | 13       |
| 2.1.4    | 트리 기반 유사도 분석 . . . . .         | 16       |
| 2.1.5    | 유사도 분석 시각화 . . . . .           | 20       |
| 2.1.6    | 다른 유사도 분석 방법들 . . . . .        | 25       |
| 2.2      | 유사도 분석 시스템 . . . . .           | 27       |
| 2.2.1    | 문서 유사도 분석 시스템 . . . . .        | 28       |
| 2.2.2    | 소스코드 유사도 분석 시스템 . . . . .      | 33       |

|          |                     |           |
|----------|---------------------|-----------|
| <b>3</b> | <b>프로그램 유사도 분석</b>  | <b>39</b> |
| 3.1      | 적응적 유사도 분석 프레임워크 구조 | 39        |
| 3.2      | 프로그램 서열 생성          | 42        |
| 3.2.1    | 프로그램 소스코드           | 42        |
| 3.2.2    | 프로그램 선형화            | 44        |
| 3.2.3    | 프로그램 정적 추적          | 46        |
| 3.2.4    | 직병렬 그래프 모델 기반 정규화   | 51        |
| 3.3      | 적응적 지역정렬            | 53        |
| 3.3.1    | DNA 서열 정렬 기법        | 53        |
| 3.3.2    | 전체정렬                | 54        |
| 3.3.3    | 지역정렬                | 55        |
| 3.3.4    | 적응적 지역정렬            | 56        |
| 3.4      | 유사도 정규화             | 62        |
| <b>4</b> | <b>클러스터링과 진화분석</b>  | <b>64</b> |
| 4.1      | 클러스터 분석을 위한 확률 모델   | 64        |
| 4.1.1    | 의사표절 정의             | 65        |
| 4.1.2    | 클러스터링 알고리즘          | 68        |
| 4.2      | 프로그램 진화분석           | 72        |
| 4.2.1    | 프로그램 진화거리           | 73        |
| 4.2.2    | 프로그램 진화 트리 생성 알고리즘  | 74        |
| <b>5</b> | <b>실험</b>           | <b>77</b> |
| 5.1      | 실험데이터               | 81        |
| 5.2      | 프로그램 표절탐색           | 84        |
| 5.3      | 유사구간 탐색             | 89        |

|                                     |            |
|-------------------------------------|------------|
| 5.4 JPlag 시스템과의 비교 . . . . .        | 92         |
| 5.5 기존 지역정렬 방법과의 비교 . . . . .       | 98         |
| 5.6 프로그램 진화 분석 . . . . .            | 102        |
| <b>6 결론</b>                         | <b>107</b> |
| <b>A 부록</b>                         | <b>123</b> |
| A.1 프로그램 집합에 대한 키워드 빈도 조사 . . . . . | 123        |

# 그림 차례

|                                                                                                      |    |
|------------------------------------------------------------------------------------------------------|----|
| 1.1 프로그램 유사성 분석 : $P_a$ 와 $P_b$ 사이에 유사하게 작성된 부분을 탐색함. . .                                            | 2  |
| 2.1 필터링 및 정규화를 이용한 소스코드 변환 . . . . .                                                                 | 11 |
| 2.2 Dot-plot을 이용한 agrep 소스코드 시각화 . . . . .                                                           | 12 |
| 2.3 토큰 기반 유사도 분석 시스템 Sim의 블록 다이어그램 . . . . .                                                         | 14 |
| 2.4 C++로 작성된 샘플코드와 변환된 토큰 서열 . . . . .                                                               | 15 |
| 2.5 트리 기반 유사도 분석 시스템 DECKARD의 블록 다이어그램 . . . . .                                                     | 17 |
| 2.6 파스트리와 파스트리에 나타날 수 있는 부분 트리 예시 . . . . .                                                          | 17 |
| 2.7 C언어로 작성된 소스코드 샘플 . . . . .                                                                       | 19 |
| 2.8 C 소스코드에 대한 AST와 AST 직렬화 . . . . .                                                                | 20 |
| 2.9 Dot-plot을 이용한 소스코드 유사도 시각화 분석: A 지역은 코드가 삭제된 것<br>을 나타내고, B 지역은 코드(클래스)가 새롭게 추가된 것을 나타냄. . . . . | 21 |
| 2.10 문자열에 대한 패턴그램과 CCP 시각화 . . . . .                                                                 | 22 |
| 2.11 CCP를 이용한 프로그램 유사도 탐색 시각화 결과 . . . . .                                                           | 23 |
| 2.12 소스코드 유사도 분석 시각화 방법 . . . . .                                                                    | 24 |
| 2.13 SCAM: 쿼리를 이용한 문서 표절검사 시스템 구조 . . . . .                                                          | 30 |
| 2.14 DEVAC 표절 검사 시스템 센터 . . . . .                                                                    | 31 |
| 2.15 DEVAC 표절 검사 클라이언트 프로그램 검사결과 . . . . .                                                           | 32 |

|                                                                                        |    |
|----------------------------------------------------------------------------------------|----|
| 2.16 AC를 이용한 소스코드 유사도 분석 시각화: 테이블 및 그래프, 히스토그램<br>을 이용한 시각화 인터페이스 제공 . . . . .         | 34 |
| 2.17 CCFinderX를 이용한 코드 클론 검사 결과 . . . . .                                              | 36 |
| 2.18 JPlag를 이용한 소스코드 표절검사 결과 . . . . .                                                 | 38 |
| 3.1 적응적 유사도 분석 프레임워크 구성도 . . . . .                                                     | 40 |
| 3.2 프로그램 $P_a$ : 거품 정렬 원본 소스코드 . . . . .                                               | 43 |
| 3.3 프로그램 $P_b$ : $P_a$ 와 유사하게 작성된 거품 정렬 프로그램 . . . . .                                 | 43 |
| 3.4 프로그램 정적 추적을 이용한 프로그램 DNA 서열 추출: 그림 3.3에 대한 토<br>큰 서열 . . . . .                     | 45 |
| 3.5 거품 정렬 소스코드의 순차적 토큰 서열 생성 그래프 . . . . .                                             | 48 |
| 3.6 PST를 이용한 거품 정렬 소스코드의 토큰 서열 생성 그래프 . . . . .                                        | 48 |
| 3.7 factorial.c : 1부터 10까지의 계승을 구하는 프로그램 . . . . .                                     | 49 |
| 3.8 직접 순환함수 호출에 대한 토큰 생성: factorial.c에 대한 토큰 생성. . . . .                               | 50 |
| 3.9 간접 순환함수 호출에 대한 토큰 생성: $\text{main} \mapsto f \mapsto g \mapsto h \mapsto f$ 순서로 호출 | 50 |
| 3.10 SP 그래프 정규화: (a) 순차블록, (b) <i>if-else</i> , (c) <i>switch-case</i> . . . . .       | 52 |
| 3.11 전체정렬을 이용한 서열 정렬: “CACAGTGT”와 “CAGGT” 서열 . . . . .                                 | 55 |
| 3.12 지역정렬을 이용한 서열 정렬: “AGGTATTG”와 “CTATGC” 서열 . . . . .                                | 57 |
| 3.13 적응적 지역정렬을 이용한 프로그램 유사도 계산 . . . . .                                               | 57 |
| 4.1 2006년 KOI의 프로그램 그룹에 대한 <i>Asym()</i> 분포(점): 굵벨 분포(실선)와 유사                          | 67 |
| 4.2 ICPC05-A 그룹에 대한 <i>Asym()</i> 분포(점): 굵벨분포를 많이 벗어남 . . . . .                        | 67 |
| 4.3 의사표절 탐색을 위한 클러스터링 알고리즘 . . . . .                                                   | 71 |
| 4.4 프로그램 진화트리 생성 과정 . . . . .                                                          | 76 |

|      |                                                                                                                   |     |
|------|-------------------------------------------------------------------------------------------------------------------|-----|
| 5.1  | <b>SoVAC</b> 시스템: 적응적 프레임워크 기반의 프로그램 유사도 분석 도구, 프로그램 $P_a$ (왼편)의 32 ~ 37 라인과 $P_b$ (오른편)의 17 ~ 21 라인이 유사. . . . . | 79  |
| 5.2  | <b>PlaView</b> : 표절 클러스터 및 진화분석을 위한 그래프 시각화 도구 . . . . .                                                          | 80  |
| 5.3  | ICPC05에서 표절로 의심되는 프로그램 $P_a$ . . . . .                                                                            | 90  |
| 5.4  | $P_a$ 의 표절로 의심되는 프로그램 $P_b$ . . . . .                                                                             | 90  |
| 5.5  | 유사도 기준 값( $x$ -축)의 변화에 따른 SoVAC과 JPlag의 민감도 실험결과 . . .                                                            | 94  |
| 5.6  | 유사도 변화에 따른 SoVAC과 JPlag의 정확도( $SP$ ) 실험결과 . . . . .                                                               | 96  |
| 5.7  | ICPC 프로그램 집합에 대한 $SN$ 과 $SP$ 교차 그래프 . . . . .                                                                     | 97  |
| 5.8  | 유사도 기준 값( $x$ -축)의 변화에 따른 적응적 방법(LAA)과 고정적 방법(LAS)의 민감도 비교 . . . . .                                              | 99  |
| 5.9  | 유사도 기준 값의 변화에 따른 적응적 방법(LAA)과 고정적 방법(LAS)의 민감도 비교 . . . . .                                                       | 100 |
| 5.10 | 적응적 지역정렬과 고정적 지역정렬의 $SN$ 과 $SP$ 교차 그래프 . . . . .                                                                  | 101 |
| 5.11 | 파라미터 조정 결과: $\alpha = 0.65$ , $\gamma = 2.0$ , and $\delta = 7.2$ . . . . .                                       | 103 |
| 5.12 | 프로그램 진화 트리: 올바른 진화 트리 (a)와 잘못된 진화 트리 (b), (c) . . . .                                                             | 104 |

# 표 차례

|      |                                                               |     |
|------|---------------------------------------------------------------|-----|
| 3.1  | ICPC06-C 프로그램 그룹에 사용된 키워드 빈도 . . . . .                        | 61  |
| 5.1  | 실험데이터: ICPC 대회 제출 프로그램(2005년부터 2008년까지) . . . . .             | 82  |
| 5.2  | 실험데이터: 2008년도와 2010년도 C++ 프로그래밍 교과목에 제출된 프로그램<br>집합 . . . . . | 83  |
| 5.3  | ICPC 표절 탐색 결과: ICPC07 그룹에서 7개 팀의 소스코드가 표절로 검출되<br>었음. . . . . | 85  |
| 5.4  | 2007년도 ICPC 동아시아대회 본선 프로그램 집합에 대한 표절탐색 결과 . .                 | 86  |
| 5.5  | 2008년도 C++ 프로그래밍 실습 과제 프로그램 집합에 대한 표절탐색 결과 .                  | 88  |
| 5.6  | 2008년도 C++ 프로그래밍 중간 및 기말고사 프로그램 집합에 대한 표절탐<br>색 결과 . . . . .  | 88  |
| 5.7  | 표절 상황 및 표절 판정에 대한 4가지 결과 . . . . .                            | 93  |
| 5.8  | 프로그램 진화분석 알고리즘 실험데이터 . . . . .                                | 102 |
| 5.9  | 진화분석 실험에 사용된 공개 소스 프로젝트 실험데이터 . . . . .                       | 105 |
| 5.10 | 공개 소스 프로젝트 진화분석 실험결과 . . . . .                                | 106 |
| A.1  | ICPC 실험데이터에 대한 전체 키워드 빈도: 총 247,798개의 키워드가 사용됨                | 123 |

|                                                                  |     |
|------------------------------------------------------------------|-----|
| A.2 C++ 프로그래밍 실험데이터에 대한 키워드 빈도: 총 97,731개의 키워드가<br>사용됨 . . . . . | 124 |
| A.3 GDB 프로젝트에 대한 키워드 빈도: 총 2,862,665개의 키워드가 사용됨 . . . .          | 124 |

# 제 1장

## 서론

프로그램 유사도 분석 연구는 두 프로그램들 사이의 유사도를 정량적으로 측정하고 프로그램에서 실제로 유사하게 작성된 코드 블록을 찾기 위한 연구이다. 본 장에서는 첫 번째로 프로그램 유사도 분석 연구가 왜 필요한지에 대해 살펴본다. 다음으로 프로그램 유사도 분석 연구에서 해결해야하는 문제들에 대해 살펴보고 이를 해결하기 위한 본 논문의 접근 방법들을 대략적으로 살펴본다.

### 1.1 프로그램 유사도 분석 연구의 필요성

프로그램 유사도 분석 연구는 두 프로그램 사이의 유사도를 정량적으로 측정하고 유사하게 작성된 코드 조각을 찾는 연구이다. 프로그램 유사도는 주로 소스코드를 이용해 계산된다. 하나의 프로그램은 작게는 수십에서 수백줄 정도로 구현되지만 규모가 큰 프로그램의 경우 소스코드는 수만에서 수십만줄에 이른다. 프로그램 유사도 분석 연구에서는 주로 소스코드를 입력으로 하여 기계적인 방법으로 유사도와 유사구간을 찾는다. 그림 1.1은 두 소스코드에서 유사 모듈을 찾는 것을 나타낸 것이다.

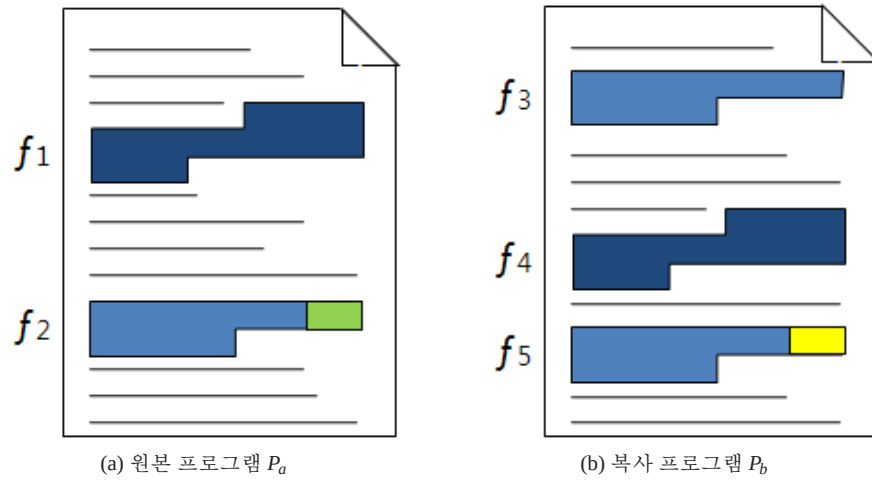


그림 1.1: 프로그램 유사성 분석 :  $P_a$ 와  $P_b$  사이에 유사하게 작성된 부분을 탐색함.

그림 1.1에서는 두 프로그램  $P_a$ 와  $P_b$ 에서 몇몇 함수들이 중복 사용된 것을 나타낸다.  $P_a$ 의 함수  $f_1$ 과  $P_b$ 의 함수  $f_3$ 은 동일하게 작성되었으며,  $f_5$ 는  $f_2$ 를 약간의 수정을 통해 작성한 것을 나타낸다. 프로그램 유사도 분석에서는 그림 1.1에서처럼 동일하게 작성되었거나 유사하게 작성된 모듈들을 찾는 것이 주요 기능이다.

프로그램 유사도 분석 연구는 주로 프로그램 표절검사에 활용된다. 표절 문제는 최근 들어 대학 등에서 심각한 문제로 인식되고 있다. 그리고, 대학생들 사이의 과제 보고서 표절은 이미 심각한 수준에 이르렀다고 할 수 있다. 한 연구에 따르면, MIT 대학의 기초 프로그래밍 강의 수강생들 중 약 30%의 학생들이 표절로 규제를 받았다는 조사 결과가 있다[1]. 이와 같은 사실은 대학 내에서 표절이 심각한 문제가 될 수 있다는 것을 나타낸다.

프로그램 소스와 같이 그 언어가 자연언어와 다른 경우에는 표절 탐색이 매우 어렵다. 또한, 소스코드의 경우 사람이 직접 비교하여 표절된 프로그램을 찾기란 불가능에 가깝다. 예를 들어, 만약 한 강의에서 100개 이상의 소스코드가 제출되었다면 모든 소스코드를 비교하기 위해서는 5000쌍 이상 코드를 비교해야 한다. 한 쌍의 소스코드를 검사하는데 평균 1분이 소요된다고 가정하더라도 1일 평균 8시간 검사기준으로 약 10일 정도가 소요된다.

이처럼 프로그램 소스코드를 사람이 직접 분석하기는 매우 어렵다. 이에 프로그램 표절 탐색의 경우 자동화된 표절검사 시스템이 필수적으로 요구된다[2]. 프로그램 표절검사 시스템은 프로그래밍 교과목에서 프로그램 유사도 분석 결과를 정량적으로 나타내어 교수나 조교로 하여금 빠른 시간에 학생들의 프로그램으로부터 표절 프로그램을 찾는 데 활용된다.

프로그램 유사도 분석은 소프트웨어 유지보수에서도 활용된다. 프로그램 개발에서 복사·붙여넣기(copy-to-paste), 템플릿 코드 사용 등과 같은 행위는 빈번하게 발생하는 것으로, 이로 인해 중복된 코드가 여러 군데에 생성된다. 한 연구에 따르면 소프트웨어 시스템에서 5%~20% 정도로 중복된 코드가 사용되고 있다고 한다[3]. 소프트웨어 개발에서 코드를 중복하여 사용하는 이유는 개발 시간을 단축 시킬 수 있고, 수정을 통한 시스템 확장이 용이하기 때문이다.

하지만, 버그가 있는 코드를 여러 군데 중복 사용할 경우 소프트웨어 시스템은 오류를 발생시킬 확률이 매우 높다. 예를 들어, 사용자 정의 라이브러리 코드에 버그가 있으면 이 라이브러리가 사용된 모든 코드에서 프로그램 오동작을 일으킬 수 있다. 이를 해결하기 위해서는 시스템에서 사용된 모든 라이브러리 코드를 찾아 수정을 해야 한다. 중복된 코드 사용은 소프트웨어 시스템에 위험을 일으킬 수 있고, 유지보수를 더욱 어렵게 만든다. 이처럼 표절검사나 소프트웨어 유지보수에서는 자동화된 프로그램 유사도 분석 도구가 필요하다.

## 1.2 프로그램 표절 탐색 연구의 문제점

프로그램 유사도 분석은 프로그램 표절 연구에 주로 이용되고 있으며, 표절 탐색을 위한 자동화된 시스템들도 많이 개발되었다. 자동화된 표절 탐색 시스템들은 다양한 유사도 측정 알고리즘을 기반으로 프로그램 집합의 소스코드들 사이의 표절을 빠르게 탐색한다. 표절 시스템들에 적용된 정량적 유사도 산출 기법들은 정보검색, 데이터마이닝, 생물정보학, 프

로그그램 분석, 소프트웨어 공학 등 다양한 분야에서 오래전부터 연구되었다.

그러나 지금까지의 표절 연구에서는 몇 가지 문제가 있다. 먼저, 한 프로그램이 다른 프로그램을 표절하였는지를 판별하는 객관적인 기준이 없다. 극단적으로 말하자면 어떤 두 프로그램이 독립적으로 작성되었다고 하더라도 거의 같은 가능성을 배제할 수 없다. 특히, 어떤 문제를 해결하고자 하는 방법이 제한적으로 주어졌을 경우, 예를 들어 정수 배열에 대한 거품정렬(bubble sort)을 작성하라고 제한하면 그 사항을 충실히 따른 프로그램은 거의 같을 수밖에 없다. 결국 이 문제는 확률의 문제로 귀결될 수밖에 없는데, 즉 어떤 길이의 프로그램 A가 다른 프로그램 B와 표절없이 거의 유사할 경우에 대한 확률모델을 제시하고 그에 근거하여 두 프로그램의 표절에 대한 가능성을 밝히는 것이 최선을 방책이라고 할 수 있다.

기존 표절 연구의 또 다른 문제점은 두 프로그램이 표절을 했을 경우 표절의 주체가 되는 원본 프로그램을 찾기가 어렵다는 것이다. 지금까지 제안된 알고리즘들은 대칭적으로 유사도를 계산한다는 것이다. 즉, 두 프로그램  $P_a$ 와  $P_b$ 의 유사도 측정 함수를  $Sim(P_a, P_b)$ 라고 하면,  $Sim(P_a, P_b)$ 와  $Sim(P_b, P_a)$ 의 유사도가 같다는 것이다( $Sim(P_a, P_b) = Sim(P_b, P_a)$ ). 프로그램 유사도 분석에서 두 프로그램 사이의 표절 관계를 분석하고 어느 부분이 표절되었는지 밝히는 것은 표절 연구에서 기본적으로 해결되어야 하는 문제이다.

표절연구에서의 마지막 어려움은 실험자료로 쓰일 실제적인 표절자료를 확보하는 것이다. 결국 개발된 시스템이 잘 동작하는지를 증명하기 위해서는 실제적인 상황에서 제대로 동작함을 보여야 한다. 하지만 표절행위는 일반적으로 불법적인 것으로 인식되어 있기 때문에 이를 정상적인 방법으로 구하기란 매우 힘들다. 한편 인위적으로 어떤 프로그램을 주고 표절한 프로그램을 양산할 수도 있지만 이는 실제상황에서 만들어진 것이 아니므로 정확한 의미에서 표절데이터로 보기는 힘들 것이다.

본 연구에는 위에서 제시한 여러 문제에 대해 현재까지 제안된 방법들 보다 진일보된 새로운 방법을 제시하고 그 방법이 보다 타당함을 다양한 데이터와 실험의 특성값을 비교함으로써 보이고자 한다. 그리고 실제 표절 데이터를 이용하여 본 논문에서 제안하는 적응적

유사도 분석 방법이 기존의 방법보다 우수함을 보인다.

### 1.3 논문의 접근방법

본 논문에서는 소스코드 기반 프로그램 유사도 분석을 위한 프레임워크를 설계하고 구현한다. 본 논문에서 제안하는 프레임워크는 프로그램 그룹의 특성에 따라 적응적으로 유사성을 분석하는 방법으로 지역정렬(local alignment)에 기반한다. 본 연구에서 제안하는 프로그램 유사성 분석 프레임워크의 특징을 요약하면 다음과 같다.

- **프로그램 선형화:** 프로그램 소스코드는 일반 문서와 다르게 구조적 정보와 흐름 정보를 포함한다. 프로그램 선형화는 유사도 분석의 전단부 작업으로 유사도 분석의 정확성 향상을 위해 프로그램 정적 추적(Program Static Tracing)을 사용해 프로그램 토큰 서열을 생성한다.
- **적응적 지역정렬 :** 생물정보학에서 DNA나 단백질(protein)의 서열분석을 위해 사용되는 정렬(alignment)을 이용해 프로그램 유사도 분석에 적합한 새로운 알고리즘인 적응적 지역정렬 알고리즘(Adaptive Local Alignment)을 제안한다. 적응적 지역정렬 알고리즘은 프로그램 소스코드의 키워드 빈도에 따른 적응적 유사도 점수 행렬(matching score matrix)을 이용하는 방법이다. 이는 정보이론에 입각한 매우 새로운 방법이다.
- **확률 모델에 기반한 클러스터링:** 기존 표절 탐색 알고리즘의 단점은 어떤 두 개체의 국부 유사도가  $p\%$  일 때, 이것을 어떻게 해석하는가에 대한 체계적인 연구가 없었다는 것이다. 이에 대해 본 연구에서는 길이가 다른  $M$ 과  $N$ 의 독립적인 프로그램에서 한 부분이 같을 때, 그 정도의 분포를 확률모델로 구성하여 그것이 우연히 나타날 가능성을 계산하고 그 값을 기준으로 두 개체의 표절 여부를 판정한다.
- **프로그램 진화과정 분석:** 프로그램 진화분석은 생물정보학 등에서 연구되고 있는 개체의 진화론적 분석 방법과 유사하게 프로그램 개체의 진화과정을 분석한 것이다. 여기서 프로

그럼 개체는 소스코드에 해당한다. 본 연구에서는 적응적 지역정렬에 기반한 비대칭 유사도 함수를 이용해 프로그램간 진화거리를 측정하고 소스코드 진화분석 알고리즘을 통해 삽입, 삭제, 수정 등의 작업에 대한 소스코드 진화과정을 추정하고 계통도를 그린다.

본 논문은 전체 6장으로 구성되었으며 각 장의 구성은 다음과 같다. 2장에서는 관련연구로 유사도 분석 연구에 대해 기술한다. 관련연구는 지금까지 연구된 유사도 분석 기법들을 분류하고 이들의 특징을 살펴본다. 또한, 일반 문서와 프로그램을 대상으로 하는 대표적인 유사도 분석 시스템에 대해 기술한다. 3장에서는 본 논문에서 제시하는 프로그램의 유사도 분석을 위한 적응적 프레임워크의 구조에 대해 소개하고 프레임워크를 이루는 모듈과 알고리즘들에 대해 기술한다. 4장에서는 프로그램 유사도 기반 표절검사에서 해결해야 하는 문제점과 해결 방법을 제시한다. 이 장에서는 기존의 연구들에서 한 단계 더 발전한 확률적 모델에 기반한 유사도 분석 알고리즘에 대해 소개한다. 또한, 비대칭 적응적 지역정렬을 이용한 프로그램 소스코드 진화분석(evolution analysis)에 대해 다룬다. 다음으로 5장에서는 실험을 통해 본 논문에서 제시하는 적응적 유사도 분석 프레임워크의 성능을 측정하고, 기존의 유사도 분석 시스템들과 비교분석 한다. 마지막으로 6장에서는 결론과 함께 향후 연구 과제에 대해 기술한다.

## 제 2장

# 관련연구

유사도 분석에 관한 연구는 약 30여년 전부터 진행된 주제로 현재까지 유사도 분석을 위한 다양한 기법들과 시스템들이 개발되었다[4, 5, 6]. 본 장에서는 지금까지 제안된 프로그램 유사도 분석 기법들을 비교 방법기준으로 분류하여 설명하고 각 방법들에 대한 유사도 분석 수준에 대해 알아본다. 또한, 제안된 기법들을 응용하여 개발된 시스템들에 대해 살펴본다.

### 2.1 프로그램 유사도 분석 기법 연구

초기의 프로그램 유사도 분석 연구에서는 일반 문서 유사도 분석에 사용된 메트릭(metric)에 기반한 방법들이 응용되어 사용되었다. 하지만, 소스코드는 일반 문서와 구조적 특성이 달라서 메트릭 기반 방법들이 효율적이지 못했다. 이에 대해, 최근의 프로그램 유사도 분석 연구에서는 문자열(string), 추상구문트리(abstract syntax tree), 토큰(token), 그래프(graph) 등을 이용한 구조적 분석(structural analysis) 기법이 많이 사용되고 있다. 본 절에서는 프로그램 유사도 분석에 사용되고 있는 기법들을 대해 자세히 살펴본다. 본 절에서 설명하는 유사

도 분석 기법들은 주로 소스코드를 대상으로 하여 두 프로그램 유사도를 산출한다.

### 2.1.1 메트릭 기반 유사도 분석

초기의 프로그램 소스코드 유사도 분석 연구에서는 일반 문서 유사도 비교에 사용된 방법들이 그대로 사용되었다. 이 방법들은 소스코드를 단순 문자열로 이루어진 텍스트 파일로 보고 두 개의 소스코드를 비교하였다. 먼저, 소스코드를 비교하기에 앞서 코드에 포함된 주석(comment) 및 공백(whitespace) 등을 제거한 다음 텍스트 파일 비교 유틸리티인 diff[7]나 grep을 사용하여 두 소스코드의 유사도를 측정하였다. diff를 이용한 방법은 소스코드 유사도 비교 방법 중 가장 단순한 방법에 속한다. 텍스트 비교 프로그램들을 소스코드 유사도 측정에 사용하는 이유는 많은 양의 파일들을 빠르게 비교할 수 있기 때문이다. diff는 소스코드 비교에서 많은 정보를 제공하지는 않지만 여러 버전의 소스코드에서 변화된 부분을 쉽게 추적할 수 있다.

메트릭 기반 소스코드 유사도 분석은 지문법(fingerprints)으로 불리는 방법으로 소스코드를 입력으로 하여 소스코드에 포함된 문법적 코드 정보(syntactic code units)들을 이용하여 유사도를 측정한다. 여기서 문법적 코드 정보란 소스코드의 라인 수(LOC:line of codes) 및 연산자(operator), 피연산자(operand), 변수(variable), 함수(function or methods)와 같은 소스코드에 포함된 통계적 정보를 의미한다. 이 방법은 두 프로그램  $P$ 와  $Q$ 를 입력으로 하여  $0 \sim 1$  사이의 유사도  $S$ 를 계산한다.

초기의 메트릭 기반 소스코드 유사도 분석 기법은 Halstead's 메트릭을 기반으로 한다[8]. Halstead's 메트릭은 소스코드에서 4가지의 간단한 속성  $H_{attr}$ 을 이용해 프로그램의 양적 통계치와 프로그래머의 정신적 노력을 정량적으로 측정하는 메트릭이다. Halstead's 메트릭의 속성  $H_{attr}$ 은 다음과 같이 정의된다.

$$H_{attr} = \langle n_1, n_2, N_1, N_2 \rangle$$

여기서  $n_1$ 은 소스코드에 사용된 연산자 수를 나타내고,  $n_2$ 는 피연산자의 수를 나타낸다. 그리고  $N_1$ 과  $N_2$ 는 각각 연산자와 피연산자가 소스코드에 사용된 횟수를 나타낸다.

Halstead's 메트릭은 최초로 Ottenstein[9]에 의해 소스코드 표절검사 시스템에 사용되었다. Ottenstein의 연구에서는 두 프로그램  $P$ 와  $Q$ 의 Halstead's 메트릭 값이 비슷하면 두 프로그램은 표절된 것으로 가정하고 검사자에게 알려준다. Ottenstein 이후의 연구에서는  $H_{attr}$  속성과 반복문, 제어문, 함수 개수와 같은 여러 가지 속성들을 메트릭에 추가하여 유사도 측정의 정확성을 높였다[10, 11].

메트릭 기반 유사도 분석법은 여러 가지의 메트릭을 결합하여 하나의 메트릭으로 만들어서 이용되기도 하였다. Johnson[12]의 연구에서는 가장 간단한 방법으로 UNIX의 유틸리티인 wc의 출력 결과인 물리적 메트릭(physical metric)과 Halstead's 메트릭을 이용하여 6개의 속성을 이용하는 결합 메트릭(composite metric)을 이용하였다. Stephens[13]의 연구에서는 위의 3가지 메트릭에 의해 계산된 유사도를 유클리드의 거리(Euclidean distance)로 나타내어 두 프로그램  $p$ 와  $q$ 의 유사도  $S_{Shephens}$ 를 계산하였다.  $S_{Shephens}$ 은 다음과 같이 계산된다.

$$S_{Shephens} = \sqrt{[(P_p - P_q)^2 + (H_p - H_q)^2 + (C_p - C_q)^2]} \quad (2.1)$$

여기서  $(P, H, C)$ 는 각각 물리적 메트릭, Halstead's 메트릭, 결합 메트릭 계산 값을 나타낸다.

이밖에도 Donalson과 Lancaster의 연구[14]에서는 포트란 프로그램을 대상으로 변수, 서브프로그램, 문장(statement)에서 추출한 8개의 속성계수(attribute counting)를 사용하여 유사도를 측정하였다. 또한, 이 연구에서는 소스코드를 문자열 표현으로 치환하여 비교하는 방법을 함께 사용하였다. 두 소스코드에서 문자열이 일치하고 속성계수가 유사하게 나타나면 두 프로그램은 표절된 것으로 판단하였다. Mayrand[15]의 연구에서는 함수에 포함된 CFG 에지, 함수호출 개수와 같은 메트릭을 사용하였으며, 함수단위 분석을 위해 중간표현으로 IRL(Intermediate Representation Language)를 이용하였다. 이와 유사하게 Patenaude[16]는 자바를 대상으로 메소드 수준 유사도 분석을 위해 메소드 호출과 문장, 사

이클로매틱 복잡도(cyclomatic complexity) 등을 이용하였다.

메트릭 기반 유사도 분석은 구현하기가 쉽고 빠르게 유사도를 계산할 수 있다는 장점이 있다. 그래서 이 방법은 일반 문서와 초기 소스코드 유사도 분석 시스템 개발에 많이 이용되었다. 이와 같은 특성을 이용하면 메트릭 기반 유사도 분석은 규모가 큰 소프트웨어 시스템의 유사도 분석에 활용할 수 있다[17].

하지만 메트릭 기반 유사도 분석은 소스코드의 부분 유사도 분석에는 적합하지 않다는 단점이 있다. 그리고 소스코드에 있는 프로그램의 구조적 특징을 유사도 계산에 포함하기 어렵기 때문에 유사도 분석의 정확도가 떨어진다. 이러한 이유로 최근의 소프트웨어 유사도 분석에는 소스코드의 구조적 특징을 이용해 유사도를 계산하는 방법이 많이 이용되고 있다[18].

### 2.1.2 텍스트 기반 유사도 분석

텍스트 기반(text-based) 유사도 분석 방법은 소스코드 유사도 측정에서 많이 이용되는 방법 중 하나로 문자열 기반(string-based) 유사도 분석으로도 불린다. 이 방법은 프로그램의 소스코드를 하나의 긴 문자열 또는 여러 줄로 이루어진 문자열로 변환하여 이들의 유사도를 측정한다. 소스코드를 문자열로 변환한 다음 문자열들의 비교에는 해시 함수가 주로 이용된다.

텍스트 기반 유사도 분석은 두 소스코드로부터 생성된 문자열 집합에서 연속적으로 가장 길게 일치하는 부분 문자열을 유사 구간으로 반환한다. 대부분의 텍스트 기반 유사도 분석은 필터링(filtering), 정규화(normalization), 비교(comparison) 단계로 진행된다. 먼저, 문자열 비교에 앞서 소스코드에 대해 필터링(filtering)과 정규화(normalization)을 수행한다. 다음으로 정규화된 문자열들을 비교하여 유사도를 계산한다. 필터링과 정규화에서는 다음의 작업을 수행한다.

- 주석제거 - 소스코드에 포함된 모든 주석을 검사에서 제외한다.

- 공백제거 - 소스코드에 포함된 탭(tab)이나 개행문자를 비롯한 모든 공백문자를 제거한다.
- 정규화 - 유사도를 구하기 위해 소스코드의 키워드나 식별자(identifier)를 특정 문자로 치환한다.

Ducassee[19, 20]의 연구에서는 잘못된 양성(false positive)과 잘못된 음성(false negative) 판정 오류를 줄이기 위해 필터링을 수행한다. 필터링은 전처리 지시자(preprocessor directive) 또는 블록이나 문장 구분자(delimiter) 같이 프로그램에 공통적으로 포함되는 요소들을 제거함으로써 잘못된 양성 판정 오류를 줄이고, 주석 또는 레이아웃과 같은 중요하지 않은 요소들로 인해 유사도가 낮아지는 것을 줄여 잘못된 음성 판정 오류를 줄인다. 그림 2.1은 Ducasse의 방법을 이용해 C 언어 소스코드를 문자열로 변환한 것을 보여준다.

```
#include <stdio.h>

static int stat=0;

int main(void)
{
    int local=1;
    int *dynamic = (int*) malloc(sizeof(int),1);
```

(a) C언어로 작성된 소스코드

```
staticintstat=0
intllocal=1
int*dynamic=(int*)malloc(sizeof(int),1)
```

(b) 정규화된 문자열

그림 2.1: 필터링 및 정규화를 이용한 소스코드 변환

Ducassee는 정규화된 문자열들의 유사도를 비교하기 위해 dot-plot (scatter-plot) 방법을 사용하였다. Ducasse의 유사도는 다음과 같이 계산된다.

- (1) 두 프로그램의 소스코드  $P$ 와  $Q$ 로부터 정규화된 문자열 코드  $P_s$ 과  $Q_s$ 를 생성한다. 여기서  $P_s$ 와  $Q_s$ 는 문자열들의 해시코드 집합이다.

- (2) 각 문자열들의 해시코드를 비교하여 동일한 값의 문자열들을 추출한다.
- (3) 두 프로그램의 비교결과는 dot-plot으로 출력된다. dot-plot의  $X$ 축과  $Y$ 축은  $P_s$ 와  $Q_s$ 의 문자열을 나타내며, 문자열이 일치할 경우 ‘.’으로 표시한다. 그림 2.2는 일반적으로 검색되는 유사코드의 경우를 dot-plot으로 나타낸 것이다.

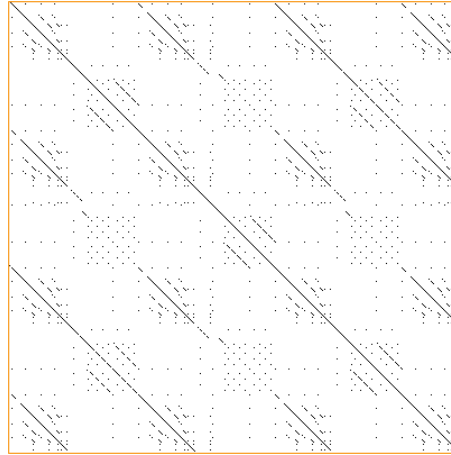


그림 2.2: Dot-plot을 이용한 agrep 소스코드 시각화

Johnson[12]의 연구도 텍스트를 비교하는 방법으로 유사 소스코드를 찾는다. Johnson의 방법에서는 슬라이딩 윈도우(sliding window)를 이용한다. 먼저, 소스코드를 고정된 길이(라인 수)로 이루어진 윈도우로 나눈다. 여기서 각각의 윈도우는 해시코드로 되어 있다. 다음으로 윈도우를 한 칸씩 이동시키면서 해시코드를 비교하여 일치하는 구간을 찾는다. Johnson은 Karp-Robin 알고리즘[21]을 이용해 윈도우들의 문자열을 비교하였다.

Manber[22]의 연구에서는 지문법과 체크섬(checksum) 방식을 이용하여 대용량의 파일 시스템에서 유사도가 매우 높은 파일들을 매우 빠른 시간안에 검색하는 sif를 개발하였다. sif는 대표 키워드를 이용하여 각 파일을 비교한다. sif는 일반적인 문자열 패턴 매칭 유틸리티인 diff에 비해 빠르게 많은 파일들을 비교하지만 근사적으로 유사도를 비교하기 때문에 유사도가 매우 높은 파일들을 찾을 때만 효율적이다.

이밖에도 Lee[23]의 연구에서는 *N-neighbor distance* 방법을 이용하여 많은 양의 소스코

드를 효율적으로 분석할 수 있는 SDD를 개발하였다. *N-neighbor distance*는 소스코드를 여러 개의 코드 조각(code fragments)으로 나누어 비교한다. 이 때 두 소스코드의 코드 조각들이  $N$ 개의 인덱스 이내로 이웃할 때는 코드가 일치하는 것으로 가정하는 방법이다. 이와 유사하게 Wettel[24]의 연구에서는 지문법과 dot-plot을 이용하여 코드 조각들의 유사도를 분석하였다. 이 방법에서도 dot-plot의 중간에 불일치 구간이 발생하더라도 이웃에 일치하는 코드 조각이 있으면 일치구간을 연결시킨다.

위의 연구들과 같이 텍스트 기반 유사도 분석 방법은 많은 부분에서 비슷한 구조로 유사도 검사가 이루어지며, 전처리 단계와 문자열의 비교 알고리즘이 중요한 요소이다. 전처리 단계는 소스코드에 포함된 불필요한 요소들을 제거하여 신뢰도를 향상시키고 문자열 비교 알고리즘에서는 근사매칭 방법과 같이 많은 파일에서 빠르게 비교하는 방법들이 주로 사용되었다.

텍스트 기반 유사도 분석은 프로그래밍 언어에 독립적이라는 장점이 있다. 특정 언어로 작성되어진 소스코드라도 중간표현인 정규화된 문자열로 치환되기 때문에 언어에 상관없이 코드를 비교할 수 있다[25]. 또한, 메트릭 기반 방법과 같이 시스템을 구현하기가 쉽다. 하지만 이 방법도 소스코드의 구조적 정보는 유사도 분석에 반영하지 못한다는 단점이 있다.

### 2.1.3 토큰 기반 유사도 분석

토큰 기반 유사도 분석은 프로그램의 소스코드를 토큰들로 구성된 서열(sequence of lexical token)로 변환한 다음 생물정보학이나 패턴인식 분야에서 사용되는 유사도 측정 알고리즘을 이용해 소스코드의 유사도를 분석하는 방법이다[26, 27]. 소스코드로부터 추출되는 토큰은 언어에 정의된 키워드나 연산자, 함수와 같은 특별한 식별자 등이 사용된다. 토큰 추출에는 주로 컴파일러의 어휘분석기(lexer)가 이용된다. 그리고 특별한 기능을 추가할 경우 구문분석기(parser)가 사용되는 경우도 있다. 소스코드로부터 토큰으로 구성된 서열이 만들어지면 다음으로 두 서열을 탐색하여 일치하는 부분 서열을 찾는다. 그림 2.3은 Gitchell[28]의

토큰 기반 유사도 측정 시스템인 Sim의 블록 다이어그램을 보여준다. 대부분의 토큰 기반 시스템은 Sim과 유사한 구조로 동작한다.

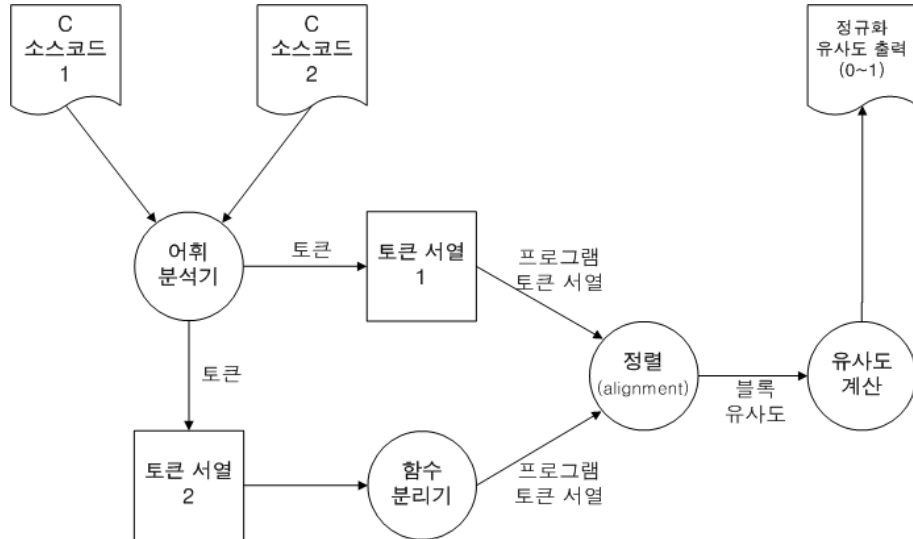


그림 2.3: 토큰 기반 유사도 분석 시스템 Sim의 블록 다이어그램

Baker[3, 29]의 연구에서는 소스코드의 각 라인으로부터 토큰을 생성하고, 서픽스 트리 알고리즘을 이용하여 서열들을 비교하였다. Baker는 DUP를 통해 완전일치(maximal exact match)와 부분일치(parameterized match)를 탐색하고 dot-plot를 이용해 시각화한다. Kamiya[30]의 연구에서는 토큰들 사이의 비교(token-by-token comparison)를 통해 대용량 소프트웨어 시스템의 소스코드 유사구간을 탐색하는 시스템인 CCFinderX를 개발하였다. Kamiya[30]의 방법도 DUP와 유사하게 먼저 렉서를 이용해 각 라인별로 토큰을 생성한 다음 모든 토큰을 하나의 서열로 결합한다. 이 때 렉서는 사전에 정의된 변환 규칙에 따라 토큰을 치환한다. 그림 2.4는 C++로 작성된 샘플코드 원본과 변환된 토큰 서열을 보여준다. 토큰으로 변환한 다음 두 서열의 비교는 서픽스 트리 기반 부분 문자열 매칭 방법을 사용한다. 또한, CCFinderX도 dot-plot을 이용해 분석결과를 시각화한다.

이밖에도 Basit[31]의 연구에서는 RTF(Repeated Tokens Finder)라는 서픽스 트리 대신 서

```

1| void print_lines(const set<string>& s) {
2|     int c = 0;
3|     set<string>::const_iterator i
4|     = s.begin();
5|     for (; i != s.end(); ++i) {
6|         cout << c << ", "
7|         << *i << endl;
8|         ++c;
9|     }
10| }
11| void print_table(const map<string, string>& m) {
12|     int c = 0;
13|     map<string, string>::const_iterator i
14|     = m.begin();
15|     for (; i != m.end(); ++i) {
16|         cout << c << ", "
17|         << i->first << " "
18|         << i->second << endl;
19|         ++c;
20|     }
21| }

```

(a) C++ 샘플코드

```

1| void print_lines ( const set & s ) {
2| int c = 0 ;
3| Const_iterator I
4| = s . begin ( ) ;
5| for ( ; i != s . end ( ) ; ++ i ) {
6| cout << c << ", "
7| << * I << endl ;
8| ++ c ;
9| }
10| }
11| void print_table ( const map & m ) {
12| int c = 0 ;
13| Const_iterator I
14| = m . begin ( ) ;
15| for ( ; i != m . end ( ) ; ++ i ) {
16| cout << c << ", "
17| << i -> first << " "
18| << i -> second << endl ;
19| ++ c ;
20| }
21| }

```

(b) 변환된 토큰

그림 2.4: C++로 작성된 샘플코드와 변환된 토큰 서열

픽스 배열을 이용해 메모리 효율을 높인 토큰 기반 검사 틀을 개발했다. RTF는 사용자에게 토큰을 선택할 수 있게 하여 검사효율을 높였다. 예를 들어, C++에서 접근제한자(private, protected, public)들을 같은 토큰으로 정규화한다. Li[32, 33]는 빈도 부분서열 마이닝 기법을 이용해 소프트웨어 시스템에서 복사·붙이기 모듈을 찾는다. Li의 연구에서 개발한 CP-Miner는 *CloSpan(Closed Sequential Pattern Mining)*[34] 알고리즘을 이용해 소스코드로부터 토큰 서열들을 추출하여 데이터베이스를 구축하고 ‘복사·붙이기’로 인해 빈번하게 나타나는 서열들을 찾는다. 또한, 토큰화(tokenization)으로 인해 발생하는 잘못된 양성 판정 오류를 줄이기 위해 *ConflictRatio* 메트릭을 비롯해 여러가지 기법들을 적용한다.

토큰 기반 유사도 분석은 소스코드 유사도 분석에 빈번하게 사용되고 있으며, 실제 표절검사 시스템이나 리팩토링 시스템에서 적용되었다[35, 36, 37, 38, 39]. 토큰을 이용한 유사도 분석은 텍스트 기반 분석 방법보다 더욱 견고하고 유연하다. 이 방법은 소스코드로부터 토큰만을 추출하여 유사도를 측정하기 때문에 코드 형식(code formatting) 변환이나 식별자 수정, 타입 변환과 같은 약간의 수정에 대해 유연하게 분석할 수 있다.

#### 2.1.4 트리 기반 유사도 분석

트리 기반 유사도 분석은 소스코드로부터 구축한 파스트리(parse tree) 또는 추상 구문 트리(abstract syntax tree)를 이용하는 방법이다. 파스트리는 소스코드의 구문분석으로 생성되는 트리 기반 자료구조로 소스코드에 포함된 모든 구문 정보를 트리에 저장한다. 그리고 추상 구문 트리는 파스 트리 보다는 축약된 정보로 중간표현을 생성하는데 필요한 정보들로만 구축한 트리이다. 트리의 생성은 주로 컴파일러 전단부(front-end) 모듈을 이용한다.

트리 기반 유사도 분석 과정은 보통 두 단계로 진행된다. 첫 번째 단계에서는 두 프로그램의 소스코드를 입력받아 어휘분석과 구문분석을 거쳐서 중간 표현인 트리를 생성한다. 트리에에는 소스코드의 구조적 정보들이 포함되어 있다[40]. 분석의 후단부에서는 중간 표현으로 생성된 트리를 이용해 두 프로그램의 유사도를 분석한다. 여기서는 부분 트리 일치 알고리즘(subtree matching algorithm)이나 트리의 구조적 속성 정보를 이용하는 구조적 메

트릭(structural metric)이 이용된다. 그림 2.5은 파스트리를 이용해 소스코드의 유사도를 측정하는 시스템인 DECKARD[41]의 유사도 측정에 대한 블록 다이어그램을 보여준다.

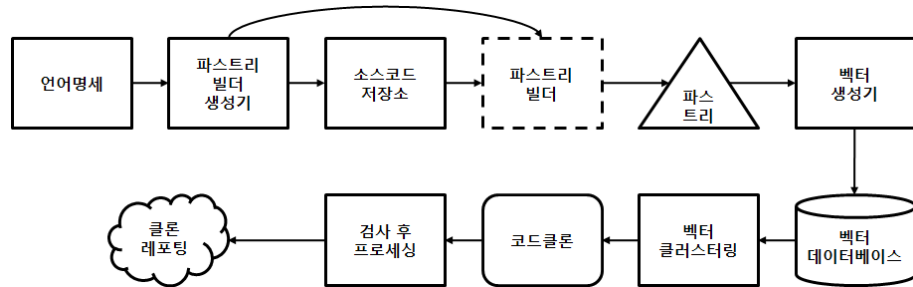


그림 2.5: 트리 기반 유사도 분석 시스템 DECKARD의 블록 다이어그램

파스트리와 함께 AST도 트리기반 분석에 많이 이용된다. GCC와 ANTLR의 파서에 의해 생성된 AST는 소스코드의 구조적 정보를 저장하고 있다. 예를 들어, C++의 경우 함수와 제어문(if, switch) 반복문(while, for) 등의 코드 블록에 대한 정보를 유지한다. AST가 코드 블록 정보를 유지하므로 코드 블록 간의 유사도 검사에도 효율적으로 사용할 수 있으며, 함수의 위치 수정이나 분할, 병합에 대해서도 견고하다. 또한, AST를 생성할 때, 주석과 구분자 같은 중요하지 않은 정보들은 제거되기 때문에 소스코드 수정에 대해 유연하게 분석할 수 있다.

트리를 이용한 유사도는 AST의 부분 트리 비교를 통해 계산된다. Son[40]의 연구에서는 파스트리 커널(parse tree kernel)의 모든 부분 트리를 벡터로 하여 비교한다. 그림 2.6은 간단한 파스트리에 대한 부분 트리의 예를 보여준다.

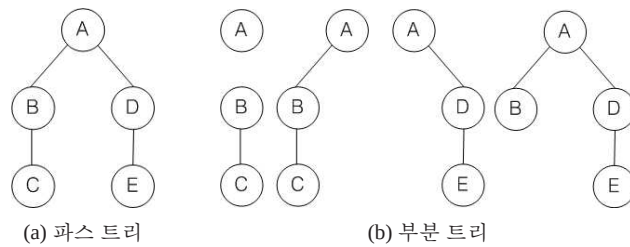


그림 2.6: 파스트리와 파스트리에 나타날 수 있는 부분 트리 예시

파스 트리 커널  $T$ 의 부분 트리로 이루어진 벡터  $V_T$ 는 다음과 같이 정의할 수 있다.

$$V_T = \{\#subtree_1(T), \#subtree_2(T), \dots, \#subtree_n(T)\}$$

여기서  $\#subtree_i(T)$ 는 파스 트리  $T$ 에서  $subtree_i$ 의 빈도를 나타낸다. 두 파스 트리  $T_1$ 과  $T_2$ 의 내적은 다음과 같이 계산된다.

$$\begin{aligned} \langle V_{T_1}, V_{T_2} \rangle &= \sum_i \#subtree_i(T_1) \cdot \#subtree_i(T_2) \\ &= \sum_i \left( \sum_{n_1 \in N_{T_1}} I_{subtree_i}(n_1) \right) \cdot \left( \sum_{n_2 \in N_{T_2}} I_{subtree_i}(n_2) \right) \\ &= \sum_{n_1 \in N_{T_1}} \sum_{n_2 \in N_{T_2}} C(n_1, n_2) \end{aligned} \quad (2.2)$$

위의 식에서  $N_{T_1}$ 과  $N_{T_2}$ 는 파스 트리  $T_1$ 과  $T_2$ 의 모든 노드를 나타내며,  $I_{subtree_i}(n_1)$  함수는 노드  $n_1$ 이 최상위 노드인 서브트리가 있으면 1, 없으면 0을 반환한다. 그리고  $C(n_1, n_2)$  다음과 같이 정의된다.

$$C(n_1, n_2) = \sum_i I_{f_i}(n_1) \cdot I_{f_i}(n_2) \quad (2.3)$$

$C(n_1, n_2)$  함수는 Collins와 Duffy의 방법에 의해 재귀적으로 계산된다. 마지막으로 두 프로그램의 일치 정도는 벡터  $V_T$ 의 내적 값을 정규화시켜 0 ~ 1 사이의 유사도로 계산된다.

앞서 기술한 Son의 연구처럼 부분 트리를 직접 비교하는 방법도 사용되지만, 복잡도가 매우 높아서 휴리스틱한 방법이 사용되기도 한다. Baxter[42]의 연구에 따르면  $N$ 개의 AST 노드에 대한 부분 트리 비교에 대한 복잡도는  $O(N^3)$ 이며, 경험적으로  $M$  라인의 소스코드의 AST노드는  $N = 10XM$ 개 정도로 모두 비교한다면 복잡도가  $O(N^4)$  정도가 된다. 그러므로 파스 트리를 이용한 다른 연구들에서는 다른 방법을 이용해 부분 트리를 비교한다[43, 44, 45, 46]. Baxter는 해시 버킷(hash bucket)을 이용해 부분 트리를 비교한다. Baxter는 이를 이용해 복잡도를  $O(N)$ 으로 줄였다. 그리고 Baxter의 해시 함수는 변수나 함수의 이름 변경 등으로 발생하는 부분 불일치(near-miss clone)를 고려한다. Baxter의 유사도는 다음과

같이 계산된다.

$$Similarity(T_1, T_2) = \frac{2 \times S}{2 \times S + L + R} \quad (2.4)$$

위의 식에서  $S$ 는 트리의 공유노드의 수를 나타내고,  $L$ 은  $T_1$ 의 불일치 노드 수를 나타낸다. 마지막으로  $R$ 은  $T_2$ 의 불일치 노드 수를 나타낸다.

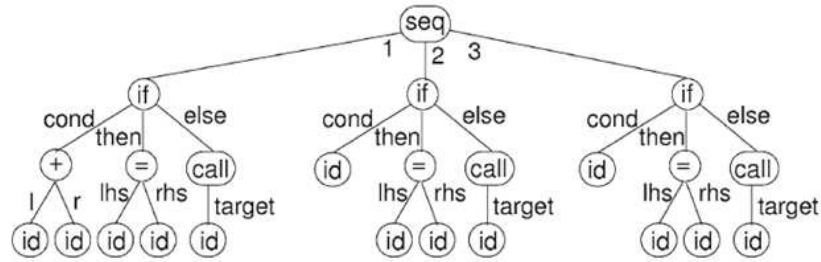
Koschke와 Falke의 연구[44, 45]에서는 소스코드에서 유사한 부분을 찾기 위해 서픽스 트리를 이용해 AST를 비교하였다. 이들의 연구에서는 AST를 직렬화(serialization)하여 서픽스 트리를 구성하고 서픽스 트리 비교 알고리즘을 이용해 유사 코드를 찾는다. Koschke의 유사도 분석은 다음과 같이 계산된다.

1. 프로그램을 파싱하여 AST를 생성한다.
2. AST를 직렬화한다.
3. 직렬화된 AST를 이용해 서픽스 트리를 생성한다.
4. 서픽스 트리 비교를 통해 복사된 코드 영역을 탐색한다.

Koschke의 유사도 분석은 서픽스 트리 비교 알고리즘을 사용하기 위해 AST 노드들을 직렬화한다. AST 직렬화는 각 노드들을 전위순회(preorder) 방식으로 방문하면서 직렬화한다. 그리고 각 노드들은 노드에 대한 id와 서브트리의 노드 개수를 기록한다. 그림 2.7는 C언어로 작성된 소스드를 나타내고, 그림 2.8은 소스코드에 대한 AST와 직렬화된 AST 노드를 보여준다.

```
if (x + y) a = i; else foo();  
if (p) a = j; else foo();  
if (q) z = k; else bar();
```

그림 2.7: C언어로 작성된 소스코드 샘플



(a) AST 노드

```
seq23
  if8 +2 id0 id0 =2 id0 id0 call1 id0
  if6 id0 =2 id0 id0 call1 id0
  if6 id0 =2 id0 id0 call1 id0
```

(b) AST 직렬화

그림 2.8: C 소스코드에 대한 AST와 AST 직렬화

트리 기반 유사도 분석 연구들은 전체 프로그램 소스코드를 파스트리나 AST로 재배열하여 패턴 매칭이나 트리의 순회법(traversal)을 이용하여 어떤 선형의 순서를 만들어 내거나 구문분석 단계에서 대략적으로 실행하여 수행되는 함수들의 순서에 따라 키워드들을 추출한다. 여기서 추출된 키워드들은 토큰으로 간주하여 토큰 기반 유사도 계산법을 사용해 비교할 수 있다. AST를 이용한 유사도 분석의 장점은 소스코드의 구조적 정보를 유사도 비교에 반영할 수 있다는 것이다. 따라서 메트릭이나 텍스트, 토큰에 비해 소스코드 수정에 대해 더욱 유연한 분석이 가능하다. 또한, 다른 유사도 분석 방법과 결합하여 사용하면 더욱 정밀한 분석이 가능하다.

## 2.1.5 유사도 분석 시각화

시각화 기반(visualization-based) 유사도 분석은 데이터 마이닝 또는 생물 정보학 등에서 사용되는 데이터 및 정보 시각화 기법들을 사용해 유사도 분석 결과를 시각화하는 방법이다. 시각화 방법은 주로 유사도 분석 결과를 효율적으로 보여주는데 사용된다. 사용자는 시각

화 결과를 통해 빠른 시간에 프로그램 전체 유사 모듈들을 파악할 수 있다. 시각화 방법은 소스코드 뿐만 아니라 문서들 간의 유사도 비교에도 빈번하게 이용되고 있다[47, 48, 49, 50, 51]. 시각화는 분석된 결과를 이용하기 때문에 검사 대상에 관계 없이 사용할 수 있다.

Dot-plot 방법은 표절검사 및 소프트웨어 유사 구간 탐색 연구에서 가장 빈번하게 사용되는 방법이다[52]. Dot-plot은  $x$ 축과  $y$ 축의 2차원 평면위에 점(dot)을 이용해 시각화한다. Dot-plot의  $x$ 축과  $y$ 축은 소스코드의 각 라인을 의미하고, 두 소스코드에서 일치하는 부분을 점으로 표시한다. 따라서 두 소스코드의 특정 부분이 연속적으로 일치할 경우 점들이 이어져 선으로 표시된다. 만약 점들이 왼쪽에서 오른쪽으로 대각선으로 이어져 나타나면, 두 소스코드는 완전히 일치(복사)된 것을 의미한다. 그림 2.9은 Rysselberghe의 연구에서 Dot-plot으로 리팩토링 코드를 분석한 예를 보여준다.

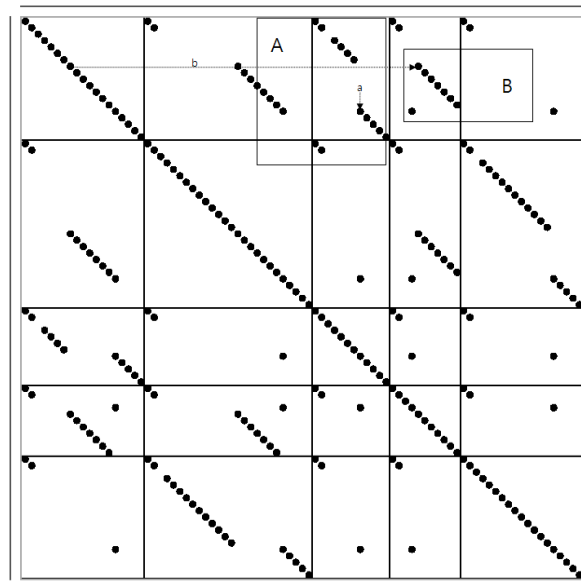


그림 2.9: Dot-plot을 이용한 소스코드 유사도 시각화 분석: A 지역은 코드가 삭제된 것을 나타내고, B 지역은 코드(클래스)가 새롭게 추가된 것을 나타냄.

Ribler[51]의 연구에서는 CCP(*Composite Categorical Patterngram*)을 이용해 시각화하여 소스코드의 표절을 탐색하는 방법을 제안했다. CCP는 여러 개의 텍스트 파일에서 공통적

으로 사용된 문자 순서열들을 나타내기 위한 시각화 방법이다. 특정한 길이(k-length)를 가지는 문자열(n-gram; n그램)을 정의한 후, 텍스트 파일을 하나의 긴 문자열로 보고 텍스트 파일들에 n그램이 나타난 횟수를 이용해 공통된 부분을 시각적으로 표현하는 방법이다. 그림 2.10은 “Toy0: This is a test.”와 “Toy1: Oh yes. This is a test too.”, “Toy2: Toy2 has little in common with the other two. This is common.”으로 구성된 세개의 문자열에 대한 일반적인 패턴그램과 CCP로 나타낸 예를 보여준다.

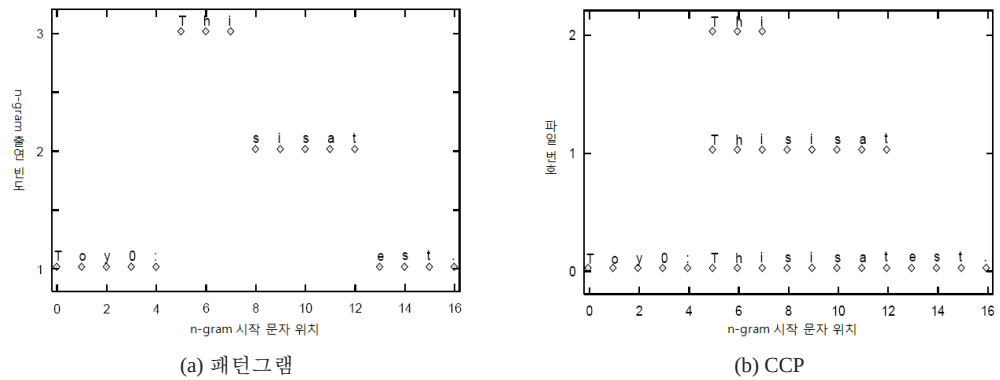


그림 2.10: 문자열에 대한 패턴그램과 CCP 시각화

그림 2.10에서처럼 CCP는 각 파일에 대해 Y축의 좌표를 할당하고 기본 파일과 비교하여 동일한 n그램이 나타나면 그래프로 표시한다. Ribler의 표절검사 연구에서는 CCP을 이용해 하나의 기본 소스코드에 대하여 소스코드 집합의 나머지 파일들 중에서 표절된 프로그램들을 찾는다. 그림 2.11은 Ribler가 제안한 방법으로 표절검사를 한 결과를 보여준다. 그림 2.11에서 푸른색은 비교대상이 되는 기본파일(submission45)을 의미하고 붉은색은 이와 유사하게 작성된 프로그램(submission46)을 나타낸다. 그림 2.11을 살펴보면 submission46 프로그램은 submission45 프로그램의 클론임을 알 수 있다.

이밖에도 Johnson[49]는 메트릭 기반 유사도 분석 결과를 Hasse 다이어그램을 이용해 유사 코드 클러스터를 시각화하였다. 이 연구에서는 GNU gcc 컴파일러 소스코드를 클러스터링하여 Hasse diagram으로 시각화하였다. 그리고 Mala[50]와 Benjamin[47]은 일반 문서에 대

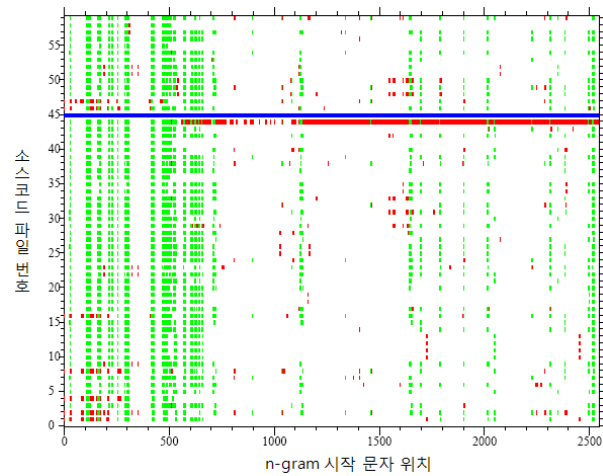


그림 2.11: CCP를 이용한 프로그램 유사도 탐색 시각화 결과

한 시각화를 수행하였다. 앞서 언급했듯이 일반 문서의 시각화 기법은 프로그램 소스코드에도 적용 가능하다. Mala의 시각화 방법은 복사와 재배치, 치환의 3가지 기준에 대해 유사 구간을 찾고 3차원 지형표면(3D terrain surface)으로 시각화한다. Benjamin은 GIMP의 GUI 툴킷을 사용해 시각화하였으며, 문서 유사도는 *CSTW(Customized Term Weighting Scheme)*을 이용해 문서들 사이의 유사도를 계산하고 *Kohoneon Map*[53]을 이용해 시각화한다. 마지막으로 Jiang은 소프트웨어 시스템의 모듈들 사이의 연결성과 응집성을 분석하고 무게 기반 그래프 배치(force-based graph layout)를 이용하여 시각화하였다. 여기서 그래프의 에지 무게는 복사된 코드의 라인 수를 이용하였다. 그림 2.12은 네 종류의 시각화 기반 유사도 분석 시스템을 보여준다.

시각화 기반 유사도 분석은 결과를 축약하여 보여준다. Dot-plot 기반 시각화에서부터 3차원 시각화까지 방법은 다르더라도 한 화면에 모든 유사도 정보를 보여주고 있다. 이 방법들을 사용하면 빠른 시간에 사용자는 유사 코드 모듈들을 탐색할 수 있다.

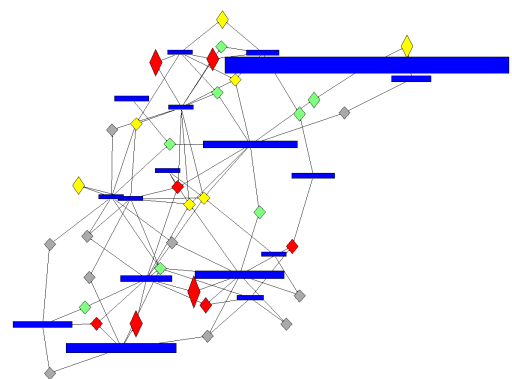
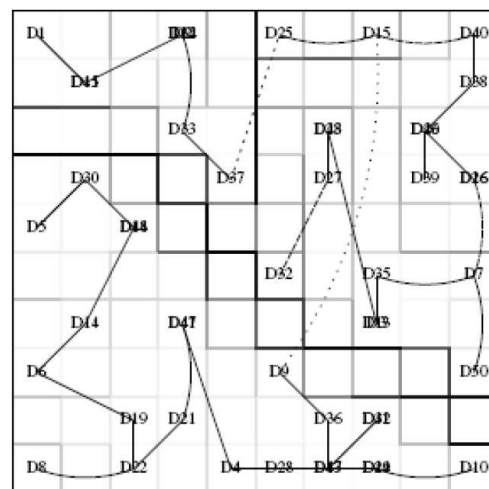
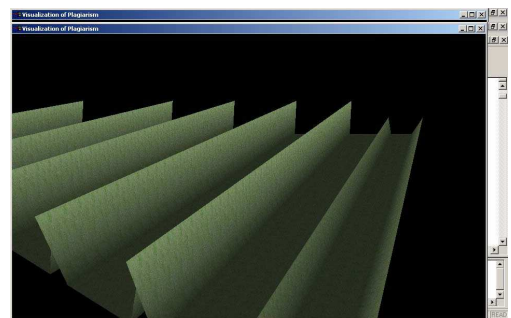
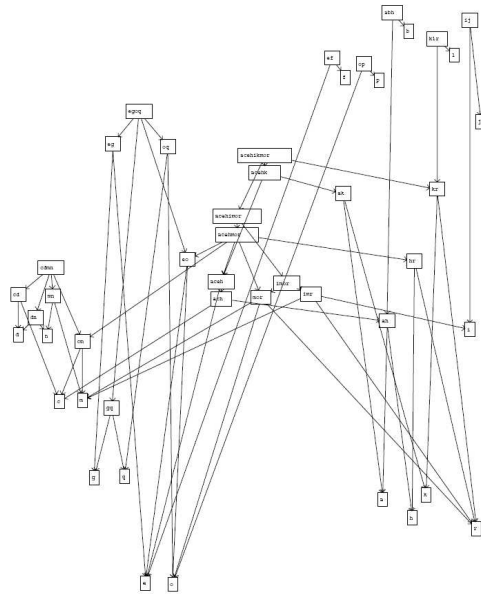


그림 2.12: 소스코드 유사도 분석 시각화 방법

### 2.1.6 다른 유사도 분석 방법들

본 절에서는 소스코드 유사도 분석에 대한 다른 방법들에 대해 살펴본다. 먼저, AST나 파스트리를 이용하는 트리 기반 분석 방법보다 한단계 더 발전한 프로그램 의존성 그래프(Program Dependency Graph, PDG) 방법이 있다[54, 55, 56, 57]. PDG 기반 유사도 분석은 컴파일러 전단부 모듈인 어휘 분석과 구문 분석에 이어 의미 분석(semantic analysis)까지 수행한다. PDG 기반 유사도 분석은 소스코드를 입력으로 하여 AST를 생성한 다음 의미 분석을 거쳐 프로그램의 호출 관계나 데이터 흐름(data flow) 등을 고려하여 그래프(graph)의 노드와 에지로 변환한다. 두 프로그램의 유사도 계산은 동형 서브그래프 비교 알고리즘(isomorphic subgraph matching algorithm)[58]을 이용하여 유사한 부분 그래프를 찾고 정규화를 거쳐 유사도를 계산한다.

Pham[57]의 연구에서는 Matlab의 Simulink 모델의 표절 검사를 위한 ModelCD를 개발하였다. ModelCD는 그래프 기반 유사도 분석 알고리즘을 이용해 Simulink 모델의 완전일치와 부분일치를 측정한다. Pham의 방법은 생성(generating), 그룹핑(grouping), 필터링(filtering)의 세 단계로 구성된다. 생성단계에서는 모델을 세분화하여 부분 그래프들의 에지 수를 이용하여 후보군을 만든다. 그룹핑 단계에서는 정규화된 라벨(canonical label)을 이용해 완전 일치 그래프와 부분 일치 그래프로 나눈다. 마지막으로 필터링 단계를 거쳐 유사 코드 구간을 시각화 한다.

Komondoor[55, 59]의 연구에서는 프로그램 슬라이싱(program slicing)을 이용하여 프로그램의 유사 PDG 그래프를 찾는다. Komondoor의 방법은 슬라이싱 기반 분석이라고도 한다. Komondoor의 PDG 기반 검사 방법은 프로그램 소스코드를 한 문장씩 나누어 문장을 노드로 하여, 제어문과 데이터의 의존성을 분석하여 PDG를 구축한다. 이때, 함수나 변수에 대한 선언 정보는 포함하지 않는다. 다음으로 동형 서브그래프 비교 알고리즘을 이용해 일치하는 하위 그래프를 찾는다.

마지막으로 Gabel[54]의 연구에서는 의미가 유사한 소스코드를 찾기 위해 PDG 기반 분

석 방법을 사용하였다. 이 연구에서는 규모가 큰 소스코드에 적용하기 위해 복잡한 PDG 분석 알고리즘 대신 PDG 구조를 트리 기반의 벡터로 변환하여 계산 문제를 단순화시켰다. 이 방법은 단순 구문 위주의 비교 방법보다 효율적이다.

PDG를 이용하는 방법은 단순 구문 분석에 속하는 트리 기반 방법들 보다 의미 분석을 한다는 점에서 더욱 발전된 형태이다. 그러므로 PDG 기반 유사도 분석은 소스코드의 추가나 삭제, 재배치를 비롯해 함수 분할, 제어흐름 변경 등의 수정에는 견고하다. 하지만 부분 그래프 검색 문제가 복잡하여 규모가 큰 소프트웨어의 유사도 분석에는 적합하지 않다.

지금까지 소개한 방법들과 전혀 다른 방법으로 문서의 세부구조를 고려하지 않고 문서나 프로그램의 Kolmogorov 복잡도와 정보이론(information theory)에 기초한 방법으로 전체 문서의 유사도를 비교하는 방법이 있다[60]. 이 방법은 주어진 스트링의 Kolmogorov 복잡도를 해당 스트링을 출력할 수 있는 최소크기의 프로그램(추상적 의미)의 크기로 정의된다. 하지만 실제 이 프로그램의 크기는 구현하는 것이 불가능하므로 일반적으로는 해당 스트링을 RSA 방법으로 압축했을 때 그 크기의 정도로 대신 사용된다. 두 문서 A와 B가 유사할 경우 A와 B를 합쳐서(concatenation) 구성된  $A \cdot B$  화일을 압축한 크기와 A혹은 B만을 압축한 크기를 비교하면 두 문서의 유사도를 알 수 있다. RSA 알고리즘은 화일의 앞 부분을 이용하여 동적 사전을 만들어 점진적으로 압축하므로 A와 B가 유사할수록 뒤에 붙어있는 B를 압축하는 효율은 좋아진다.

이 방식은 이론적으로 볼 때 매우 효과적인 유사도 탐색 방법이지만 한 가지 단점은 부분적인 유사도 분석에 대해서는 큰 도움을 주지 못한다는 것이다. 다시 말해서 주어진 두 문서가 정보이론적으로 유사한지를 밝히는데에는 도움을 주지만 어떤 일부분이 서로 일치하는지에 대해서는 아무런 정보를 주지 못한다. 이 방법이 유용한 경우는 관련연구자들도 지적인 바와 같이 초대규모의 전유전체(whole genome)의 상동성(homology)를 계산하여 진화적 거리(evolutionary distance)를 연구하는데는 도움을 줄 수는 있지만 이에 비해서 그 크기가 작은 200 라인 안팎의 작은 단위의 소스코드의 유사도 측정에는 적합하지 않다.

마지막으로 Arwin[61]의 연구에서는 GNU gcc의 중간표현인 RTL을 이용하여 서로 언

어가 다르게 작성되어진 두 프로그램의 유사도를 분석한다. RTL 코드는 언어에 관계없이 컴파일러의 전단부 과정에서 동일한 동작을 하는 코드는 유사하게 생성된다는 점을 이용하여 언어가 다른 두 프로그램의 유사도 분석에 이용하였다. 이밖에도 Ji[62]와 Lim[63]등의 연구자들은 자바 소스코드 대신 바이트코드를 이용하여 역으로 자바 소스코드의 유사도를 분석하는 방법을 제안하였다. Ji와 Lim의 연구에서는 바이트코드 명령어를 토큰으로 하여 서열로 만든 다음 DNA 유전체 비교 알고리즘을 이용해 유사도를 측정하였다. 또한, Baker의 연구에서는 diff와 dup, siff 유틸리티를 결합하여 바이트코드의 유사도를 분석었다. 바이트코드 분석은 소스코드를 직접 사용하지 않고 소스코드의 유사도를 분석할 수 있다는 점에서 보안에 장점이 있다.

지금까지 일반 문서와 소스코드를 대상으로 한 유사도 분석 방법들을 살펴보았다. 일반 문서의 유사도 분석에는 문서의 속성계수를 이용하는 지문법이 많이 사용되었으며, 소스코드의 유사도 분석은 구조적 정보를 유사도 분석에 이용하는 토큰과 트리, 그래프를 이용하는 방법들이 주를 이루었다. 이처럼 소스코드의 경우 작성된 언어에 따른 구조적 정보 및 의미들이 소스코드에 포함되어 있기 때문에 이들에 대한 속성들을 유사도 계산에 반영하여 계산의 정확성을 높인다.

다음 절에서는 공개소스 및 상용 시스템으로 활발하게 이용되고 있는 유사도 분석 시스템에 대해 살펴볼 것이다. 이들 시스템들은 문서 및 소스코드 기반 표절검사 시스템이나 소프트웨어 역공학을 위한 분석이다.

## 2.2 유사도 분석 시스템

앞 절에서 설명한 유사도 분석 기법들은 실제로 표절 검사 시스템이나 리팩토링을 위한 클론 탐색 시스템 개발에 사용되고 있다. 유사도 분석 시스템은 그 대상에 따라 문서 유사도 분석 시스템과 프로그램 유사도 분석 시스템으로 나눌 수 있다. 하지만, 많은 시스템들에서 일반 문서와 프로그램의 유사도 분석을 동시에 지원하고 있는 경우도 많다. 이것은 프로그

램 유사도 분석 기법들이 일반 문서에도 응용될 수 있음을 보여준다. 본 절에서는 텍스트 파일이나 DOC, HWP, PDF 파일 등의 문서 파일을 입력으로 하여 두 문서 사이의 유사도를 분석하는 문서 유사도 분석 시스템과 C, C++, 자바와 같은 프로그램 소스코드를 입력으로 하여 유사도를 계산하는 소스코드 유사도 분석 시스템들에 대해 살펴본다. 이들 시스템들은 주로 독립적(standalone) 형태로 동작하거나 웹(web) 또는 서버 클라이언트 방식으로 동작한다.

### 2.2.1 문서 유사도 분석 시스템

문서 유사도 분석 시스템은 주로 표절 검사 시스템으로 이용된다. 문서 기반 표절검사 시스템은 두 문서 사이에 표절되었거나 유사한 문장이나 문단들을 찾아준다. 문서 표절 검사 시스템은 주로 지문법이나 문자열 비교 방법이 많이 사용된다. 문서 표절 검사에 대한 연구는 오래전부터 진행되었으며 많은 시스템들이 개발되어 사용되고 있다.

- Diff

diff는 파일들 사이의 공통 부분을 찾는 가장 오래된 도구 중 하나이다. diff는 두 파일 사이의 차이점을 찾기위해 가장 긴 공통 부분 서열 일치 알고리즘(longest common subsequence matching algorithm)을 사용한다. diff는 두 파일을 입력으로 하여 문장의 입력이나 삭제가 있을 경우 최소 길이 수정 스크립트(minimal length edit script)를 통해 두 파일의 차이점을 보여준다. diff는 여러 가지 버전이 개발되었지만 Myer[64]의 알고리즘을 기반으로 GNU에서 구현한 diff가 가장 많이 이용되고 있다. 그리고 현재에는 리눅스의 gdiff나 윈도우의 windiff와 같이 GUI를 지원하는 diff가 많이 이용되고 있다.

diff는 사용하기가 쉬워 수정이 작은 파일들의 비교에는 이용하기 쉬워 많이 사용되고 있지만, 대용량 문서에서 문장들이 많이 수정되었을 경우에는 속도가 느려져서 적합하지 않다. 또한, 문자열을 가장 길게 연속적으로 일치되는 부분을 찾기 때문에 문장들의 순서를 이동시켰을 경우 효율이 떨어진다.

- SCAM

SCAM[65]은 스탠포드 전자 도서관 프로젝트(Stanford Digital Library Project)의 일환으로 개발된 표절 검사 시스템인 COPS[66]의 단점을 개선한 전자 문서 표절 검사 시스템이다. COPS의 방법은 등록할 문서를 일련의 문장 단위로 쪼개어 데이터베이스 서버에 저장한 후 질의(query) 문서를 동일한 방법으로 나누어 등록된 문서들과 비교하여 중복(overlap)되는 부분을 찾는다. 만약 질의 문서가 일정 유사도 이상되면 사용자에게 알려준다.

SCAM은 COPS 방법의 문제점을 개선하여 문서를 문장 단위가 아닌 단어 단위로 나누었다. 문서를 단어로 나누어 비교하면, 지역성(locality)이 향상되어 부분 표절을 더욱 정확하게 찾을 수 있다. SCAM은 문서에 사용된 단어의 빈도수를 벡터로 나타내고 이들 벡터들 간의 코사인 유사도를 이용하여 문서간의 유사성을 검사한다. SCAM 시스템은 등록된 문서

의 단어들을 반전된 인덱스(inverted index) 구조로 저장한다. 문서들은 단어의 조각(chunk) 단위로 나누어지며 등록시간에 따라 인덱스가 생성된다. 조각 하나는 문서번호와 빈도수로 구성된다. 그림 2.13의 구조를 보여준다.

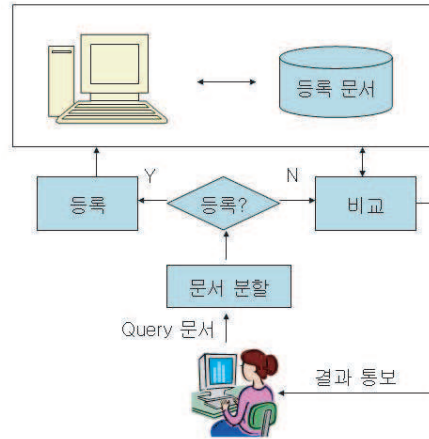


그림 2.13: SCAM: 쿼리를 이용한 문서 표절검사 시스템 구조

SCAM에서는 단어의 빈도수를 코사인 벡터 함수를 이용하여 유사도를 측정한다. 다음 은 문서  $R$ 에 대한 쿼리  $Q$ 의 코사인 유사도 측정 함수  $sim(R, Q)$ 를 보여준다.

$$sim(R, Q) = \frac{\sum_{i=1}^N \alpha_i^2 \times F_i(R) \times F_i^2(Q)}{\sqrt{\sum_{i=1}^N \alpha_i^2 \times F_i^2(R) \times \sum_{i=1}^N \alpha_i^2 F_i^2(Q)}} \quad (2.5)$$

여기서  $\alpha_i$ 는  $i$ 번째 조각이 발견된 횟수를 나타낸다.  $sim$  함수는 0 ~ 1 사이의 값으로 계산되며, 1에 가까울수록 유사도가 높음을 의미한다.

SCAM은 문서의 전체 유사도에 근거한 표절 탐지 능력이 뛰어나다. 또한, 데이터베이스화 하여 문서들을 관리하고 쿼리 문서를 비교하기 때문에 검사 속도 또한 빠르다. 하지만 전체 유사도를 기준으로 문서의 유사성을 판단하기 때문에 부분 표절 탐색에는 단점이 있다.

- DEVAC

한글 말뭉치를 이용한 문서표절 시스템인 DEVAC(Document EVolution Analyzing Center)은 한글로 작성된 문서들에 대해서 표절을 탐색한다[67, 68]. DEVAC 시스템은 서버-클라이언트 구조로 동작한다. 먼저 DEVAC 센터에서 클라이언트를 다운받아 설치한 후 클라이언트를 통해 표절검사를 원하는 문서 집합을 선택하면 이를 센터로 보내어 검사를 하고 검사 결과를 받아 시각화한다. DEVAC 시스템은 다음 주소에서 내려받아 사용할 수 있다. 그림 2.14은 DEVAC은 센터를 보여준다.

DEVAC : <http://devac.cs.pusan.ac.kr:8080>



그림 2.14: DEVAC 표절 검사 시스템 센터

DEVAC에서는 한글 문서를 입력으로 받아 사전 구조로 변환한 후 이를 통해 공통 앵커를 탐색한다. 앵커는  $k$ -mer로 된 한글 어절( $k$ 는 어절의 길이)로 이를 사전 구조로 된 엔트리(entry)로 사용하고 어절의 위치를 레퍼런스(reference)로 사용한다. 앵커의 길이가 짧으면 표절된 문장들을 찾는 정확성이 높아진다. 하지만 앵커의 길이가 짧을 수록 계산량이 늘거나 검사 비용이 증가한다. 이 때문에 DEVAC 시스템에서는 실험을 통하여 정확성을 떨어

앵커를 이용해 1차적으로 전체 문장을 필터링했으면, 다음으로 표절이 의심되는 문장들 사이의 유사도를 분석한다. 유사도 분석에는 지역정렬(local alignment)을 사용한다. DEVAC에서 사용하는 지역정렬은 기본 갭 감점(general gap penalty)가 적용된 지역정렬 매트릭스를 한글 표절 검사에 적합하도록 수정한 아핀 갭 감점(affine gap penalty)[69] 지역정렬을 사용한다. 아핀 갭 지역정렬은 생물정보학에서 DNA 서열 분석에서 연속된 갭 추가나 삭제에 대한 점수를 수정한 모델이다. 그림 2.15는 DEVAC 시스템의 클라이언트 프로그램을 이용한 문서 표절 검사 결과를 보여준다.

그림 2.15: DEVAC 표절 검사 클라이언트 프로그램 검사결과

32

보를 보여주고, 하단에는 시스템 로그를 출력한다.

## 2.2.2 소스코드 유사도 분석 시스템

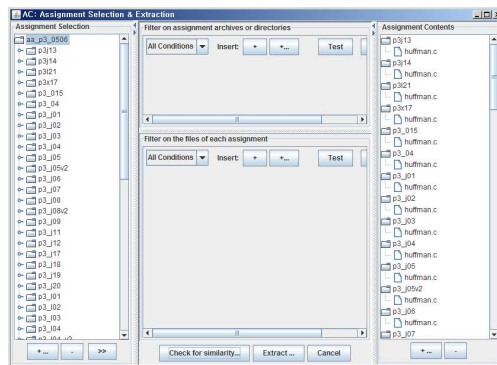
프로그램 유사도 분석에 대한 연구와 함께 여러 가지 기법들을 적용한 자동화된 시스템들이 개발되었다[48, 28, 37, 39]. 이들 시스템들은 소스코드 표절 검사 및 소프트웨어 리팩토링을 위한 검사에 이용되고 있다. 본 절에서는 소스코드 유사도 분석을 기반으로 한 표절 검사 및 리팩토링 지원을 위한 대표적인 시스템들에 대해 살펴본다. 본 절에서 설명하는 시스템들은 2.1절에서 설명한 구조적 분석 기법들이 적용되었다.

- AC

AC 시스템[48]은 마드리드 대학에서 개발된 시스템으로 표절검사를 위한 그래프 및 히스토그램(histogram) 시각화를 지원한다. AC는 자바로 작성되었으며, 오픈소스로 소스코드가 공개되어 있다. AC는 테이블과 그래프, 히스토그램으로 구성된 세 종류의 시각화 인터페이스를 제공한다. 유사도 검사 결과인 거리 매트릭스(distance matrix)에 따라 시각화 결과를 보여준다. 테이블을 이용한 시각화는 소스코드 쌍과 이들 사이의 유사도를 보여준다. 테이블에는 유사도를 기준으로 정렬되어 보여지기 때문에 유사도가 높은 순서쌍들을 쉽게 파악할 수 있다.

그래프 시각화는 노드와 에지로 구성된 그래프를 이용해 일정 유사도 이상의 거리 매트릭스로 구성된 프로그램 쌍들의 관계를 전체적으로 보여준다. 그래프 시각화에서 노드는 소스코드의 id를 나타내고, 색상으로 표시된 에지는 두 소스코드의 유사도 거리를 나타낸다. 그래프 시각화는 유사하게 작성된 소스코드들의 클러스터를 쉽게 파악할 수 있도록 시각화한다. 또한 그래프 시각화는 간접 표절(indirected plagiarism)을 쉽게 파악할 수 있다. 예를 들어, 두 프로그램  $P_a$ 와  $P_b$ 의 유사도 거리가 가깝고,  $P_b$ 와  $P_c$ 의 유사도 거리가 가깝다면,  $P_a$ 와  $P_c$  또한 표절을 의심해야 한다.

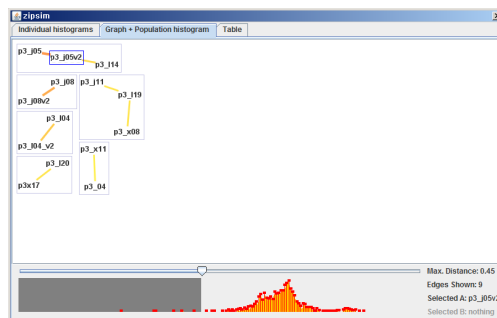
마지막으로 히스토그램 시각화는 색상으로 표시된 히스토그램으로 상대적인 유사도 거리를 시각화한다. 히스토그램의 왼편 부분이 낮은 유사도 거리를 나타내므로 왼편의 프로그램 쌍들은 표절로 의심되는 코드를 나타낸다. 또한, AC는 각각의 소스코드에 대해 상대적 유사도 거리에 대한 히스토그램을 확인할 수 있는 개별 히스토그램(individual histogram)을 제공한다. 그림 2.16은 AC를 이용한 소스코드 유사도 분석 시각화 예를 보여준다.



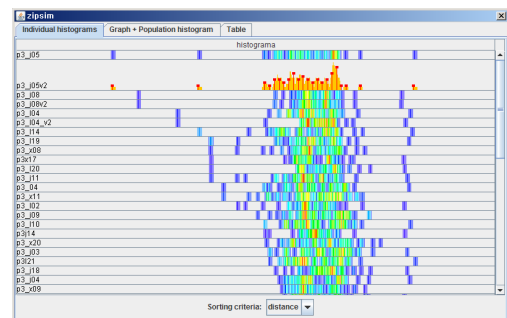
(a) 소스코드 선택

| Distance   | One       | The other |
|------------|-----------|-----------|
| 0.20223048 | p3_j05v2  | p3_j08    |
| 0.25448722 | p3_j08v2  | p3_j08    |
| 0.33711344 | p3_j04_v2 | p3_j04    |
| 0.38215613 | p3_j14    | p3_j08    |
| 0.38215613 | p3_j14    | p3_j05v2  |
| 0.4058069  | p3_x08    | p3_j19    |
| 0.4058069  | p3_j20    | p3_j17    |
| 0.4058069  | p3_j20    | p3_j11    |
| 0.40715733 | p3_j19    | p3_j11    |
| 0.4331307  | p3_x11    | p3_j04    |
| 0.4612217  | p3_x08    | p3_j11    |
| 0.48309602 | p3_j19    | p3_j02    |
| 0.47877204 | p3_j02    | p3_j11    |
| 0.4851645  | p3_x11    | p3_j14    |
| 0.5045848  | p3_x11    | p3_j09    |
| 0.51534845 | p3_x08    | p3_j02    |
| 0.51747215 | p3_j05    | p3_j04    |
| 0.51747215 | p3_j05v2  | p3_j04    |
| 0.51985284 | p3_j19    | p3_j05    |
| 0.51985284 | p3_j19    | p3_j05v2  |
| 0.52009794 | p3_j04_v2 | p3_j14    |
| 0.5220358  | p3_x20    | p3_x11    |
| 0.52431613 | p3_j14    | p3_j04    |
| 0.5271375  | p3_x11    | p3_j05    |
| 0.5271375  | p3_x11    | p3_j05v2  |
| 0.52846337 | p3_j04    | p3_j14    |
| 0.52880048 | p3_j09    | p3_j03    |
| 0.5302927  | p3_j04    | p3_j21    |

(b) 테이블 시각화



(c) 그래프 시각화



(d) 히스토그램 시각화

그림 2.16: AC를 이용한 소스코드 유사도 분석 시각화: 테이블 및 그래프, 히스토그램을 이용한 시각화 인터페이스 제공

그림 2.16-(a)는 표절검사의 입력 소스코드를 선택하는 화면을 보여준다. 소스코드 선택

화면에서 소스코드가 있는 디렉토리를 선택하면 하위의 디렉토리에 포함된 소스코드들을 자동으로 추가한다. 그리고 그림 2.16-(b)는 테이블 시각화로 두 프로그램 쌍과 이들의 상대적 유사도 거리를 보여준다. 테이블 시각화에서 두 프로그램의 상대적 유사도 거리 값이 작을 수록 프로그램 쌍은 표절 프로그램일 가능성이 높다. 다음으로 그림 2.16-(c)는 그래프 시각화 화면을 보여준다. 그래프 시각화 화면에서는 화면 하단의 히스토그램에서 유사도 기준 값보다 거리가 가까운 프로그램 쌍들의 클러스터를 그래프를 이용해 나타낸다. 마지막으로 그림 2.16-(d)는 개별 히스토그램 시각화 화면을 보여준다. 이 화면에서는 히스토그램을 이용해 각 파일 별로 상대적 유사도를 나타낸다.

- CCFinderX

CCFinderX는 Higo와 Kamiya의 연구팀에 의해 개발되었으며, 소프트웨어 유지보수를 위한 클론(clone) 분석 시스템이다[39]. 특히 소프트웨어 리팩토링 지원을 위한 시스템으로 개발되었다. 현재 이클립스 플러그인 형태로 배포되어 사용된다. CCFinderX 다음의 4가지 과정을 통해 소프트웨어의 유사도를 분석한다.

- 어휘분석(lexical analysis) : 소스코드의 각 라인을 프로그래밍 언어가 정의하는 어휘분석 규칙에 의해 토큰으로 나눈다. 다음으로 모든 소스파일의 토큰을 하나로 붙여서 긴 서열을 생성한다.
- 변환(transformation) : 토큰 서열은 변환 규칙에 의해 변환된다. 여기서 토큰들은 추가 또는 삭제, 사전 정의된 심볼로 변환된다.
- 클론 일치 탐색(Clone Match Detection) : 토큰 서열에서 일치하는 모든 부분 서열(substring)을 검출한다.
- 형식화(Formatting) : 클론 일치 탐색을 통해 검색된 코드 조각들을 원본 소스코드와 연결 시킨다.

CCFinderX에서는 유사도 분석을 통해 발견한 코드 조각들을  $NOC_{th}(F)$ ,  $ROC_{th}(F)$  등의 메트릭으로 나타내고 파일리스트와 도트플롯(dot-plot), 메트릭 그래프(metric graph)로 시각화한다. 메트릭  $NOC$ 는 하나의 소스코드에서 발견된 코드 클론 조각의 수를 나타내,  $ROC$ 는 코드 클론 조각들의 비율을 나타낸다. 그림 2.17는 CCFinderX의 분석 결과를 보여준다.

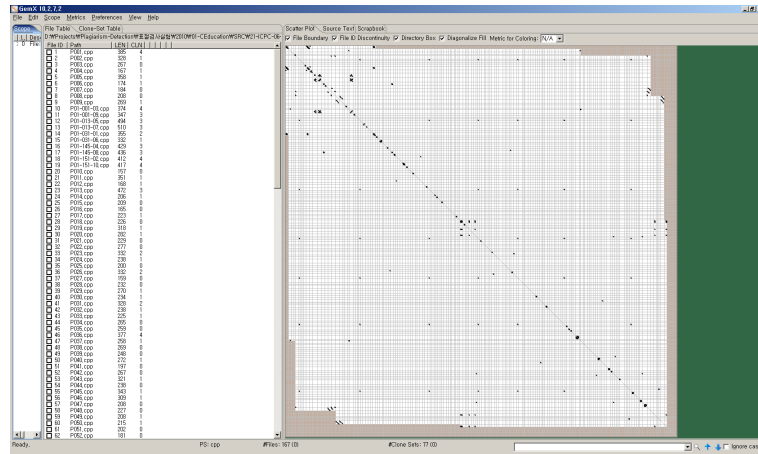


그림 2.17: CCFinderX를 이용한 코드 클론 검사 결과

그림 2.17의 가장 왼편에 위치한 탭은 클론 탐색 이력(history)을 보여준다. 그리고 중앙에는 클론 탐색에 입력된 소스코드를 보여주는 파일 테이블이 위치한다. 마지막으로 오른편에 위치한 탭은 도트플롯을 이용해 프로그램 집합에 포함된 코드 클론들을 보여준다.

- JPlag

JPlag는 현재 가장 많이 이용되고 있는 소스코드 표절 검사 시스템 중 하나로, 독일의 Karlsruhe 대학에서 1996년에 Guido Malpohl에 의해 처음 개발되었다[37]. JPlag는 현재 자바 웹 스타트(Java Web Start) 방식으로 사용자에게 서비스가 되고 있으며, 표절검사는 서버에서 이루어진다. 사용자는 클라이언트를 통해 소스코드 집합을 지정하면, 소스코드는 서버로 전송되어 비교되고, 웹페이지 형식으로 구성된 결과 파일이 클라이언트로 전송된다. 자바웹스타트와 웹페이지로 전송되기 때문에 별도의 프로그램 설치 없이 계정 등록만으로 이용할 수 있다.

JPlag는 토큰 기반 유사도 분석으로 표절 프로그램을 찾는다. JPlag는 2단계로 표절검사를 수행한다. 전단부에서는 JavaCC나 ANTLR을 이용해 토큰을 생성하고 후단부에서는 Greedy String Tiling(GST)[70] 알고리즘을 이용해 유사도와 유사구간을 찾는다. GST 알고리즘은 문자열 매칭 알고리즘으로 두 문자열에서 일치하는 부분 문자열(substring)을 찾아준다. GST 알고리즘은 최소 길이(minimum length) 이상으로 연속적으로 일치하는 모든 부분 문자열들을 찾는다. GST 알고리즘은 문자열 패턴 매칭이나 생물정보학의 DNA 서열 매칭 알고리즘으로 사용되고 있다. 그림 2.18는 JPlag를 이용한 표절탐색 비교 결과를 보여준다.

그림 2.18와 같이 JPlag는 프로그램 유사도 분석 결과를 웹 페이지로 보여주기 때문에 웹 브라우저를 이용해 결과를 확인할 수 있다. 그림 2.18는 프로그램 쌍의 유사도 분석 결과를 나타낸다. 웹 페이지 상단에는 두 프로그램의 유사도를 보여주고, 중앙에는 프로그램 소스코드 쌍에서 동일하게 작성된 구간을 여러 가지 색으로 표시하여 나타낸다.

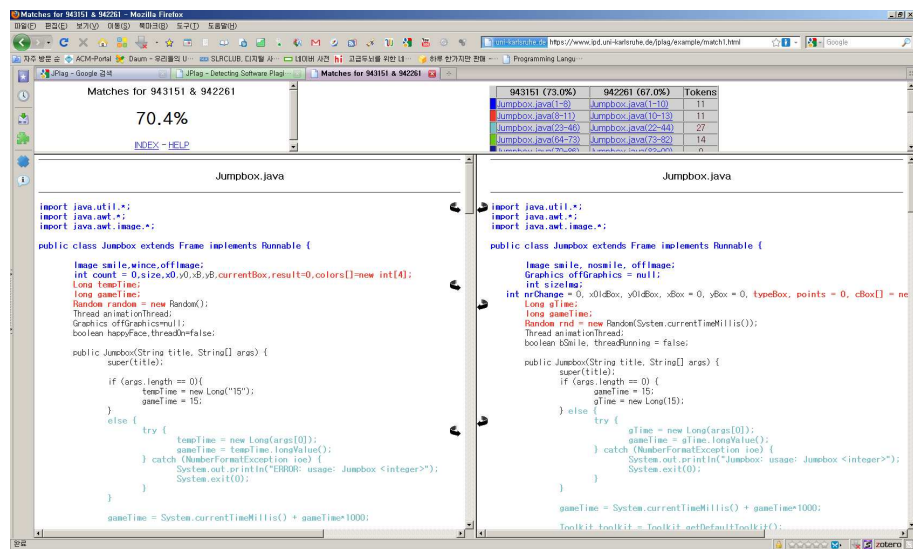


그림 2.18: JPlag를 이용한 소스코드 표절검사 결과

## 제 3장

# 프로그램 유사도 분석

본 장에서는 프로그램 유사도 분석을 위한 적응적 프레임워크에 대해 살펴본다. 적응적 유사도 분석 프레임워크에 대한 설명에 앞서 적응적 프레임워크의 구조를 대략적으로 기술한다. 다음으로 두 프로그램을 입력받아 이들 사이의 유사도를 계산하는 과정을 자세히 기술한다. 적응적 유사도 분석 방법은 생물정보학에서 사용되는 DNA 서열 정렬 기법에 기반을 두고 있으며, 이를 프로그램 유사도 분석에 적합하도록 확장하였다. 프로그램 유사도는 소스코드를 입력으로 하여 프로그램 서열 생성, 적응적 지역정렬, 정규화 과정을 거쳐 계산된다.

### 3.1 적응적 유사도 분석 프레임워크 구조

본 논문에서는 소스코드를 이용하여 프로그램의 유사도를 적응적으로 분석하는 적응적 프로그램 유사도 분석 프레임워크(적응적 프레임워크)를 제시한다. 적응적 프레임워크는 크게 유사도 계산과 유사도 분석의 두 부분으로 구성된다. 그림 3.1은 프로그램 유사도 분석을 위한 적응적 프레임워크의 구성을 보여준다.

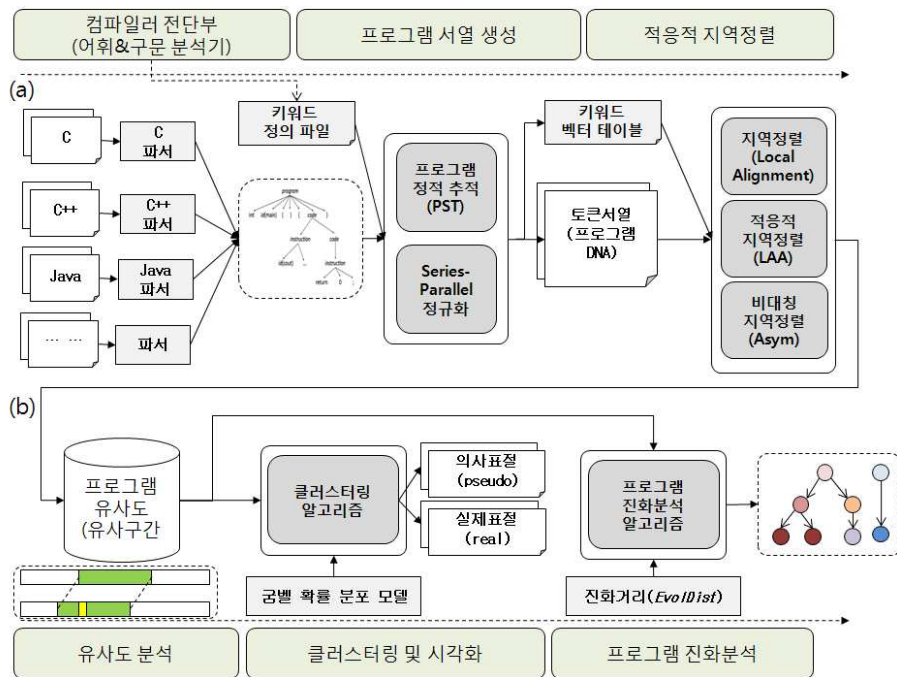


그림 3.1: 적응적 유사도 분석 프레임워크 구성도

그림 3.1은 적응적 프레임워크가 프로그램 유사도를 계산하고 이를 이용하여 분석하는 과정을 순서대로 보여준다. 그림 3.1의 적응적 프레임워크는 6개의 모듈들이 서로 연동하여 동작하며, 각 모듈은 선택적으로 이용 된다.

그림 3.1-(a) 단계에서는 프로그램 서열 생성과 정렬을 이용하여 프로그램 쌍들의 유사도를 계산한다. 적응적 프레임워크는 프로그래밍 언어의 컴파일러 진단부 모듈을 이용해 프로그램 서열을 생성한다. 컴파일러 진단부의 어휘분석기와 구문분석기를 이용해 프로그램 소스코드로부터 파스트리(parse tree)를 생성한다. 본 연구에서는 C/C++ 프로그램 분석을 위해 OpenC++ 컴파일러를 확장하여 토큰 서열(프로그램 DNA) 추출 모듈을 구현하였다. 프로그램 서열 생성 단계에서는 프로그램 그룹에 사용된 키워드 빈도 계산과 프로그램 정적 추적(Program Static Tracing), 정규화(Canonical Form)를 수행한다. 다음으로 적응적 지역정렬 단계에서는 프로그램 서열들 사이의 유사도를 계산한다. 유사도 계산에는 일반적인 DNA 서열 분석에 사용되는 고정적 지역정렬과 프로그램 유사도 분석에 적합하도록 지역정렬을 확장한 적응적 지역정렬(adaptive local alignment)을 사용한다. 또한, 적응적 프레임워크에서는 두 프로그램의 유사도를 비대칭적으로 계산할 수 있는 비대칭 지역정렬(Asym())을 지원한다. 유사도 계산에서는 프로그램 집합에 포함된 모든 소스코드들의 쌍에 대한 유사도와 유사구간이 계산된다.

적응적 프레임워크의 후단부인 그림 3.1-(b)는 유사도 계산 결과를 이용해 분석하는 단계이다. 이 단계에서는 클러스터링 알고리즘과 진화분석 알고리즘을 이용해 의사표절(pseudo plagiarism)과 프로그램 진화분석(evolution analysis)를 수행한다. 의사표절 정의와 분석 알고리즘에 대해서는 4장에 자세하게 설명한다. 적응적 프레임워크는 유사도 계산에만 초점을 맞추는 기존 연구들에 비해 한 단계 진일보한 프로그램 유사도 분석 프레임워크이다. 마지막으로 본 논문에서는 C와 C++, Java 소스코드를 대상으로 프로그램 유사도 분석 도구인 **SoVAC**(Software Verification and Analysis Center)를 구현하였다. 다음 절에서는 적응적 유사도 분석 프레임워크를 구성하는 각 모듈의 수행 및 알고리즘에 대해 자세히 설명한다.

## 3.2 프로그램 서열 생성

본 절에서는 프로그램 소스코드로부터 프로그램 DNA를 생성하는 프로그램 선형화에 대해 기술한다. 프로그램 선형화는 프로그램 집합으로부터 적응적 지역정렬에 필요한 서열을 추출하고, 프로그램 집합에 사용된 키워드 빈도 벡터를 생성한다. 프로그램 선형화 단계에서는 프로그램 유사도 분석의 정확성 향상을 위해 프로그램 정적 추적(Program Static Tracing, PST)과 직병렬 그래프(Series-Parallel, SP) 모델에 기반한 정규화 과정을 거친다. 본 절에서는 프로그램 정적 추적과 정규화 과정에 대해 자세하게 설명한다.

### 3.2.1 프로그램 소스코드

프로그램 유사도 분석은 프로그램 소스코드와 이진 파일(binary file)을 대상으로 한다. 대부분의 프로그램 유사도 분석 연구는 소스코드를 입력으로 사용하고 있으며, 자바의 바이트코드(bytecode)와 같은 컴파일된 이진 파일을 이용하기도 한다[36, 62]. 본 논문에서는 C 또는 C++, Java 언어로 작성된 소스코드를 입력으로 유사도를 분석한다. 그림 3.2와 그림 3.3은 C언어로 거품 정렬을 구현한 소스코드 예제를 보여준다.

거품 정렬 프로그램 소스코드는 프로그램 선형화 방법을 보여주기 위해 작성된 예제 코드이다. 그림 3.3의 소스코드  $P_b$ 는 그림 3.2의 소스코드  $P_a$ 를 원본으로 하여 유사하게 작성되었다.  $P_b$ 는  $P_a$ 에서 다음 항목들을 수정하였다.

1. 주석 제거 :  $P_a$ 의 코드에서 주석을 제거하였다.
2. 함수 위치 변경 :  $P_a$ 의 코드에서 bubble() 함수 정의 위치를 main() 앞으로 변경하였다.
3. 함수 이름 변경 : bubble() 함수 이름을 bubble\_sort으로 변경하였다.
4. 함수 분리 :  $P_a$ 의 bubble() 함수를 bubble\_sort()와 swap() 함수로 분리하였다.

```

1 #include <stdio.h>
2
3 void bubble(char *, int);
4
5 int main() {
6     char str[] = "193258623742010597";
7     int len;
8     // calculating the length of str.
9     len = (sizeof(str)/sizeof(str[0]))-1;
10
11     printf("bubble sorting : str = %s", str);
12     bubble (str, len);
13     printf("bubble sorting : str = %s", str);
14
15     return 1;
16 }
17
18 void bubble(char *arr, int len) {
19     int i, j;
20     char temp;
21
22     for (i = 0; i < len-1; i++) {
23         for (j = 0; j < len - i - 1; j++) {
24             if ( arr[j] > arr[j+1] )
25             {
26                 temp = arr[j];
27                 arr[j] = arr[j+1];
28                 arr[j+1] = temp;
29             }
30         }
31     }
32     return;
33 }

```

그림 3.2: 프로그램  $P_a$ : 거품 정렬 원본 소스코드

```

1 #include <stdio.h>
2
3 void swap(char *x, char *y)
4 {
5     if ( *x > *y)
6     {
7         char temp = *x;
8         *x = *y;
9         *y = temp;
10    }
11 }
12
13 void bubble_sort(char *arr, int len)
14 {
15     int i, j;
16
17     for (i = 0; i < len-1; i++) {
18         for (j = 0; j < len - i - 1; j++) {
19             swap((arr+j), (arr+j+1));
20         }
21     }
22
23     return;
24 }
25
26 int main()
27 {
28     char string[] = "913258623742010597";
29     int len;
30
31     len = (sizeof(string)/sizeof(string[0])) -1;
32
33     printf("bubble sorting : str = %s", len);
34     bubble_sort (string, len);
35     printf("bubble sorting : str = %s", len);
36
37     return 1;
38 }

```

그림 3.3: 프로그램  $P_b$ :  $P_a$ 와 유사하게 작성된 거품 정렬 프로그램

그림 3.3의 소스코드 수정에 사용된 방법들은 빈번하게 사용되는 프로그램 표절 방법들 중 하나이다. 프로그램 유사도 분석은 이와 같은 코드 수정에 대해 유사 코드를 탐색할 수 있어야 한다.

### 3.2.2 프로그램 선형화

본 절에서는 지역정렬을 이용해 두 프로그램  $P_a$ 와  $P_b$ 의 유사도와 유사구간을 측정하기에 앞서, 기본적인 정의들과 프로그램 선형화 방법에 대해 기술한다. 먼저 본 절에서는 프로그램 유사도 분석에 필요한 기본적인 용어들을 정의한다.

**정의 1.** 프로그램 집합  $\mathbb{P}$ 는  $m$ 개의 프로그램으로 구성되며, 프로그램  $P_i$ 는  $n$ 개의 소스코드  $S$ 로 작성된다.

$$\mathbb{P} = \{P_1, P_2, \dots, P_m\}$$

$$P_i = \{S_1, S_2, \dots, S_n\}$$

여기서 하나의 소스코드  $S_i$ 는  $|S_i|$ 개의 줄로 작성된 코드를 나타낸다. 프로그램 유사도 분석은 프로그램 집합  $\mathbb{P}$ 의 소스코드들에서 복사 또는 수정, 삭제를 통해 유사하게 작성된 코드 블록  $C$ 을 찾는 것이다.

**정의 2.** 코드 블록  $C(S, L_i, L_j)$ 는 소스코드  $S$ 에서 유사하게 작성된 코드 블록을 나타낸다. 여기서  $L_i$ 와  $L_j$ 는 코드 블록의 시작과 끝의 줄 번호를 나타낸다( $i \leq j$ ).

코드 블록  $C_n$ 는 두 소스코드가 유사하게 작성되었을 때, 각각의 소스코드에서 검출된  $n$ 번째 코드 블록 쌍을 나타낸다. 예를 들어, 소스코드  $S_a$ 의 70~80번째 줄과  $S_b$ 의 120~130번째 줄이 같다면 코드 블록  $C_n$ 는 다음과 같다.

$$C_n = \langle C(S_a, L_{70}, L_{80}), C(S_b, L_{120}, L_{130}) \rangle$$

소스코드 기반 프로그램 유사도 분석에서는 프로그램 집합  $\mathbb{P}$ 에서 유사하게 작성된 모든 코드 블록 집합  $C = \{C_1, C_2, \dots, C_n\}$ 를 찾는다. 여기서  $n$ 은 코드 블록 개수를 나타낸다.

프로그램 선형화는 소스코드에서 분석에 필요한 토큰을 추출하는 단계이다. 이 단계에서는 토큰 추출에 앞서 토큰 집합을 먼저 정의해야 한다. 토큰 집합은 프로그래밍 언어에 따라 다르게 정의된다. 일반적으로는 프로그래밍 언어에서 사용하는 키워드를 토큰으로 사용할 수 있다. 예를 들어, C 언어의 경우 `int`, `for`, `return` 등과 같은 키워드들이 토큰으로 사용된다. 본 논문에서 키워드 백터  $\mathcal{K}$ 는 다음과 같이 정의된다.

**정의 3.** 키워드 백터  $\mathcal{K}$ 는 소스코드로부터 토큰을 추출하기 위해 정의한 키워드와 연산자로  $\mathcal{K} = \{k_1, k_2, \dots, k_r\}$  같이 정의된다. 여기서  $r$ 은 키워드 전체 개수를 나타낸다.

본 논문에서 C/C++ 프로그램의 유사도 분석에서는 키워드 백터는 총 81개의 원소로 구성된다( $r = 81$ ). 그림 3.4은 그림 3.2의 코드를 선형화한 것이다.

| 토큰 서열                                                 |
|-------------------------------------------------------|
| 1. FUNC_CALL, # main(),                               |
| 2. BLOCK_START, CHAR, ASSIGN_EQ                       |
| 3. INT, ASSIGN_EQ, SIZEOF, DEVIDE, SIZEOF, MINUS      |
| 4. UNREACHABLE_FUNC_CALL, # printf                    |
| 5. FUNC_CALL, # bubble.sort(),                        |
| 6. BLOCK_START, INT, FOR, ASSIGN_EQ, LT, MINUS, INCR, |
| 7. FOR, ASSIGN_EQ, LT, MINUS, MINUS, INCR,            |
| 8. BLOCK_START, PLUS, PLUS, PLUS,                     |
| 9. FUNC_CALL, # swap                                  |
| 10. BLOCK_START, IF, PTR, GT, PTR,                    |
| 11. BLOCK_START, CHAR, ASSIGN_EQ,                     |
| 12. PTR, PTR, ASSIGN_EQ,                              |
| 13. PTR, PTR, ASSIGN_EQ,                              |
| 14. BLOCK_END, BLOCK_END, BLOCK_END, BLOCK_END,       |
| 15. RETURN, BLOCK_END,                                |
| 16. UNREACHABLE_FUNC_CALL, # printf                   |
| 17. RETURN, BLOCK_END,                                |
| 18. FUNC_END # main                                   |

그림 3.4: 프로그램 정적 추적을 이용한 프로그램 DNA 서열 추출: 그림 3.3에 대한 토큰 서열

그림 3.4은 그림 3.3의 거품 정렬 소스코드에서 `main`과 `bubble_sort`, `swap` 함수의 코드

로부터 추출한 토큰 서열을 보여준다. 이들 토큰 서열은 지역정렬에서 해시코드로 변환하여 빠르게 계산된다. 그림 3.4의 토큰 서열 추출에서는 프로그램 정적 추적이 사용되었으며, 이는 프로그램 유사도 분석의 정확성을 높이기 위한 방법이다.

### 3.2.3 프로그램 정적 추적

프로그램 정적 추적(PST)은 프로그램을 소스코드 단계에서 정적으로 실행함으로써 제어 흐름을 고려하여 토큰 서열을 생성한다. PST에서 프로그램의 정적 실행은 구문분석에서 생성된 파스트리를 이용한다. 일반적으로 프로그램은 main 함수에서 시작하여 main가 return문을 만나면 종료한다. 그리고 프로그램 실행 중간에 시스템 콜이나 사용자 정의 함수를 만나게 되면 프로그램 카운터를 함수로 옮겨 실행하고, return문을 만나면 복귀하여 계속 진행한다. PST에서는 프로그래밍 언어에 정의된 제어구문과 함수 호출을 고려한다. PST의 토큰 생성 규칙을 요약하면 다음과 같다.

1. PST는 main 함수에서 시작하여 main가 return하면 종료된다.
2. 사용자 정의 함수 내에서는 순차적으로 토큰을 생성한다. 예를 들어, C언어의 ‘for’나 ‘while’과 같은 반복문도 순차적으로 1회 탐색한다.
3. ‘if’문이나 ‘switch’문과 같은 제어문의 경우, 참(true)과 거짓(false)의 모든 조건에 대해 탐색한다.
4. 토큰 생성 중에 시스템 콜 함수를 만나면 ‘UNREACHABLE\_FUNC\_CALL’ 토큰으로 기록하고 추적을 하지 않는다.
5. 사용자 정의(user-defined) 함수를 만나면 ‘FUNC\_CALL’ 토큰을 기록하고 함수로 이동하고 복귀하면 추적을 계속 진행한다.
6. 순환함수 호출이 될 경우 1회 함수 순환에 대해서만 토큰을 생성하고 다음으로 진행한다.

PST는 제어문과 함수에 대해 실제 프로그램 실행과 다르게 동작한다. 예를 들어, C 언어의 'if'나 'if-else'문의 경우 실제 프로그램 실행에서는 조건식(conditional expression)에 따라 한가지 경우에 대해서 실행된다. 하지만, 프로그램 정적 분석에서는 파스트리를 순회하면서 모든 조건에 대해 순서대로 추출한다. 'switch'문의 경우에도 모든 'case'에 대해 토큰 서열을 생성한다. 이와 유사하게, 'for'문이나 'while'과 같은 반복문에 대해서도 조건식에 관계 없이 한번만 정적으로 실행하여 토큰을 생성한다.

함수의 경우 두 가지로 나누어 진행된다. 먼저, 'open()'이나 'read()'와 같은 시스템 콜 함수나 'printf()' 같은 라이브러리 함수의 경우 실제 함수 정의를 알 수 없기 때문에 추적을 할 수 없다. 이 경우 'UNREACHABLE\_FUNC\_CALL' 토큰을 기록하고 다음으로 진행한다. 'UNREACHABLE\_FUNC\_CALL' 토큰은 프로그래밍 언어에서 정의하는 토큰이 아니고 PST를 위해 추가로 정의한 토큰이다. 사용자 함수의 경우 'FUNC\_CALL' 토큰을 기록하고 실제 함수 정의로 점프를 한다.

프로그램 유사도 분석에서 함수 호출을 고려하는 것은 유사도 분석 결과의 정확성을 높인다. 일반적으로 유사도 분석을 위한 토큰 서열은 순차적으로 추출된다. 순차적 추출은 소스파일의 첫 번째 줄부터 마지막 줄까지 파스트리를 순회하면서 토큰 서열을 생성하는 방법이다. 예를 들어, 순차적 추출로 그림 3.2에서 토큰을 추출하면 `main` → `bubble`의 순서로 토큰을 생성하고, 그림 3.3의 경우 `swap` → `bubble_sort` → `main`의 순서로 토큰을 생성한다. 그림 3.5는 거품 정렬 코드에 대한 토큰 서열 생성 순서를 그래프로 나타낸 것이다. 그림 3.5에서 노드는 함수를 나타내고 화살표는 토큰 서열 생성 순서를 나타낸다. 순차적 토큰 생성은 함수 정의의 위치가 다르면 함수가 같더라도 다른 순서로 토큰을 생성한다.

토큰 생성 순서는 유사도 계산의 정확성과 관계가 있다. 일반적으로 서열이나 문자열 비교에서 유사도는 연속적으로 일치하는 문자의 개수에 따라 달라진다. GST 알고리즘에서 높은 유사도의 문자열들은 연속적으로 일치하는 문자가 많다는 것을 의미한다. 이는 DNA 정렬 알고리즘에서도 동일하다. 프로그램 유사도 분석에서 소스코드의 위치가 다르더라도 코드 내용이 같으면 동일하게 토큰을 생성해야 한다. PST는 함수 호출 순서에 따라 토큰을

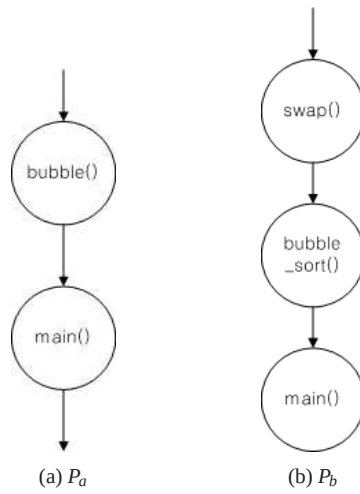


그림 3.5: 거품 정렬 소스코드의 순차적 토큰 서열 생성 그래프

생성하기 때문에 코드가 같다면 함수 위치가 다르더라도 동일한 토큰 서열을 생성한다. 그림 3.6은 PST를 이용한 토큰 생성 순서를 그래프로 나타낸 것이다.

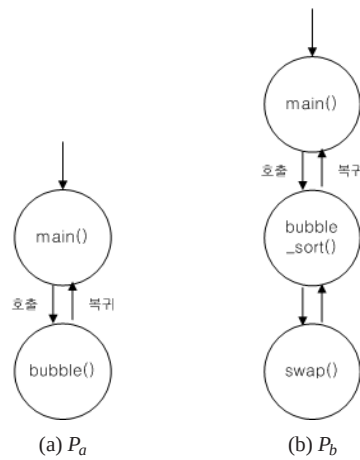


그림 3.6: PST를 이용한 거품 정렬 소스코드의 토큰 서열 생성 그래프

그림 3.6은 함수 정의 위치가 다르더라도 두 소스코드의 토큰 서열이 거의 유사하게 생성될 수 있음을 보여준다. 그림 3.6에서 아래쪽으로 향한 화살표는 함수 호출을 나타내고

위쪽으로 향한 화살표는 함수 실행 후 복귀됨을 나타낸다.

PST 방법으로 토큰을 생성할 때 직접 순환함수와 간접 순환함수 호출을 고려해야 한다. 직접 순환함수의 경우 실제 프로그램 실행에서는 복귀를 위한 조건 검사가 있어서 함수를 순환적으로 호출하다가 다시 복귀가 가능하다. 하지만 정적 분석에서는 실제로 복귀 조건 검사를 하지 않기 때문에 함수를 순환적으로 계속 호출할 수 없다. 또한 토큰 서열을 반복적으로 생성하는 것은 유사도를 떨어뜨린다. 그래서 직접 순환함수의 경우 함수에 대한 토큰 서열을 한번만 생성한다. 그림 3.7는 1부터 10까지의 계승(factorial)을 계산하는 소스코드이고 그림 3.8는 이에 대한 토큰 생성을 그래프로 나타낸 것이다.

```
1 #include <stdio.h>
2
3 int factorial(int n);
4
5 int main(void)
6 {
7     int i,result;
8
9     result = factorial(10);
10
11     return 0;
12 }
13
14 int factorial(int n)
15 {
16     if(n <= 1)
17         return 1;
18     else
19         return n * factorial(n-1);
20 }
```

그림 3.7: factorial.c : 1부터 10까지의 계승을 구하는 프로그램

PST 방식으로 토큰을 생성할 때 또하나 고려해야 하는 것이 함수의 간접 순환호출이다. 예를 들어, C언어로 작성된 프로그램에서 f에서 g를 호출하고, g에서 h를 호출하고, h에서 다시 f를 호출한다면( $f \rightarrow g \rightarrow h \rightarrow f$ ), 세 함수는 순환적으로 호출된다. 직접 순환함수와 동일하게 간접 순환함수 호출에 대한 토큰 서열도 한번만 생성된다. 그림 3.9는 간접 순환 함수 호출에 대한 토큰 생성을 그래프를 이용해 나타낸 것이다.

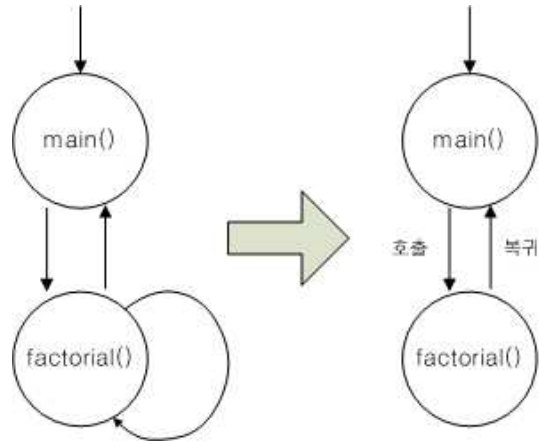


그림 3.8: 직접 순환함수 호출에 대한 토큰 생성: factorial.c에 대한 토큰 생성.

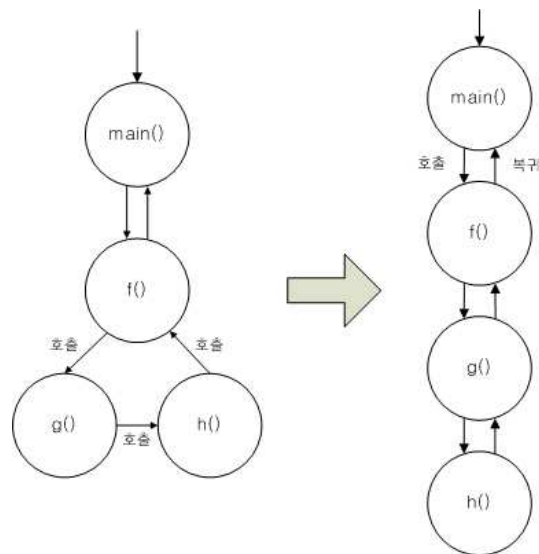


그림 3.9: 간접 순환함수 호출에 대한 토큰 생성:  $\text{main} \rightarrow \text{f} \rightarrow \text{g} \rightarrow \text{h} \rightarrow \text{f}$  순서로 호출

### 3.2.4 직병렬 그래프 모델 기반 정규화

적응적 프레임워크는 프로그램 정적 추적과 함께 소스코드의 단순한 문법적 변환에 대해 직병렬(SP) 그래프를 기본 모델로 하는 프로그램 DNA 서열 정규화를 수행한다. 정규화는 프로그래밍 언어의 기본 제어 구문들을 DNA 서열로 변환할 때 사용된다.

원본 프로그램으로부터 표절이나 유사 프로그램을 생성할 때 가장 많이 사용되는 방법으로는 (1) 복사-붙이기, (2) 식별자 변환, (3) 제어구문 변환 등이 있다. 본 절에서는 (3) 항목인 제어구문 변환에 초점을 맞춘다. C/C++, Java 프로그램에서 if-else 구문 변환이나 for  $\rightarrow$  while는 쉽게 변환할 수 있다. 예를 들어, C언어로 작성된 프로그램에서 프로그램 구조에 대한 깊은 이해 없이 *if(A) B; else C;* 구문을 *if( $\neg$ A) C; else B;* 구문으로 쉽게 바꿀 수 있다. 또한, 이와 유사하게 *for(A;B;C) {D}* 구문을 *A; while(B) {D; C;}* 같이 변환 가능하다. 그러나 단순 어휘 분석이나 구문 분석만으로는 이들을 찾아내기가 어렵다.

토큰 서열을 생성하는 순차적 방법과 PST방법만으로는 제어 구문 변환은 유사도 분석에 반영할 수 없다. 이전의 방법만으로는 *if(A) B; else C;*와 *if( $\neg$ A) C; else B;*의 토큰 서열은 많이 다르게 생성된다. 이에 대해 적응적 프레임워크에서는 SP 그래프 모델은 구문 단위로 토큰 서열을 정규화하기에 적합하다.

정규화는 C/C++와 Java 같은 절차형 언어(procedural language)의 세 가지 기본 블록(순차, 조건, 반복)에 대해 적용한다. 예를 들어, C/C++의 경우, 조건문에는 *if-else*나 *switch-case*가 있고, 반복문에는 *for* 또는 *while* 루프가 해당된다. 토큰 서열 정규화에서는 위의 제어구문에 대해 SP 그래프 모델을 적용한다 기본블록에서 순차 블록과 반복 블록에 대해서는 SP 그래프의 직렬 에지(serial edge)로 표현하고 조건 블록에 대해서는 병렬 에지(parallel edge)로 표현한다. 정규화에서는 구문들을 SP 그래프의 에지로 변환한 후 재배열한다. 그림 3.10은 순차 블록과 *if-else*, *switch-case* 블록을 SP 그래프 표현한 것이다.

프로그램 토큰 서열의 정규화 형식은 재귀적으로 추출할 수 있다. 소스코드 블록 *B*의 프로그램 토큰 서열을 *dna(B)*라고 가정한다. 그리고 *dna(.)*는 사전식 순서로 정렬된 서열을

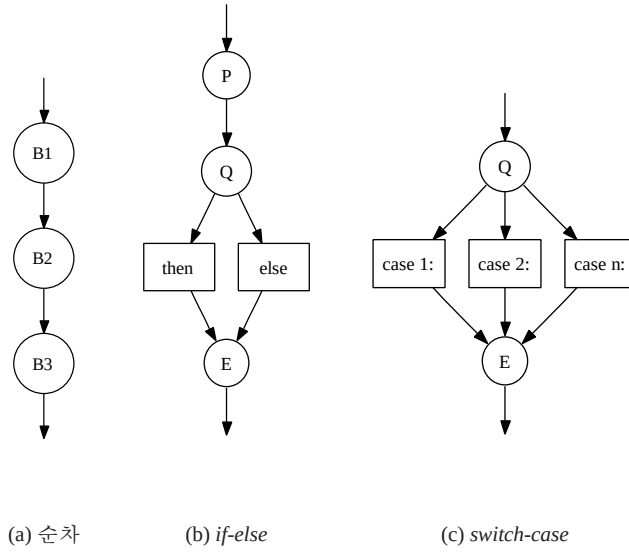


그림 3.10: SP 그래프 정규화: (a) 순차블록, (b) if-else, (c) switch-case

반환한다. 여기서  $dna(\cdot)$ 는 문자열이나 해시코드와 같은 정수형의 서열이다. 예를 들어, 그림 3.10-(a)와 같이 소스코드 블록  $B$ 가 순차 구문들로만 구성된 경우, 전체  $dna(B)$ 은 각 블록의  $dna$ 를 연결한다. 다시 말해서,  $dna(B) = dna(B_1) \odot dna(B_2) \odot dna(B_3)$ 가 된다. 여기서  $\odot$ 은 문자열 연결 연산을 나타낸다. 그림 3.10-(b)의 if-else 구문은 먼저 *then*과 *else* 블록에 대해 각각  $dna(\cdot)$ 를 수행하여 사전 순서로 정규화한다. 다음으로  $dna(\text{if } (A) \text{ then } B \text{ else } C)$  전체에 대한 정규화를 다음과 같이 수행한다.

$$dna(\text{if } (A) \text{ then } B \text{ else } C) = dna(A) \odot first\{dna(B), dna(C)\} \odot last\{dna(B), dna(C)\},$$

여기서  $first\{A, B\}$ 는  $A$ 와  $B$  중에 사전식으로 빠른 서열을 반환하고,  $last$ 는 반대로 사전식으로 늦은 서열을 반환한다. switch-case의 경우, 마찬가지로 각 *case* 블록에 대해  $dna(\cdot)$ 를 수행하고 전체 블록을 정규화한다.

### 3.3 적응적 지역정렬

본 절에서는 프로그램 선형화 단계에서 추출한 두 프로그램의 토큰 서열 사이의 유사도 계산을 위한 적응적 지역정렬 방법에 대해 살펴본다. 적응적 지역정렬 설명에 앞서 생물정보학에서 사용되고 사용되고 있는 DNA 서열 정렬 기법에 대해 살펴보고, 대표적인 정렬 기법인 전체정렬(global alignment)과 지역정렬(local alignment)을 살펴본다. 다음으로 본 논문에서 새롭게 정의하고 있는 프로그램 유사도 분석을 위한 적응적 지역정렬(adaptive local alignment)과 비대칭 지역정렬(asymmetric local alignment)의 유사도 계산에 대해 자세하게 살펴본다.

#### 3.3.1 DNA 서열 정렬 기법

정렬(alignment)은 생물정보학에서 많은 연구가 되어 있는 분야 중 하나로 서열들을 비교하여 유사한 영역을 찾아내는 분석방법이다. 생물정보학의 유전체 서열 분석 연구는 유전자나 단백질(protein) 서열들을 비교한다. 유전체 서열 정렬의 기본은 동일한 서열을 나타내는 유전자는 동일한 기능을 한다는 가정이다. 이러한 가정에서부터 시작하여 동종 또는 타종의 유전자들 사이의 유사성을 비교한다. 예를 들어, 새로운 종을 발견하여 그것의 기능을 알아내기 위해 가장 흔하게 사용하는 방법은 유전자 은행(DNA bank)이나 유전체 데이터베이스(genome database)에서 쿼리 유전자 서열과 일치하거나 매우 유사한 서열을 찾아내어 새로운 유전자의 기능을 유추한다.

유전자 분석에서 두 서열 사이에 동일한 영역이나 비슷한 부분을 찾는 것은 서열 분석과 기능 유추를 위해 매우 중요하다. 그리고 다중정렬(multialignment)을 이용하여 여러 유전체들 사이의 진화 분석(evolutionary analysis)나 종별 연구 등을 할 수 있다. 이밖에도 유전체 서열 정렬 기법은 유전자나 단백질 유사성 분석 뿐만 아니라 문자열이나 토큰, 프로그램 소스코드의 유사성 분석에도 많이 이용되고 있다[28, 71]. 정렬은 선형 구조를 이루는 모든 객체의 유사성 비교에 적용할 수 있다.

유전체 서열 정렬에서 정렬이란 일차적으로 서열에 공백(gap)을 넣어서 두 서열을 같은 길이로 만드는 것을 말한다. 서열 정렬 기법에는 전체 정렬(global alignment)과 지역정렬, 복합정렬(semi-global) 정렬 기법들이 있다. 전체 정렬은 두 서열 전체를 비교하고, 지역정렬은 두 서열에서 부분적으로 일치하는 영역을 찾는다. 마지막으로 복합정렬은 길이 차이가 많은 서열의 비교에 이용된다. 이들 정렬은 동적 프로그래밍(dynamic programming)을 이용하여 쉽게 계산할 수 있다.

### 3.3.2 전체정렬

전체정렬은 두 서열의 전체적인 유사도를 분석하기 위한 방법으로 서열의 원소가 일치하는 경우와 일치하지 않는 경우, 공백을 삽입하는 경우에 대해 점수를 부여한다. 두 서열에 대해 모든 원소에 대해 일치, 불일치, 공백을 통해 부여하고 난 후 점수 합계가 높은 서열 정렬 결과를 찾는다. 일치와 불일치, 공백에 대한 점수는 방법에 따라 달라질 수 있으며, 일반적으로 일치에 대해서 +1점, 불일치에 대해 -1점, 공백 삽입에 대해서 -2점을 부여한다. 전체정렬을 이용한 유사도 계산은 다음과 같이 정의된다.

**정의 4.** 두 서열  $P$ 와  $Q$ 에서  $p_i$ 는  $P$ 의 원소이고  $q_i$ 는  $Q$ 의 원소일 경우( $i \leq |P|, j \leq |Q|$ )  $p_i$ 와  $q_i$ 의 점수  $score(p_i, q_i)$ 는 다음과 같이 정의된다.

$$score(p_i, q_j) = \begin{cases} m, & \text{if } p_i = q_i \\ d, & \text{if } p_i \neq q_i \end{cases} \quad (3.1)$$

식 3.1에서  $m$ 은 일치에 대한 점수를 나타내고,  $d$ 는 불일치에 대한 감점을 나타낸다.  $score$  함수를 이용한 전체정렬의 점수 행렬은 다음과 같이 정의될 수 있다.

**정의 5.**  $Score_G(i, j)$  함수는  $p_i$ 와  $q_j$  원소에서 최적의 정렬점수(optimal alignment score)로 다

음과 같이 계산된다.

$$\text{Score}_G(i, j) = \max \{ \begin{aligned} &\text{Score}(i, j-1) + g, \\ &\text{Score}(i-1, j-1) + \text{score}(p_i, q_i), \\ &\text{Score}(i-1, j) + g \end{aligned} \quad (3.2)$$

$\text{Score}_G(i, j)$  함수에서  $g$ 는 공백에 의한 감점이다. 두 서열  $P$ 와  $Q$ 의  $i$ 번째 원소와  $j$ 번째 원소의 점수는 위의 세가지 경우에 대해서 가장 큰 값을 취한다. 다음은 ACGT로 구성된 두 DNA 서열 “CACAGTGT”와 “CAGGT”를 전체정렬을 이용해 비교하면 그림 3.11과 같이 정렬된다. 그림 3.11에서 붉은 색으로 표시한 “CA”와 “G”, “GT”는 두 서열에서 일치하는 것을 나타내고, “-”는 공백 삽입을 나타낸다.

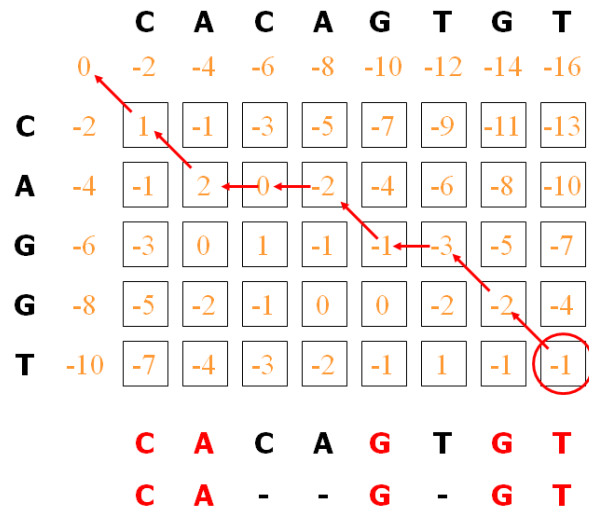


그림 3.11: 전체정렬을 이용한 서열 정렬: “CACAGTGT”와 “CAGGT” 서열

### 3.3.3 지역정렬

지역정렬은 1981년에 Smith와 Waterman이 제안한 방법으로 Smith-Waterman 알고리즘으로

도 불린다[72]. 이 알고리즘은 처음에는 핵산이나 단백질에서 유사한 영역을 찾기 위해 개발되었다. 지역정렬은 전체정렬과는 반대로 두 서열에서 부분서열이 가장 길게 일치하도록 비교한다. 지역정렬도 전체정렬과 마찬가지로 동적 프로그래밍을 통해 구현할 수 있다.

지역정렬의 점수 행렬  $Score_L(i,j)$ 은 전체정렬의  $Score_G(i,j)$ 에서 한 가지 조건이 추가되었다. 지역정렬에서는  $Score_L(i,j)$  함수에 0을 추가하여 연속적인 불일치나 공백을 삽입하더라도 점수가 0 이하로 내려가지 않는다.  $Score_L(i,j)$  함수는 다음과 같이 정의된다.

**정의 6.**  $Score_L(i,j)$  함수는 지역정렬의 점수행렬(scoring matrix)로  $p_i$ 와  $q_j$  원소에서 최적의 정렬점수를 계산한다.  $Score_L(i,j)$ 은 항상 0 이상의 값을 반환한다.

$$Score_L(i, j) = \max \{ \begin{array}{l} Score(i, j-1) + g, \\ Score(i-1, j-1) + score(p_i, q_j), \\ Score(i-1, j) + g, \\ 0 \end{array} \quad (3.3)$$

지역정렬에서는 두 서열의 원소가 처음으로 일치하는 곳에서부터 연속적으로 가장 길게 일치하는 부분까지의 점수로 가장 높은 점수의 부분까지를 유사구간으로 반환한다. 지역정렬에서의 0 값의 의미는 서열의 유사구간이 끊김을 의미한다. 다시 말해서, 원소의 일치로 점수가 0보다 큰 값을 가지다가 다시 0으로 될 경우에는 0인 구간은 두 서열은 일치하지 않음을 나타낸다. 0으로 인해 지역정렬은 두 서열의 부분 서열의 유사도를 계산할 수 있다. 그림 3.12은 두 서열 “AGGTATTG”와 “CTATGC” 서열에 대한 지역정렬 계산을 보여준다.

### 3.3.4 적응적 지역정렬

프로그램 선형화 다음으로 유사도 계산을 위한 지역정렬을 수행한다. 지역정렬은 두 토큰 서열들 사이에 유사도 점수와 유사 구간을 계산한다. 본 논문에서는 두 프로그램 사이의 유사도를 적응적으로 계산하는 새로운 알고리즘인 적응적 지역정렬을 제안한다. 적응적 지

|          |   |          |          |          |          |          |          |          |          |
|----------|---|----------|----------|----------|----------|----------|----------|----------|----------|
|          |   | <b>A</b> | <b>G</b> | <b>G</b> | <b>T</b> | <b>A</b> | <b>T</b> | <b>T</b> | <b>A</b> |
|          | 0 | 0        | 0        | 0        | 0        | 0        | 0        | 0        | 0        |
| <b>C</b> | 0 | 0        | 0        | 0        | 0        | 0        | 0        | 0        | 0        |
| <b>T</b> | 0 | 0        | 0        | 0        | 1        | 0        | 1        | 1        | 0        |
| <b>A</b> | 0 | 1        | 0        | 0        | 0        | 2        | 0        | 0        | 2        |
| <b>T</b> | 0 | 0        | 0        | 0        | 1        | 0        | 3        | 2        | 0        |
| <b>G</b> | 0 | 0        | 1        | 1        | 0        | 0        | 1        | 2        | 1        |
| <b>C</b> | 0 | 0        | 0        | 0        | 0        | 0        | 0        | 0        | 1        |

|          |          |          |          |          |          |          |          |
|----------|----------|----------|----------|----------|----------|----------|----------|
| <b>A</b> | <b>G</b> | <b>G</b> | <b>T</b> | <b>A</b> | <b>T</b> | <b>T</b> | <b>A</b> |
| -        | -        | <b>C</b> | <b>T</b> | <b>A</b> | <b>T</b> | <b>G</b> | <b>C</b> |

그림 3.12: 지역정렬을 이용한 서열 정렬: “AGGTATTG”와 “CTATGC” 서열

역정렬은 고정적 지역정렬 알고리즘을 프로그램 유사도 분석에 적합하게 개선한 방법으로 프로그램 집합  $\mathbb{P}$ 의 키워드 벡터  $\mathcal{K}$ 를 알고리즘에 적용한 모델이다. 그림 3.13은 적응적 지역정렬을 이용한 유사도 계산 방법을 보여준다.

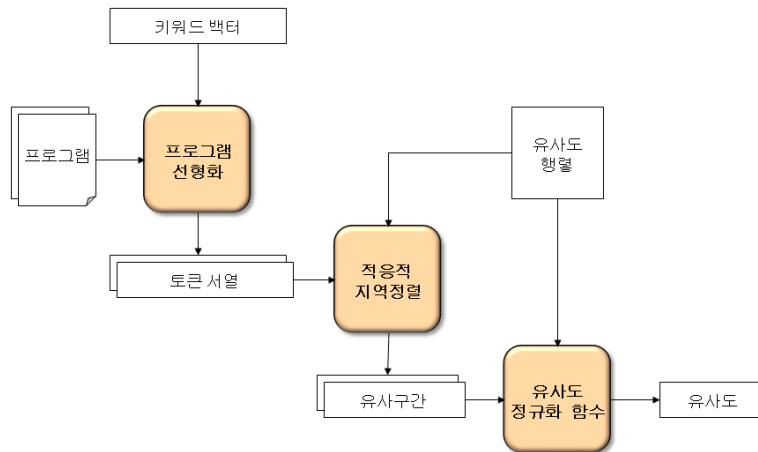


그림 3.13: 적응적 지역정렬을 이용한 프로그램 유사도 계산

앞 절에서 설명했듯이 기존의 지역정렬 알고리즘에서는 유사도 행렬의 점수를 정수로 나타낸다. 키워드  $k_i$ 와  $k_j$ 가 일치할 경우에는  $Score_L(k_i, k_j)$ 는 1, 일치하지 않으면 -1을 부여하고 있다. 갭을 이용하여 일치시켜야 하는 경우에는 -2 점을 부여한다. 이렇게 정수 단위로 유사도 행렬을 정의하는 방법은 기존 생물학적 DNA 서열에 대해 유사도를 산출할 때 사용하던 방법이다.

적응적 지역정렬 알고리즘은 특정 프로그램 집합에 따라 유사도 행렬을 적응적으로 변경한다. 프로그램 집합  $\mathbb{P}$ 에 사용된 키워드 빈도는 키워드마다 서로 다르다. 키워드 빈도는 프로그램 집합에 대해서도 확장하여 생각할 수 있다. 같은 목적으로 작성된 프로그램 그룹에 대해서 키워드 빈도는 매우 다르게 나타날 수 있는데, 적응적 지역정렬 알고리즘에서는 키워드 빈도를 바탕으로 유사도 행렬을 결정한다.

적응적 지역정렬 알고리즘에서는 빈도가 높은 키워드의 일치에 대해서는 낮은 점수를 부여하고, 빈도가 낮은 키워드의 일치에 대해서는 높은 점수를 부여한다. 예를 들어, 어떤 프로그램  $p_a$ 에서 사용 빈도가 매우 낮은 키워드  $k$ 를 사용하였다고 할 때, 다른 프로그램  $p_b$ 에 같은 키워드가 사용될 확률은 매우 낮다. 그러므로 이렇게 매우 낮은 빈도를 보이는 키워드를 함께 사용한 프로그램에 대해서는 상대적으로 높은 유사도가 있다고 볼 수 있다. 따라서 키워드 빈도에 따라 유사도 행렬의 점수를 변경하는 것이 자연스럽다.

적응적으로 유사도 행렬을 결정하기 위해서는 먼저 키워드 빈도를 산출해야 한다.  $n$ 개의 프로그램으로 구성된 프로그램 그룹  $\mathbb{P} = \{p_1, p_2, \dots, p_n\}$ 가 있다고 할 때, 키워드 빈도 벡터는 다음과 같이 정의된다.

**정의 7.** 키워드 빈도 벡터  $f^{\mathbb{P}} = \langle f_1^{\mathbb{P}}, f_2^{\mathbb{P}}, \dots, f_r^{\mathbb{P}} \rangle$ 는 각 키워드  $k_i$ 가 해당 프로그램 그룹 내의 모든 소스코드에서 사용된 횟수로부터 결정된다. 키워드  $k_i$ 가 프로그램 그룹  $\mathbb{P}$ 에서 사용된 횟수를  $occur(\mathbb{P}, k_i)$ 라고 하면,  $k_i$ 가 사용된 빈도  $f_i^{\mathbb{P}}$ 는 다음과 같이 정의한다.

$$f_i^{\mathbb{P}} = occur(\mathbb{P}, k_i) / \sum_{j=1}^r occur(\mathbb{P}, k_j) \quad (3.4)$$

정의 7에 따르면 임의의 키워드  $k_i$ 의 빈도  $f_i^{\mathbb{P}}$ 는 0이상 1이하의 값( $0 \leq f_i^{\mathbb{P}} \leq 1$ )이며, 키워드 벡터의 모든 원소 합은 1이다( $\sum_{i=1}^r f_i^{\mathbb{P}} = 1$ ).

유사도 행렬은 키워드 빈도에 따라 적응적으로 정한다. 본 논문에서는 두 키워드의 빈도 곱의 로그 값으로 유사도 점수를 정했는데, 어떤 신호값(signal)의 세기(intensity)를 상대적으로 비교하기 위하여 그 값에 로그를 취한 값(log odd)를 취하는 것은 공학에서 흔히 이용하는 방법이다. 예를 들어 마이크로어레이 실험에서 각 스팟별 신호의 강도를 상대적으로 비교하는 것 역시 그 상대값의 비율에 로그를 취한 것을 사용한다. 이에 대한 이론적인 배경은 없지만 실험자들의 실험과정에서 확립된 이론이다. 다시 말해서 어떤 특정신호 값이  $p$ 이고 다른 신호값이  $q$ 라면 두 신호의 상대적인 차이는  $p/q$ 가 아니라  $\log(p/q)$ 로 계산한다.

실제로 로그를 취한 값을 사용할 것인지 아닌지는 실험에 따라서 달라져야 하겠지만 프로그램 유사도 검출의 경우에는 실험결과 로그를 취한 값을 사용하는 것이 바람직하다고 판단하였다. 왜냐하면 키워드 빈도의 차이가 크므로 키워드의 상대적인 비율을 그대로 매칭 가중치에 반영하는 것은 바람직하지 못하기 때문이다. 예컨대, 빈번하게 나타나는 ‘=’와 같은 키워드는 약 17%임에 비해서 잘 나타나지 않는 특별한 연산자들은 약 0.003%(대략 1000개당 3개 정도)이다. 이 값의 비율을 직접적으로 비교하면 약 5000배에 해당하므로 이 값을 그대로 쓰는 것은 거의 의미가 없다고 할 수 있다. 따라서 적응적 지역정렬에서는 그 출현빈도가 약 2배 차이가 날 때 실제 가중치는 1 정도 차이가 나는 것이 합리적이라 판단하여  $\log_2$ 로 계산하기로 했다.

이상에서 설명한 것과 같이 유사도 행렬의 유사도 값은 키워드 곱의 로그 값으로 정한다. 구체적으로 말해서, 키워드  $k_i$ 와  $k_j$ 가 일치할 때의 점수는 음의 로그로 유사도 점수를 나타내고, 불일치일 경우에는 양의 로그로 벌점을 나타낸다. 키워드 빈도는 0과 1사이의 수치이므로 결과적으로 일치할 경우에는 양의 값을, 불일치할 경우에는 음의 값을 갖게 된다. 적응적 유사도 정렬에서 사용하는 유사도 행렬은 다음과 같이 정의된다.

정의 8. 프로그램 그룹  $\mathbb{P}$ 에 대한 적응적 유사도 정렬에 사용할 유사도 행렬 함수  $Score_{AM}^{\mathbb{P}}$ 는 키워드 빈도 벡터  $\mathcal{K}$ 를 이용하여 다음과 같이 정의된다.

$$Score_{AM}^{\mathbb{P}}(k_i, k_j) = \begin{cases} -\alpha \log_2 f_i^{\mathbb{P}} f_j^{\mathbb{P}} & k_i = k_j \\ \beta \log_2 f_i^{\mathbb{P}} f_j^{\mathbb{P}} & k_i \neq k_j \\ 4\beta \log_2 f^{\mathbb{P}} & \text{공백이 아닌 키워드의 빈도가 } f^{\mathbb{P}} \text{ 일 때} \end{cases} \quad (3.5)$$

여기서  $\alpha$ 와  $\beta$ 의 합은 1로 정한다( $\alpha + \beta = 1$ ). 위와 같이 유사도 행렬을 결정하면 일치와 불일치에 대한 상대적 가중치를 매개변수  $\alpha$ 와  $\beta$ 로 조정할 수 있다는 장점이 있다.

본 논문에서 제안하는 적응적 지역정렬 알고리즘은 1972년에 Karen에 의해 제안된 TF/IDF(Term Frequency and Inverse Document Frequency)[73] 모델에 기반하고 있다. TF/IDF 모델은 패턴인식 및 정보검색, 텍스트 마이닝 등 많은 분야에서 사용되고 있다[74, 75]. TF/IDF는 문서집합에서 쿼리 문자열에 대한 단어 벡터와 이것을 포함하고 있는 문서의 개수에 기반한 통계적 측정법이다[76]. 일반적으로 많은 소스코드에 공통적으로 사용되는 키워드들은 유사도 분석에서 좋은 구분 요소가 되지 못하므로 이들 키워드들에 대해 낮은 점수를 준다. 그러면, 프로그램에 필수적으로 사용되어야 하는 높은 빈도의 키워드들에 대해서 유사도 계산에서 무시할 수 있어 분석의 정확성을 향상시킬 수 있다.

키워드(예를 들어, ‘=’)들의 일치에 대해서는 낮은 점수를 부여하고, 프로그램에서 자주 사용되지 않는 키워드(예를 들어, ‘if’, ‘switch’)들에 대해서는 높은 점수를 할당한다. 불일치나 겹에 의한 감점(penalty)에 대해서도 동일하다. 실험결과 적응적 지역정렬은 고정적 지역정렬에 비해 ‘의미 없는 코드 삽입’ 및 ‘키워드 수정’ 등과 같은 표절에 대해 더욱 견고하게 유사 코드를 찾을 수 있었다. 표 3.1는 실험에 사용된 한 프로그램 그룹의 키워드 출현빈도를 보여준다. ICPC06-C 프로그램 그룹에서는 총 9993개의 키워드가 사용되었다. 표 3.1에서 많이 사용되지 않은 ‘switch’ 키워드의 정렬 점수는 빈번하게 사용된 ‘=’ 키워드에

비해 빈도가 약 4배 더 높다.

표 3.1: ICPC06-C 프로그램 그룹에 사용된 키워드 빈도

| 높은 빈도 키워드   |      |        | 낮은 빈도 키워드    |          |       |
|-------------|------|--------|--------------|----------|-------|
|             |      | 빈도     |              |          | 빈도    |
| ASSIGN_EQ   | “=”  | 12.64% | STRUCT       | “struct” | 0.01% |
| BLOCK_START | “{”  | 10.26% | DELETE       | “delete” | 0.01% |
| BLOCK_END   | “}”  | 10.26% | BOOL         | “bool”   | 0.02% |
| EQ          | “==” | 6.40%  | MINUS_ASSIGN | “-=”     | 0.02% |
| IF          | “if” | 6.28%  | SWITCH       | “switch” | 0.02% |

본 논문에서는 추가로 정의 8의 적응적 지역정렬의 유사도 행렬을 비대칭 함수로 확장한다. 비대칭 유사도 행렬은 두 프로그램  $P_a$ 와  $P_b$ 의 유사도를 계산에서  $P_a$ 를 원본으로  $P_b$ 가 유사하게 작성되었을 경우와 반대의 경우에 대해 유사도가 서로 다르다. 비대칭 유사도 행렬  $Score_{Asym}^{\mathbb{P}}$ 는 다음과 같이 정의된다.

정의 9. 비대칭 유사도 행렬  $Score_{Asym}^{\mathbb{P}}$ 는 공백 삽입 및 삭제에 대해 서로 다른 가중치를 주어 두 프로그램  $P_a$ 와  $P_b$  사이의 유사도를 서로 다르게 계산한다.

$$Score_{Asym}^{\mathbb{P}}(k_i, k_j) = \begin{cases} -\alpha \log_2 f_i^{\mathbb{P}} f_j^{\mathbb{P}} & k_i = k_j \\ \beta \log_2 f_i^{\mathbb{P}} f_j^{\mathbb{P}} & k_i \neq k_j \\ \gamma \log_2 f_i^{\mathbb{P}} & \text{공백}(gap) \text{ 삽입} \\ \delta \log_2 f_j^{\mathbb{P}} & \text{공백}(gap) \text{ 삭제} \end{cases} \quad (3.6)$$

본 논문에서는  $Score_{Asym}^{\mathbb{P}}$ 를 사용하는 유사도 함수를  $Asym(P_a, P_b)$ 로 정의한다.  $Asym()$  지역정렬 방법은 두 가지 특징이 있다. 첫 번째는 적응적 점수 행렬을 사용한다는 것이고, 두 번째는 4가지 제어변수를 사용한다는 것이다. 이로 인하여, 비대칭 거리 측정법인  $Asym()$ 은,  $P_b$ 를 기준으로 하였을 때  $P_a$ 와의 유사성과  $P_a$ 를 기준으로 하였을 때  $P_b$ 와의 유사성이 다르다는 특징이 있다( $Asym(P_a, P_b) \neq Asym(P_b, P_a)$ ).

### 3.4 유사도 정규화

유사도 행렬  $Score_{AM}^{\mathbb{P}}$ 은 두 프로그램에서 유사하다고 발견된 유사 구간의 유사도 점수를 산출하는데 사용된다. 두 프로그램의 토큰 서열에 대해 유사한 구간(aligned subrange)이 발견되면, 위에서 정의한 유사도 행렬을 이용하여 유사도 점수를 산출한다. 두 프로그램  $A$ 와  $B$ 를 정렬하여 유사구간을 산출하는 함수를 *align*이라고 하고 이 함수가 유사구간 쌍  $a_1, a_2, \dots, a_m, b_1, b_2, \dots, b_m$ 에 대해 다음과 같이 키워드 쌍의 벡터 형태로 유사구간을 산출한다고 하자.

$$align(A, B) = \langle (a_1, b_1), (a_2, b_2), \dots, (a_m, b_m) \rangle$$

이 때,  $align(A, B)$ 는 두 프로그램으로부터 추출된 서열들 사이의 유사구간을 나타낸다. 그러므로 서열의 각 유사구간들은 실제 소스코드 사이의 유사구간인 정의 2의  $C(A, B) = \{C_1, C_2, \dots, C_n\}$ 으로 연결된다( $C_n = \langle C(S_A, L_i, L_j), C(S_B, L_k, L_l) \rangle$ ). 그리고 이 구간의 유사도 점수는 다음 함수로 정의된다.

$$SIM_{abs}(A, B) = \sum_{(a,b) \in align(A,B)} Score_{AM}^{\mathbb{P}}(a, b) \quad (3.7)$$

이는 정수 단위의 유사도 행렬을 이용하는 기존 방법에서의 유사도 산출 방법과 거의 동일하다.

유사도 점수  $SIM_{abs}$ 를 0 ~ 100% 구간으로 정규화한 것을 유사도라고 한다. 앞서 기술한 유사도는 유사구간의 점수를 합한 것이기 때문에 프로그램 길이에 비례하는 경향을 보인다. 따라서 정확히 비교하기 위해서는 정규화 과정이 필요한데, 정규화된 유사도를 산출하는 방식은 두 가지가 있다. 일반적으로 사용되는 방식은 두 프로그램의 유사도 점수를 각 프로그램에 대하여 자신과 정렬시킨 경우, 즉 최대 유사도 점수가 나오게 되는 경우의 유사도 점수의 합으로 나누는 방식이다. 이 방식의 유사도를  $SIM_{sum}(A, B)$ 의 유사도 산출식은 다음과 같다.

$$SIM_{sum}(A,B) = \frac{2SIM_{abs}(A,B)}{SIM_{abs}(A,A) + SIM_{abs}(B,B)} \quad (3.8)$$

유사도 점수를 정규화하는 다른 방법으로서  $SIM_{abs}(A,A)$ 와  $SIM_{abs}(B,B)$  중 작은 값으로 나누는 방식이 있다. 비교할 두 프로그램 중에서 어느 하나의 프로그램 길이가 매우 큰 경우  $SIM_{sum}$ 의 값은 매우 낮아질 수 있다. 길이가 매우 큰 프로그램(예컨대, A)에 대하여 자신과의 유사도 점수( $SIM_{abs}(A,A)$ )가 매우 크게 되기 때문이다. 작은 점수를 기준으로 정규화한 유사도를  $SIM_{min}(A,B)$ 로 정의한다.

$$SIM_{min}(A,B) = \frac{SIM_{abs}(A,B)}{\min\{SIM_{abs}(A,A), SIM_{abs}(B,B)\}} \quad (3.9)$$

다음 장에서는 이상에서 기술한 적응적 유사도를 이용하여 유사하게 작성된 소스코드들을 구별하는 소스코드 클러스터링 분석과 생물정보학에서 DNA 서열이나 종들의 진화 분석처럼 소스코드를 수정함에 따른 프로그램 진화과정을 유추하는 것에 적응적 지역정렬을 적용한다.

## 제 4장

# 클러스터링과 진화분석

프로그램 유사도 분석을 위한 적응적 프레임워크는 프로그램 표절 검사 시스템 구현에 직접적으로 사용될 수 있다. 지금까지 제안된 프로그램 표절 탐색 연들은 주로 두 프로그램 사이의 유사도 계산에 초점을 맞추었다. 하지만, 프로그램 표절 연구에서는 기본적으로 해결해야 하는 문제들이 있다. 첫 번째로 두 프로그램이 유사하게 작성되었을 때, 이들이 실제로 악의적으로 표절을 한 것인지, 또는 여러 가지 제약사항들로 인해 우연히 같게 작성된 것인지 판단할 수 있어야 한다. 두 번째로 표절한 두 프로그램 사이에 원본과 복사본을 찾아내는 문제이다. 이에 본 논문에서는 위의 두 가지 문제에 대해 극단치 분포 확률 모델에 기반한 클러스터링 알고리즘과 비대칭 지역정렬에 기반한 프로그램 진화트리 생성 알고리즘을 제안한다. 본 절에서는 소스코드 클러스터링 알고리즘과 진화트리 생성 알고리즘에 대해 기술한다.

### 4.1 클러스터 분석을 위한 확률 모델

본 절에서는 프로그램 표절 연구에서 기본적으로 해결해야 하는 표절에 대한 객관적 판정

문제에 대해 다룬다. 예를 들어, 어떤 두 프로그램이 독립적으로 작성되었다고 하더라도 거의 같은 가능성이 있다. 특히, 문제 해결 방법이 제한적으로 주어졌을 경우 그 사항을 충실히 따른 프로그램들은 거의 같을 수밖에 없다. 본 절에서는 어떤 길이의 프로그램 A가 다른 프로그램 B와 표절없이 거의 유사할 경우에 대한 확률모델을 제시하고 그에 근거하여 표절 프로그램들을 판정하는 클러스터링 알고리즘을 제시한다.

#### 4.1.1 의사표절 정의

프로그램 표절 검사 시스템은 주로 C언어나 C++, 자바 등을 이용한 프로그래밍 강의에서 활용된다. 다음은 프로그래밍 강의에서 빈번하게 일어날 수 있는 상황이다. 예를 들어, 프로그래밍 과제를 수행할 때 다음과 같은 요구사항들이 주어진다고 가정한다. 프로그래머는 코드를 작성할 때, 특정 자료구조 또는 템플릿 코드(template code)를 사용해야 하며, 프로그램 수행시간 및 실행파일 크기가 기준치 이하여야 한다. 이와 같은 제약사항들로 인해 프로그램의 자료구조와 알고리즘들이 유사하게 작성될 수 있다.

학생들에게 거품 정렬(bubble sorting) 알고리즘을 이용해 정렬 프로그램을 작성하도록 하면, 프로그램이 서로 독립적으로 작성되었지만, 모든 프로그램들 사이의 유사도는 매우 높아지게 된다. 이와 같은 현상은 프로그래밍 대회(programming contest)에서 빈번하게 나타난다. 한 예로 ICPC 지역 예선전에서 1번과 2번 문제는 학생들이 10분 이내에 직관적으로 쉽게 작성할 수 있었으며, 모든 소스코드는 100라인 이하로 작성되었다. 이와 같은 경우, 두 프로그램의 유사도가 매우 높다고 하여 두 프로그램 사이에 표절을 했다고 단정 지을 수 없다. 이와 같은 표절의 종류는 이전의 연구에서는 정의된 바가 없다. 본 논문에서는 새로운 두 종류의 표절형태인 의사표절(pseudo plagiarism)과 실제표절(real plagiarism)을 정의한다.

**정의 10. 실제표절(real plagiarism)**은 프로그래머에 의해 악의적인 의도로 표절을 한 경우를 말한다. **의사표절(pseudo plagiarism)**은 프로그램에 주어진 제약사항들로 인해 독립적으

로 작성되었지만 두 프로그램 사이의 유사도가 매우 높아 표절로 탐색되는 경우이다.

의사표절을 찾아내는 것이 표절검사에서 중요한 문제가 된다. 실험을 통하여 ICPC 동아시아 결선 대회에서 인터넷 사용 금지 및 참가자들의 이동 금지, 정해진 시간 내에 프로그램 제출 등과 같은 제약적인 환경인 경우 프로그램들 사이의 ( $Asym()$ )값의 분포는 굼벨 분포 함수를 따른다는 것을 발견할 수 있었다. 생물학적 서열 검색에서 널리 이용되는 굼벨 분포는 독립된 두 프로그램  $P_a$ 와  $P_b$ 의 유사도 ( $Asym()$ ) 값의 확률을 계산하기 위한 좋은 모델이 된다.

지역정렬 점수의 분포에 대한 이해는 신뢰할 수 있는 생물학적 서열(DNA, RNA, Protein 등) 탐색에 있어 중요하기 때문에 오래전부터 연구가 진행되어왔다[77, 78]. 공백을 사용하지 않는(ungapped) 지역정렬의 점수는 이미 극단치 분포 중의 하나인 굼벨 분포(Gumbel Distribution)의 특징이 있다는 것은 연구를 통해 밝혀졌다[79]. 하지만 갭을 사용하는(gapped) 지역정렬의 점수가 굼벨 분포의 특징을 따르는지는 명확하게 밝혀지지 않았다. 단지, 실험을 통해 갭을 사용하는 지역정렬도 파라미터 조정을 통해 굼벨 분포의 형태를 따른다는 것이 알려져 있다. 굼벨 분포의 확률 밀도 함수  $Gumbel(x; \mu, \rho)$ 과 두 개의 조정 파라미터  $\mu$ 와  $\rho$ 는 다음과 같다.

$$Gumbel(x; \rho, \mu) = \frac{1}{\rho} \cdot \exp\left(\frac{-(x-\mu)}{\rho}\right) \cdot \exp\left(-\exp\left(\frac{-(x-\mu)}{\rho}\right)\right), \quad (4.1)$$

$$\text{where } \rho = \sqrt{6} \cdot d_{\mathbb{P}} / \pi, \quad \mu = a_{\mathbb{P}} - \rho \cdot \gamma$$

위 식에서  $a_{\mathbb{P}}$ 는 평균을 나타내고,  $d_{\mathbb{P}}$ 는 집합  $\mathbb{P}$ 의 유사도 점수들 사이의 표준편차를 나타낸다. 그리고  $\gamma$ 는 Euler-Mascheroni 상수이다( $\gamma = 0.57721\dots$ ). 본 연구에서는 두 프로그램의 유사도가  $p\%$  이면 이것이 우연히 일어날 가능성(p-value)를 굼벨 모델을 이용하여 계산하고 그 값으로 두 개체의 표절 가능성을 확률 값으로 제시한다. 그림 4.1은 굼벨 분포와 이에 대한 2006년도 KOI 대회에 제출된 프로그램 그룹의  $Asym()$  값 분포를 보여준다.

그림 4.1에서와 같이, 만약 프로그램 유사도가 매우 높거나 유사도 값들이 평균 유사도

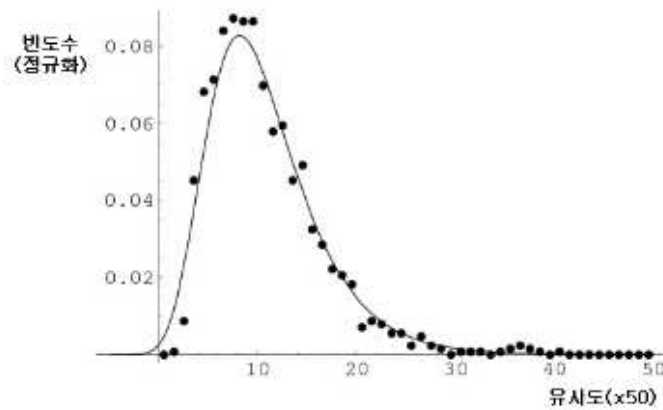


그림 4.1: 2006년 KOI의 프로그램 그룹에 대한  $Asym()$  분포(점): 굼벨 분포(실선)와 유사

분포에 비해 매우 좁은 간격으로 분포될 경우에 이 분포는 굼벨 분포를 따르지 않는다. 그림 4.2는 ICPC05-A 프로그램 그룹에 대한  $Asym()$  분포로 굼벨 분포와 다른 경우를 보여준다. 또한, 프로그램에 대해 기능적 요구사항이 매우 강하거나 구체적일 경우에도 굼벨 분포를 벗어날 수 있다. 이 경우는 의사표질이 일어나는 특징을 나타내는 전형적인 상황이다.

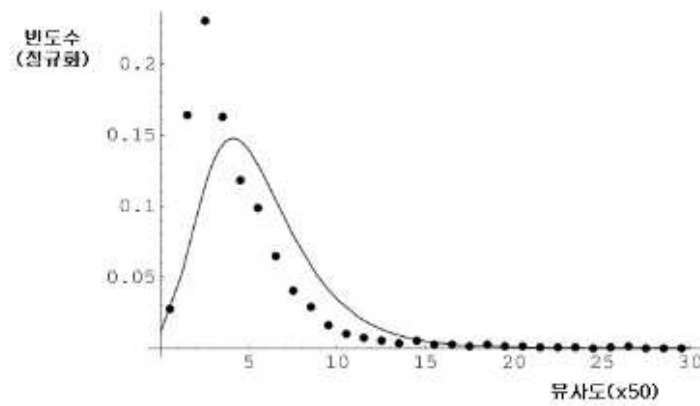


그림 4.2: ICPC05-A 그룹에 대한  $Asym()$  분포(점): 굼벨분포를 많이 벗어남

#### 4.1.2 클러스터링 알고리즘

본 절에서는 프로그램 그룹에 대해 표절된 프로그램 검출을 위한 새로운 알고리즘을 제안한다. 먼저, 표절검사 알고리즘에서는 클러스터링을 통해 의사표절과 실제표절을 구분한다. 다음으로 표절된 프로그램 쌍들이 있을 경우 사용자에게 이들을 출력한다. 지금부터 알고리즘 설명을 위한 몇 가지 사전 정의를 한다.

**정의 11.**  $pdist(p_a, p_b | \mathbb{P})$  - 적응적 점수 행렬  $Score_{Asym}^{\mathbb{P}}$ 를 이용해 표절거리(plagiarized distance)를 계산하는 정규화 함수( $p_a, p_b \in \mathbb{P}$ ). 여기서  $pdist(p_a, p_b)$ 는 다음과 같이 계산된다.

$$pdist(p_a, p_b | \mathbb{P}) = 1 - Asym(p_a, p_b) / Asym(p_b, p_b)$$

$Asym(p_a, p_b)$ 는  $Asym(p_b, p_b)$ 보다 클 수 없기 때문에  $pdist(p_a, p_b)$ 의 값은 항상 1보다 작다( $0 \leq pdist(p_a, p_b | \mathbb{P}) \leq 1$ ). 그리고  $pdist(p_a, p_b | \mathbb{P})$ 의 값이 0일 때, 두 프로그램이 거의 유사함을 의미한다( $p_a \equiv p_b$ ). 또한,  $pdist(p_a, p_b | \mathbb{P})$ 는 비대칭 함수이다( $pdist(p_a, p_b | \mathbb{P}) \neq pdist(p_b, p_a | \mathbb{P})$ ,  $pdist(p_a, p_b | \emptyset) \neq pdist(p_a, p_b | \mathbb{P})$ ). 프로그램 그룹  $\mathbb{P}$ 가 고정된 경우에는 편의상  $pdist(p_a, p_b | \mathbb{P})$ 를  $pdist(p_a, p_b)$ 로 표시한다. 다음으로  $pdist$ 를 이용하여 표절방향그래프 PDG를 정의한다.

**정의 12.**  $PDG(V, E)$  - PDG 함수는 표절방향그래프(Plagiarism Direction Graph)로 프로그램 그룹  $\mathbb{P}$ 와  $Asym()$  함수로부터 구축된다. PDG 함수에서 정점( $V$ )은 프로그램을 나타낸다( $p_i \in \mathbb{P}$ ). 그리고 프로그램  $p_a$ 와  $p_b$ 로 이루어진 각 프로그램 쌍들은 하나의 방향성 있는 간선( $E$ )로 나타낸다.

PDG의 모든 간선들은 방향성과 무게(거리) 값으로 표시된다. 본 논문에서는 의 각각의 간선들의 무게를 다음과 같이 정의한다.

**정의 13.**  $pdist(p_a, p_b) > pdist(p_b, p_a)$ 의 경우, 간선 방향은  $(\overrightarrow{p_b, p_a})$ 로 표시하고, 반대의 경우인  $pdist(p_a, p_b) \leq pdist(p_b, p_a)$ 는  $(\overrightarrow{p_a, p_b})$ 로 표시한다. PDG 간선의 무게는  $w(p_u, p_v) =$

$\min\{pdist(p_u, p_v), pdist(p_v, p_u)\}$  이다.

예를 들어,  $pdist(p_a, p_b) = 0.2$ 와  $pdist(p_b, p_a) = 0.5$ 인 경우에 프로그램  $p_b$ 에서 프로그램  $p_a$  방향으로 표절이 발생한 것보다  $p_a$ 에서  $p_b$ 방향으로 표절이 발생했을 확률이 높다고 가정한다. 그 이유는 일반적으로 표절된 프로그램들은 유사도가 더욱 높아지기 때문이다. PDG는 완전 그래프(complete graph)이기 때문에, 그래프로부터 의미 있는 클러스터를 생성하기 위해 표절거리를 이용해 간선들을 지우는 작업을 수행한다. PDG로부터 간선의 무게를 기준으로 임계값(threshold value)보다 큰 간선들을 삭제하여 새로운 서브 그래프인  $PDG_c$ 를 생성한다.

**정의 14.**  $PDG_c(V, E)$  - PDG로부터 생성된 서브 그래프,  $PDG_c$ 는 다음과 같이 생성된다.  $PDG_c(V, E) = PDG(V, E) - \{(p_i, p_j) | pdist(p_i, p_j)D > c\}$ , 여기서  $c$ 는 임계값을 나타낸다.

$PDG_c(V, E)$ 는 간선의 무게가  $c$ 보다 큰 간선들을 제거한다. PDG는 프로그램 그룹의 유사도를 간선으로 나타내지만, 프로그램 그룹의 유사도 분포 특징을 반영하지 못한다. 본 논문에서는 PDG 단점을 보완하기 위하여 굽벨 분포를 유사도 분석에 적용한다. PDG에 굽벨 분포를 적용하기 위해 다음을 정의한다.

**정의 15.**  $gdist(p_a, p_b | \mathbb{P})$  -  $p_a$ 와  $p_b$  사이의 굽벨 거리로  $gdist(p_a, p_b | \mathbb{P}) = \int_c^1 G(x; \mu_{\mathbb{P}}, \rho_{\mathbb{P}}) dx$ 는 확률밀도 함수이고,  $c = Asym(p_a, p_b)$ 와 같이 정의한다.

**정의 16.**  $GDG(V, E)$  - 굽벨 방향 그래프로 PDG와 동일한 방법으로 생성된다( $GDG(V) = PDG(V)$ 와  $GDG(E) = PDG(E)$ ).  $GDG(E)$ 의 간선 무게는  $gdist(p_a, p_b | \mathbb{P})$  이다.  $GDG_c(V, E)$ 는  $PDG_c(V, E)$ 와 동일하게 정의된다.

마지막으로  $GDG_c(V, E)$ 를 이용해 표절된 프로그램 쌍들을 분류하는 최대 표절 컴포넌트  $MPC_c(V, E)$ 를 정의한다.

**정의 17.**  $MPC_c(V, E)$  -  $GDG_c(V, E)$ 의 최대 표절 컴포넌트(Maximal Plagiarism Component)로  $GDG_c(V, E)$ 에서 정점이 가장 많은 컴포넌트를 의미한다.

$gdist(a,b)$ 와  $pdist(a,b)$ 는 모두  $0 \sim 1$  사이로 정규화된 거리 메트릭이다. 그러나  $gdist$ 는 파라미터들을 통해 프로그램 그룹의 유사도 분포 특징들까지 정규화에 반영할 수 있다.

실험결과를 통해 분석한 결과  $gdist(p_a, p_b | \mathbb{P}) \leq 0.001$ 에 대해  $p_a$ 와  $p_b$ 에 대해 실제로 표절이 있는 경우로 밝혀졌다[80]. 이 경우는  $GDG(E)$ 의 모든 간선 무게가 0.001보다 작은 경우 표절했을 확률이 매우 높음을 의미한다. 또한, 실험결과 10명 이상의 참가자들이 표절에 연관되기는 매우 힘든 것으로 나타났다[80]. 다시 말해서,  $|MPC_{0.001}(V, E)| \geq 10$ 인 경우는 의사표절이 발생했을 가능성이 매우 높다. 그림 4.3은 본 논문에서 정의한 의사표절과 실제표절 구분을 위한 클러스터링 알고리즘이다.

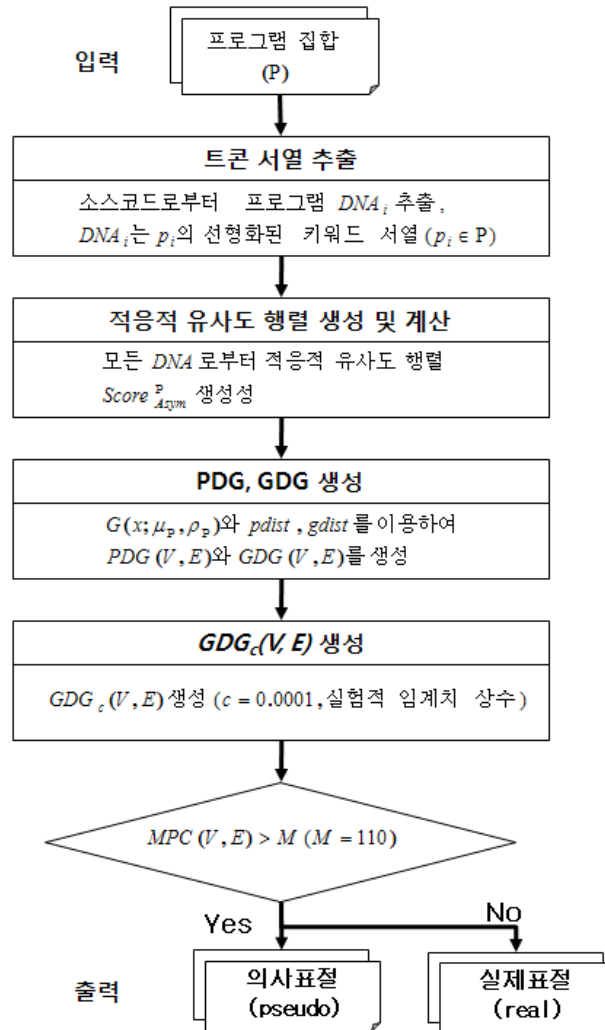


그림 4.3: 의사표절 탐색을 위한 클러스터링 알고리즘

## 4.2 프로그램 진화분석

본 절에서는 프로그램 유사도 분석 프레임워크를 이용한 프로그램 진화분석(Evolution Analysis) 방법에 대해 기술한다. 프로그램 진화분석은 생물 정보학 등에서 연구되고 있는 개체의 진화론적 분석 방법과 유사하게 프로그램 개체의 진화과정을 분석한 것이다. 여기서 프로그램 개체는 소스코드에 해당한다. 프로그램 소스코드는 처음 작성된 후 요구사항 수정 및 기능 명세의 변경, 버그 수정 등으로 인해 소스코드가 추가되거나 삭제되면서 진화해 간다. 프로그램 진화분석은 소스코드도 생물학적 개체와 같이 진화된다는 가정 아래 소스코드의 변화를 추적하는 것이다.

본 논문에서는 프로그램 유사도에 기반한 프로그램 진화과정을 분석한다. 본 논문의 분석법은 유지보수와 프로그램 품질 검사에서 활용될 수 있다. 프로그램 유지보수는 기존의 연구들에서와 마찬가지로 프로그램이 발전하면서 수정되거나 추가되는 코드를 찾는 것이다. 프로그램 품질 검사에서의 진화과정 분석은 더욱 중요하다. 대부분의 품질 연구에서는 표절된 프로그램 쌍들을 찾는 것에만 초점을 맞추지만 정작 품질 연구에서 중요한 요소는 원본과 복사본을 구별하는 것이다. 예를 들어, 두 프로그램  $P_a$ 와  $P_b$ 가 유사하게 작성되었을 때,  $P_a$ 를 원본으로 하여  $P_b$ 가 작성되었는지( $P_a \rightarrow P_b$ ),  $P_b$ 를 원본으로  $P_a$ 가 표절했는지 찾는 것 또한 중요하다. 본 절에서는 다음의 두 가지 문제에 대해 다룬다.

**문제 1:** 소스코드 사이의 진화거리 계산

**입력:** 두 프로그램  $P_a$ 와  $P_b$ 에서 프로그램  $P_b$ 가  $P_a$ 로부터 작성되었을 경우, 진화방향을 유추하기 위해 프로그램  $P_a$ 를 기준으로 계산한 유사도와 프로그램  $P_b$ 를 기준으로 계산한 유사도는 달라야 한다.

**출력:** 두 종류의 비대칭 유사도( $P_a \rightarrow P_b$ 와  $P_b \rightarrow P_a$ )

**문제 2:** 동종의 소스코드에 대한 계통분석도 구축

**입력:** 하나의 원본 코드로부터 파생된 소스코드들의 집합. 예를 들어, 프로그래밍 경진대회에 한 사람이 문제를 풀기 위해 수정하면서 여러번 제출한 소스코드

출력: 입력 소스코드들의 유도 이력(derivation history) 계통도(phylogenetic forest).

#### 4.2.1 프로그램 진화거리

본 논문에서 제안하는 프로그램 진화분석은 적응적 점수 기반 비대칭 유사도 행렬  $Asym()$ 을 기본 함수로 사용한다. 본 논문의 프로그램 진화분석에서는  $Asym$  함수의  $\gamma$  값이  $\delta$  값보다 같거나 작다고 가정했다( $\gamma \leq \delta$ ). 그 이유는 일반적으로 프로그램을 수정할 때 프로그램에 키워드를 추가하는 것보다 기존에 작성된 프로그램에서 삭제하는 것이 더욱 어려운 작업이기 때문이다. 표절의 경우 구문삭제로 인해 프로그램이 오동작할 수 있기 때문에 신중을 기해야 한다.

비대칭 유사도 함수  $Asym()$ 을 이용해 다음의 진화거리(evolutionary distance)를 정의한다. 두 프로그램  $P_a$ 와  $P_b$ 의 진화거리  $EvolDist(P_a, P_b)$ 는 정규화된 유사도를 통해 계산된다. 만약  $P_b$ 를 기준으로 두 프로그램의 정규화된 유사도  $Asym(P_a, P_b)$ 는 다음과 같이 계산된다.

$$Asym(P_a, P_b) = \frac{2 \cdot AsymScore(P_a, P_b)}{AsymScore(P_a, P_a) + AsymScore(P_b, P_b)}$$

여기서  $Asym(P_a, P_b)$ 는 0 1의 값으로 정규화된( $Asym(P_a, P_b) \sim [0, 1]$ ). 다음으로  $Asym()$ 을 이용한 진화거리는 다음과 같이 계산된다.

$$EvolDist(P_a, P_b) = 1 - Asym(P_a, P_b)$$

$EvolDist(P_a, P_b)$ 는 프로그램이 진화하면서 프로그램 키워드의 삽입과 삭제, 수정되는 작업 정도를 평가할 수 있다. 5.6절의 실험에서는 진화분석에 사용되는 적응적 유사도 매트릭스의 일치를 비롯해 불일치, 삽입, 삭제에 대한 매개변수의 최적값을 찾는다. 이 최적값은 실험적으로 변수 값을 변경하면서 프로그램 계통도를 구축했을 때 정확도가 가장 높은 경우의 값으로 설정하였다.

$EvolDist(P_a, P_b)$ 에 사용된 비대칭 적응적 지역정렬은 기존의 고정적 지역정렬과 비교해 볼 때 두 가지의 차이가 있다. 첫 번째는 두 프로그램을 비교할 때, 비교 대상에 따라 유사도가 다르게 계산된다는 것이다( $EvolDist(P_a, P_b) \neq EvolDist(P_b, P_a)$ ). 그리고 두 번째 특성은 4개의 파라미터 무게를 다르게 적용함으로써 프로그램 집합의 특성에 따라 가중치를 다르게 줄 수 있다는 것이다.  $EvolDist$  함수는  $EvolDist(P_a, P_b) \leq EvolDist(P_b, P_a)$  인 경우 프로그램  $P_b$ 는 프로그램  $P_a$ 로부터 유도되었을 확률이 높다고 가정한다.

프로그램의 진화분석은 위의 가정을 기반으로 모든 프로그램 쌍들의  $EvolDist(p_i, p_j)$  값을 이용하여 진화 트리를 생성하는 것이다. 지금부터 프로그램 집합이 주어졌을 때 진화트리를 생성하는 알고리즘에 대해 기술한다.

#### 4.2.2 프로그램 진화 트리 생성 알고리즘

프로그램 소스코드로부터 진화트리를 생성하기 위해 프로그램 유사도와 진화방향이 결정해야 한다. 여기서 진화방향은 두 프로그램에서 원본 소스코드로부터 수정된 소스코드를 추정하는 것이다. 진화트리 생성 알고리즘의 유사도는  $EvolDist$ 를 통해 계산된다.

진화 방향은 진화거리를 통해 쉽고 빠르게 결정할 수 있다. 두 프로그램  $P_a$ 와  $P_b$ 가 주어진 경우, 먼저 두 프로그램의 진화거리  $EvolDist(P_a, P_b)$ 와  $EvolDist(P_b, P_a)$ 를 계산한다. 다음으로 진화거리가 가까운 쪽으로 방향을 결정한다. 만약  $EvolDist(P_a, P_b) \leq EvolDist(P_b, P_a)$ 인 경우 프로그램  $P_b$ 는  $P_a$ 로부터 유도된 것으로 결정한다. 즉, 진화거리가 짧은 쪽으로 진화방향을 선택한다.  $EvolDist$ 는 키워드의 삽입, 삭제, 수정의 정도를 통해 프로그램의 진화정도를 나타낸다.

진화방향은 진화거리를 기준으로 결정하는 것이 자연스럽다고 할 수 있다. 예를 들어,  $EvolDist(P_a, P_b)$ 는 프로그램  $P_b$ 가 프로그램  $P_a$ 로부터 진화된 것을 나타내는데, 이것을 간략하게 기술하면  $(\overrightarrow{P_a, P_b})$ 으로 나타낼 수 있다. 여기서 만약 진화거리 값이 0이면, 두 프로그램은 정확히 동일하다고 볼 수 있다. 이는 프로그램을 복사-붙이기만으로도 쉽게 유도할 수 있다.

프로그램 진화트리는 진화 그래프로 표현이 가능하다. 다시 말해서, 프로그램 진화트리는 프로그램의 소스코드를 노드로 하는 그래프의 부분그래프(subgraph)가 된다. 진화트리 생성은 다음의 단계로 진행된다. 먼저,  $N$ 개의 소스코드로 구성된 프로그램 집합을 입력으로 하여 유사도 분석과 같이 키워드 벡터를 추출한다. 다음으로 비대칭 적응적 유사도 행렬을 이용해 모든 프로그램 쌍들에 대해 진화거리  $EvolDist(P_i, P_j)$ 를 구한다. 다음으로 모든 프로그램 쌍들에서 진화방향을 결정하고 거리가 먼 에지들을 제거해 방향성이 있는 완전 그래프(completed graph)를 생성한다. 다음 단계에서는 최소신장트리(minimum spanning tree)알고리즘을 이용해 진화트리를 생성한다. 마지막으로 진화트리에서 여러 엔트리(multiple entries)로 구성된 노드의 에지를 제거해 여러 개의 진화트리로 구성된 최소신장숲(minimum spanning forest)을 형성한다. 최종 결과물은 프로그램 진화 그래프[81]로 구성된 진화트리 집합이다. 다음은 프로그램 진화과정을 분석하는 알고리즘을 보여준다.

---

**알고리즘 1** Phylogenetic Forest Construction Algorithm

---

```

procedure PHYLOFOREST( $V$ )
   $F \leftarrow FrequencyVector(V)$ ;
   $M \leftarrow SimilarityMatrix(F)$ ;
   $E \leftarrow \{(\vec{v_i}, \vec{v_j}) | v_i, v_j \in V, i \neq j, D(v_i, v_j) < D(v_j, v_i)\}$ ;
   $T \leftarrow MinSpanningTree(Undergraph(G(V, E)))$ ;
   $\hat{T} \leftarrow T$ 
  while  $\hat{T}$  contains a multiple-entry vertex  $v$  do
     $\{e_i\} \leftarrow$  incoming edges of  $v$ ;
     $e^* \leftarrow$  maximal weight edge in  $\{e_i\}$ ;
     $\hat{T} \leftarrow \hat{T} - e^*$ ;
  end while
  return  $\hat{T}$ ;
end procedure
function  $D(v_1, v_2)$ 
   $MaxS \leftarrow AsymScore(v_1, v_1) + AsymScore(v_2, v_2)$ ;
  return  $1 - 2 \cdot AsymScore(v_1, v_2) / MaxS$ ;
end function

```

---

알고리즘 1은 프로그램 입력  $V = v_1, v_2, \dots, v_n$ 으로부터 진화트리를 재구성한다. 위의 과정을 요약하면 진화트리  $T$ 는 그래프  $G(V, E)$ 의 최소신장트리로 에지  $E$ 는 비대칭 적응적 지

역정렬로 계산된 진화거리이다. 계통숲(phylogenetic forest)  $\hat{T}$ 은  $T$ 에서 다중 엔트리를 제거한 개선된 모형이다. 그림 4.4은 본 논문의 프로그램 진화분석 알고리즘을 그림으로 나타낸 것이다.

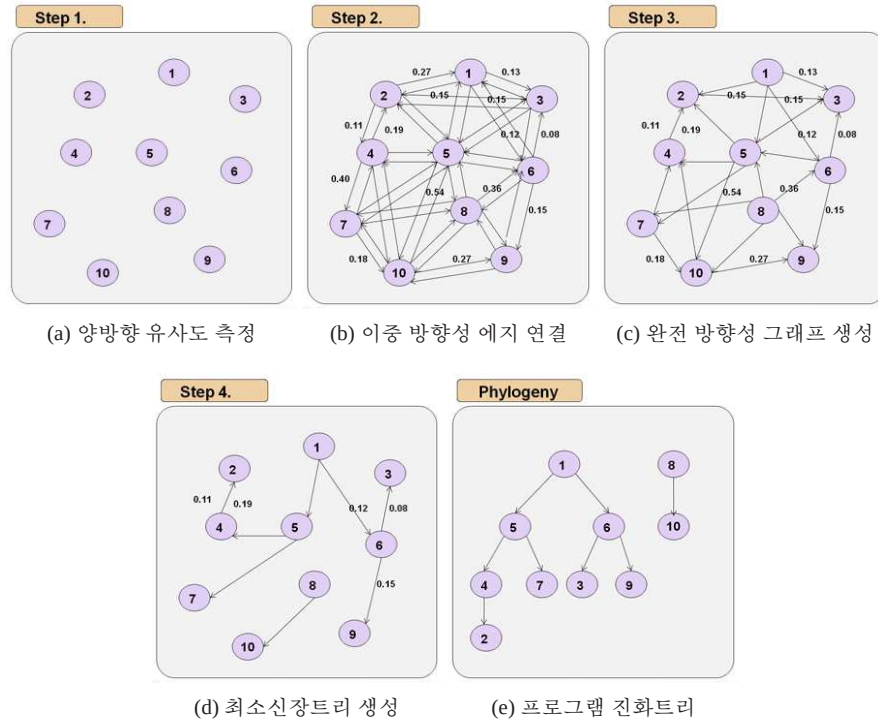


그림 4.4: 프로그램 진화트리 생성 과정

그림 4.4의 프로그램 진화분석 과정은 총 5단계로 진행된다. 먼저, 1단계에서는 프로그램 집합을 입력으로 하여 모든 프로그램 쌍들에 양방향 유사도 (*Asym()*)를 측정한다. 그림 4.4-(a)에서 노드는 프로그램 집합의 소스코드를 나타낸다. 다음으로 2단계에서는 모든 프로그램 노드를 이중의 방향성 에지로 연결된다. 이때 에지의 무게는 진화거리로 계산된다. 3단계에서는 모든 양방향 에지에서 진화거리가 짧은 에지를 선택하여 완전 방향성 그래프를 생성한다. 4단계에서는 방향성 그래프로부터 최소신장트리를 생성한다. 마지막으로 5단계에서는 신장 숲 생성 알고리즘으로 프로그램 진화트리들을 구축한다.

## 제 5장

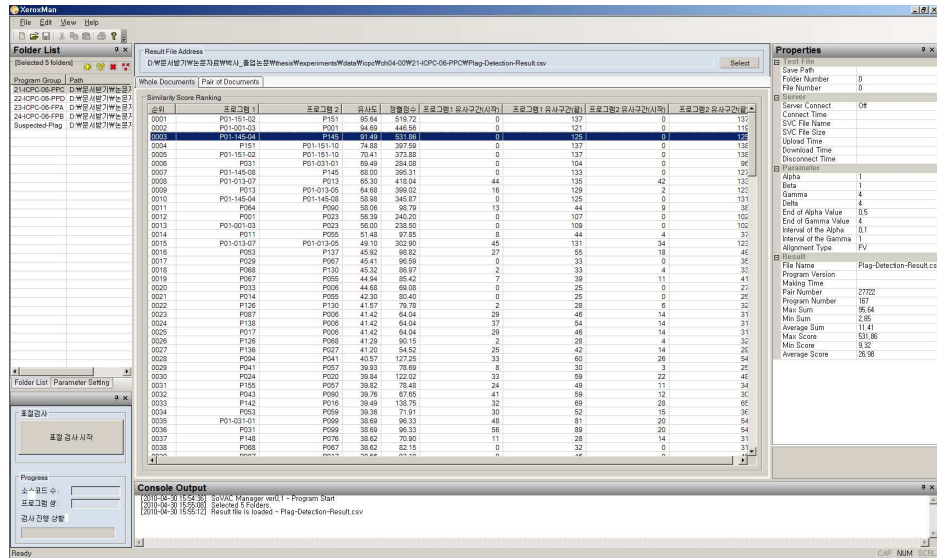
# 실험

본 장에서는 앞서 설명한 적응적 유사도 분석 프레임워크를 이용한 프로그램 유사도 분석 시스템을 구현하고 실제 프로그램에 적용한다. 그리고 실험을 통해 적응적 프레임워크의 성능을 평가한다. 실험에서는 적응적 지역정렬을 비롯해 유사도 분석 프레임워크를 구성하는 방법들의 성능을 평가한다. 실험은 크게 다음의 항목들을 평가한다.

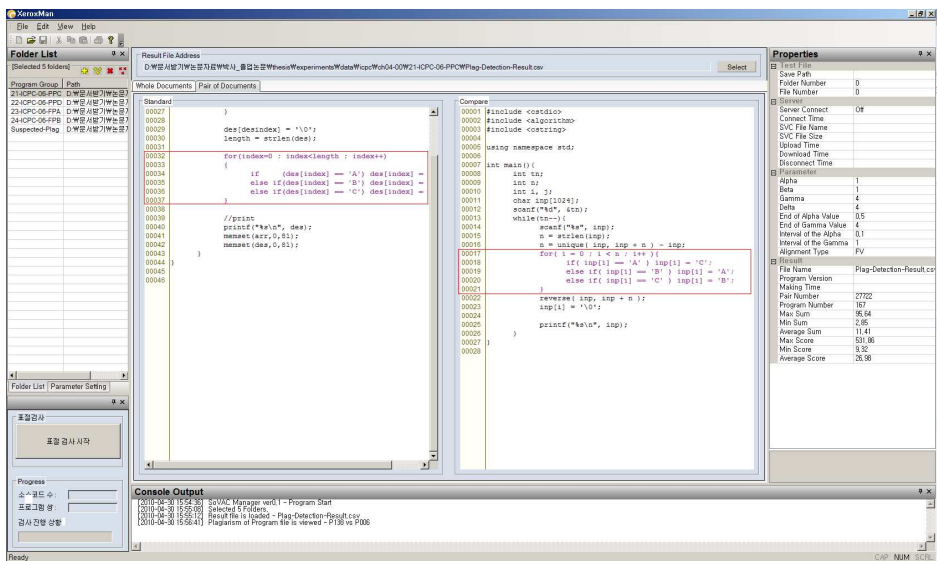
- 표절 프로그램 탐색 : 실제 프로그래밍 경시대회에 제출된 프로그램과 프로그래밍 교과목에서 학생들이 제출한 프로그램들에 대해 적응적 프레임워크를 이용해 표절을 탐색하고 효과를 살펴본다.
- 적응적 유사도 행렬 비교 : 적응적 지역정렬 알고리즘의 효과를 실험해 보기 위해 고정적 유사도 행렬(fixed similarity matrix)을 채택하였을 경우와 적응적 유사도 행렬(adaptive similarity matrix)을 채택하였을 경우에 대하여 지역정렬 알고리즘을 수행하였다
- 프레임워크 성능 평가 : 유사도 분석의 정확성을 알아보기 위해 민감도(sensitivity)와 특이도(specificity) 실험을 통해 JPlag 시스템과 성능을 비교한다.

- 프로그램 진화분석 : 프로그램 진화분석 알고리즘의 성능을 평가하고 적응적 프레임워크를 이용해 공개소스 프로그램의 버전 업에 따른 클론을 분석한다.

실험에 앞서 적응적 프레임워크를 이용한 프로그램 유사도 분석 도구 개발에 대해 설명한다. 본 논문에서는 SoVAC(Software Verification and Analysis Center)와 PlaView 프로그램 유사도 분석 시스템을 구현하였다. 본 논문에서 구현한 SoVAC 시스템은 적응적 프레임워크를 기반으로 프로그램 유사도를 분석하며, C언어와 C++, Java를 지원한다. SoVAC은 프로그램 유사도 테이블과 소스코드 뷰(view)를 지원한다. 프로그램 유사도 테이블은 각 소스코드 쌍들의 유사도와 유사구간을 테이블 형식으로 보여준다. 그리고 소스코드 뷰는 유사구간을 실제 소스코드와 연결하여 유사하게 작성된 구간을 보여준다. 그림 5.1은 SoVAC 시스템의 유사도 분석 결과를 보여준다. 그리고 그림 5.2는 PlaView에 대한 스크린 샷을 보여준다. PlaView는 적응적 프레임워크 유사도 분석 결과와 그래프 시각화 라이브러리를 이용하여 소스코드 클러스터링 및 진화분석 등 여러 가지 분석 기능을 제공하고 있다. 이처럼 그래프 시각화 도구와 같은 외부 분석 도구를 함께 이용하면 시스템을 다양하게 확장할 수 있다.

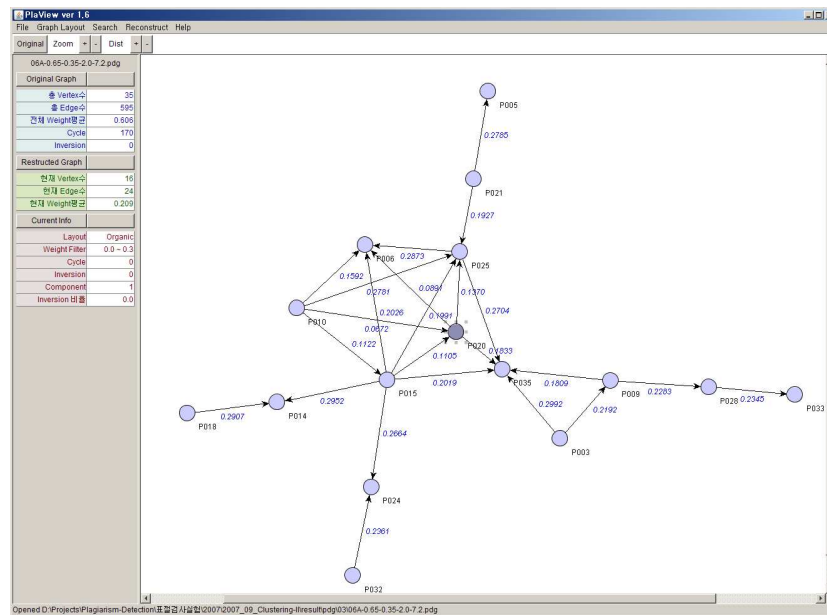


(a) 프로그램 유사도 테이블

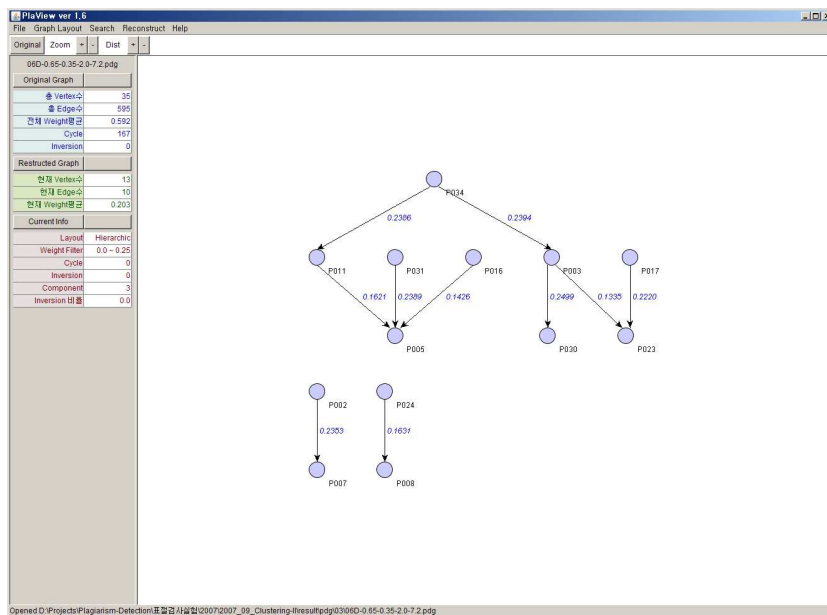


(b) 소스코드 뷰(유사구간)

그림 5.1: SoVAC 시스템: 적응적 프레임워크 기반의 프로그램 유사도 분석 도구, 프로그램  $P_a$ (왼편)의 32 ~ 37 라인과  $P_b$ (오른편)의 17 ~ 21 라인이 유사.



(a) 소스코드 클러스터링



(b) 프로그램 진화분석

그림 5.2: PlaView: 표절 클러스터 및 진화분석을 위한 그래프 시각화 도구

## 5.1 실험데이터

본 논문에서는 적응적 프레임워크의 유사도 비교 방식의 효과를 살펴보기 위해 인위적으로 표절하지 않은 프로그램 집합을 실험데이터로 사용하였다. 적응적 프레임워크는 프로그램 집합에 따라 적응적으로 유사도를 산출하므로 같은 목적으로 작성된 프로그램을 실험데이터로 구성되어야 한다. 그래서 본 논문에서는 프로그래밍 경시대회 및 프로그래밍 교과목에서 학생들이 제출한 프로그램을 실험데이터로 선정하였다. 5.2절의 SoVAC을 이용한 표절탐색 실험에서는 2005년부터 2008년도까지의 ACM 국제 대학생 프로그래밍 경진대회(ICPC : International Collegiate Programming Contest) 인터넷 예선 및 동아시아 지역본선에 제출된 프로그램과 부산대학교에서 2008년도와 2010년도의 C++ 프로그래밍 교과에서 제출된 프로그램들을 실험데이터로 선택하였다. 표 5.1과 표 5.2는 실험에 사용된 프로그램 집합의 통계적 정보를 보여준다.

표 5.1에서 총 32개의 프로그램 집합을 실험 그룹으로 나누었으며, 각 프로그램 집합은 ICPC에서 한 문제에 대하여 제출된 프로그램들로 구성된다. 그리고 2005년부터 2008년까지의 ICPC 인터넷 예선과 동아시아 지역본선에서 매년 1번 문제에서 4번 문제까지, 8개 문제를 프로그램 집합으로 선택하였다. 표 5.1에서 A~D 그룹까지는 인터넷 예선을 나타내고, E~H까지는 지역 본선을 나타낸다. ICPC 대회에 제출된 프로그램들은 작게는 20~30줄부터 많게는 500줄 정도로 작성되었다. 그리고 평균적으로 쉬운 문제의 경우 50줄 안팎으로 작성되었으며, 난이도가 높은 경우 100줄 이상의 코드들도 많이 있었다.

표 5.2의 데이터 집합은 C++ 프로그래밍 교과목의 한 클래스에서 제출된 프로그램들이며, C++언어로 작성된 프로그램들이다. 프로그램 집합들은 프로그래밍 실습과제(CPP08-P01 ~ CPP08-P15, CPP10-P01 ~ CPP10-P07)와 중간고사(CPP08-P16 ~ CPP08-P20), 기말고사(CPP08-P21 ~ CPP08-P25)로 구성된다. 그리고 각 프로그램 집합은 약 40개 내외의 프로그램들로 구성되며, 소스코드는 약 80 ~ 100줄 내외로 작성되었다.

본 실험에서는 수집된 데이터 프로그램 집합에 대하여 실제 표절 프로그램을 찾아내기

표 5.1: 실험데이터: ICPC 대회 제출 프로그램(2005년부터 2008년까지)

| 번호 | 그룹       | 프로그램 | 쌍(Pair) | 소스코드 |     |        |       |
|----|----------|------|---------|------|-----|--------|-------|
|    |          |      |         | 최고   | 최소  | 평균     | 표준편차  |
| 1  | ICPC05-A | 153  | 11,628  | 144  | 21  | 44.46  | 15.85 |
| 2  | ICPC05-B | 113  | 6,328   | 139  | 24  | 65.73  | 22.73 |
| 3  | ICPC05-C | 34   | 561     | 258  | 45  | 80.38  | 39.05 |
| 4  | ICPC05-D | 9    | 36      | 182  | 62  | 106.11 | 39.57 |
| 5  | ICPC05-E | 60   | 1,770   | 53   | 20  | 33.45  | 6.79  |
| 6  | ICPC05-F | 57   | 1,596   | 167  | 20  | 46.91  | 22.60 |
| 7  | ICPC05-G | 32   | 496     | 323  | 48  | 104.84 | 55.42 |
| 8  | ICPC05-H | 41   | 820     | 278  | 33  | 75.63  | 49.05 |
| 9  | ICPC06-A | 179  | 15,931  | 216  | 19  | 47.60  | 20.86 |
| 10 | ICPC06-B | 174  | 15,051  | 180  | 19  | 43.78  | 20.46 |
| 11 | ICPC06-C | 167  | 13,861  | 234  | 18  | 57.77  | 27.10 |
| 12 | ICPC06-D | 76   | 2,850   | 110  | 25  | 57.68  | 20.63 |
| 13 | ICPC06-E | 68   | 2,278   | 214  | 29  | 69.60  | 28.87 |
| 14 | ICPC06-F | 70   | 2,415   | 227  | 38  | 76.21  | 27.70 |
| 15 | ICPC06-G | 18   | 153     | 320  | 81  | 174.27 | 67.17 |
| 16 | ICPC06-H | 24   | 276     | 277  | 55  | 146.33 | 63.36 |
| 17 | ICPC07-A | 253  | 31,878  | 213  | 20  | 59.90  | 22.11 |
| 18 | ICPC07-B | 175  | 15,225  | 468  | 21  | 89.31  | 45.84 |
| 19 | ICPC07-C | 63   | 1,953   | 414  | 36  | 143.44 | 53.76 |
| 20 | ICPC07-D | 49   | 1,176   | 570  | 32  | 75.61  | 74.53 |
| 21 | ICPC07-E | 56   | 1,540   | 128  | 29  | 64.14  | 19.50 |
| 22 | ICPC07-F | 53   | 1,378   | 413  | 29  | 59.89  | 53.52 |
| 23 | ICPC07-G | 21   | 210     | 236  | 64  | 138.67 | 48.71 |
| 24 | ICPC07-H | 42   | 861     | 462  | 64  | 163.12 | 91.86 |
| 25 | ICPC08-A | 271  | 36,585  | 231  | 25  | 61.31  | 23.66 |
| 26 | ICPC08-B | 267  | 35,511  | 151  | 22  | 71.51  | 24.26 |
| 27 | ICPC08-C | 148  | 10,878  | 213  | 25  | 76.59  | 31.22 |
| 28 | ICPC08-D | 14   | 91      | 179  | 63  | 126.79 | 38.14 |
| 29 | ICPC08-E | 65   | 2,080   | 111  | 21  | 40.31  | 16.62 |
| 30 | ICPC08-F | 63   | 1,953   | 201  | 35  | 83.25  | 33.60 |
| 31 | ICPC08-G | 33   | 528     | 331  | 54  | 143.58 | 58.46 |
| 32 | ICPC08-H | 24   | 276     | 422  | 144 | 226.96 | 63.06 |

표 5.2: 실험데이터: 2008년도와 2010년도 C++ 프로그래밍 교과목에 제출된 프로그램 집합

| 번호 | 그룹        | 프로그램 | 쌍(Pair) | 소스코드 |     |        |       |
|----|-----------|------|---------|------|-----|--------|-------|
|    |           |      |         | 최고   | 최소  | 평균     | 표준편차  |
| 1  | CPP08-P01 | 41   | 820     | 171  | 40  | 101.80 | 31.09 |
| 2  | CPP08-P02 | 28   | 378     | 174  | 40  | 91.07  | 29.27 |
| 3  | CPP08-P03 | 38   | 703     | 84   | 24  | 39.89  | 16.76 |
| 4  | CPP08-P04 | 41   | 820     | 118  | 42  | 72.78  | 20.10 |
| 5  | CPP08-P05 | 35   | 595     | 97   | 31  | 50.49  | 15.24 |
| 6  | CPP08-P06 | 32   | 496     | 541  | 57  | 136.72 | 91.41 |
| 7  | CPP08-P07 | 34   | 561     | 187  | 63  | 123.29 | 28.44 |
| 8  | CPP08-P08 | 34   | 561     | 256  | 117 | 195.47 | 38.98 |
| 9  | CPP08-P09 | 39   | 741     | 83   | 27  | 51.36  | 14.20 |
| 10 | CPP08-P10 | 33   | 528     | 219  | 59  | 146.45 | 35.98 |
| 11 | CPP08-P11 | 33   | 528     | 84   | 42  | 59.42  | 10.53 |
| 12 | CPP08-P12 | 37   | 666     | 248  | 66  | 135.46 | 43.53 |
| 13 | CPP08-P13 | 14   | 91      | 130  | 26  | 63.57  | 28.77 |
| 14 | CPP08-P14 | 30   | 435     | 199  | 85  | 135.70 | 30.65 |
| 15 | CPP08-P15 | 36   | 630     | 68   | 25  | 38.39  | 12.67 |
| 16 | CPP08-P16 | 38   | 703     | 57   | 23  | 36.39  | 10.43 |
| 17 | CPP08-P17 | 38   | 703     | 115  | 42  | 61.47  | 22.20 |
| 18 | CPP08-P18 | 16   | 120     | 57   | 20  | 31.38  | 14.21 |
| 19 | CPP08-P19 | 32   | 496     | 74   | 27  | 41.91  | 15.30 |
| 20 | CPP08-P20 | 39   | 742     | 456  | 24  | 82.21  | 81.32 |
| 21 | CPP08-P21 | 36   | 630     | 92   | 24  | 47.31  | 12.80 |
| 22 | CPP08-P22 | 28   | 378     | 191  | 42  | 78.43  | 40.90 |
| 23 | CPP08-P23 | 29   | 406     | 113  | 38  | 61.38  | 21.46 |
| 24 | CPP08-P24 | 31   | 465     | 153  | 37  | 82.52  | 28.99 |
| 25 | CPP08-P25 | 19   | 171     | 166  | 69  | 106.89 | 28.98 |
| 26 | CPP10-P01 | 43   | 903     | 86   | 35  | 53.12  | 14.34 |
| 27 | CPP10-P02 | 40   | 780     | 77   | 25  | 40.67  | 10.91 |
| 28 | CPP10-P03 | 45   | 990     | 135  | 38  | 68.40  | 18.61 |
| 29 | CPP10-P04 | 46   | 1,035   | 112  | 36  | 55.22  | 12.15 |
| 30 | CPP10-P05 | 46   | 1,035   | 169  | 40  | 79.61  | 17.66 |
| 31 | CPP10-P06 | 41   | 820     | 113  | 35  | 76.00  | 12.29 |
| 32 | CPP10-P07 | 13   | 78      | 308  | 50  | 169.92 | 60.29 |

위해서 모든 프로그램 집합에 대해 표절을 검사했다. 5.2절에 설명하는 실험 결과는 적응적 프레임워크 기반으로 구현한 SoVAC으로 검사한 결과들이다. 과거 발표된 연구 결과에서는 주로 인위적인 표절을 수행한 프로그램에 대해 표절 검사를 수행하였다. 그러나 이러한 인위적인 표절은 실제 현장에서 행해지고 있는 표절과는 차이가 있다. 따라서 본 연구에서는 실제 프로그래밍 경진대회나 실제 과제 제출 프로그램을 대상으로 하여 표절을 검사하였으며, 그 결과 표절로 의심되는 다수의 프로그램 쌍들을 찾을 수 있었다. 개인 정보 보호를 위해 표절한 출전팀을  $T_n$ 으로 지칭하겠다. 여기서  $n$ 은 팀번호로 본 논문에서는 실제 팀과 다르게 임의로 지정하였다. 표절로 의심되는 코드의 유사 구간에 대해서는 5.3절에서 살펴보겠다.

## 5.2 프로그램 표절탐색

본 절에서는 실험데이터의 프로그램 집합에 대해 실제 표절을 검출한 것에 대해 기술한다. 먼저 ICPC 프로그램 집합에 대한 표절 검사에 대해 기술하고, 다음으로 C++ 프로그래밍 교육목의 프로그램들에 대한 표절검사 결과를 살펴본다.

ICPC는 ACM의 주체로 매년 개최대는 국제 대학생 프로그래밍 경진대회이다. ICPC는 각 나라와 대륙 별로 지역 예선을 치른 후 상위 입상자들을 대상으로 본선 대회를 치른다. 우리나라에서는 ICPC에 참가하는 대학생들을 대상으로 먼저 지역예선을 치른 후 동아시아 지역 본선을 서울에서 치른다. 여기서 1차 지역예선과 동아시아 지역 본선은 대회 진행 방식이 다르다. 1차 지역예선은 인터넷을 통해 프로그램을 제출하는 온라인 방식으로 치루어지고, 2차 동아시아 지역 본선은 한 장소에 모여서 시험을 치르는 오프라인 방식으로 대회가 진행된다. 이 때문에 온라인 방식으로 진행되는 1차 지역예선은 각 학교 별로 감독관의 지도하에 자율적으로 대회가 진행된다. 그리고 2차 본선은 프로그래밍 환경이 갖춰진 대회장에서 감독관들의 감독하에 대회가 진행된다. 2차 본선에서는 내부 네트워크로 서버에 연결되고 외부 인터넷 연결은 차단된다.

본 논문에서 구현한 SoVAC은 ICPC 대회의 표절 검사 도구로 실제 사용되고 있다. 매년 ICPC 대회 후 제출된 프로그램들을 모아 SoVAC을 이용해 1차적으로 표절 의심 프로그램들을 찾고 심사관들의 판정하에 최종 표절 프로그램 판정을 내린다. 대회에서 표절이 적발된 팀에 대해서는 대회 출전 자격정지와 같은 징계가 내려진다. 지금부터 SoVAC을 이용해 ICPC 대회에서 실제 표절을 검출한 사례에 대해 설명한다.

표 5.1의 실험데이터는 총 32개의 프로그램 그룹으로 4년간 치뤄진 인터넷 예선과 지역 본선에서 각각 4 그룹씩 선정한 것이다. 32개의 그룹에서 표절 프로그램은 2006년도와 2007년도 ICPC 대회에서 검출되었다. 표 5.3은 2007년도 ICPC 1차 지역예선에서 발견된 표절 의심 프로그램들을 보여준다.

표 5.3: ICPC 표절 탐색 결과: ICPC07 그룹에서 7개 팀의 소스코드가 표절로 검출되었음.

| 번호 | 그룹       | 프로그램 A   | 프로그램 B   | 유사도    |
|----|----------|----------|----------|--------|
| 1  | ICPC07-A | $T_{01}$ | $T_{02}$ | 100.00 |
| 2  | ICPC07-A | $T_{03}$ | $T_{04}$ | 97.10  |
| 3  | ICPC07-A | $T_{05}$ | $T_{06}$ | 93.23  |
| 4  | ICPC07-B | $T_{01}$ | $T_{02}$ | 95.60  |
| 5  | ICPC07-D | $T_{02}$ | $T_{07}$ | 100.00 |

표 5.3에서 프로그램 A와 프로그램 B 항목의 팀이름은 실험에서 임의로 부여한 것이다. 그리고 두 프로그램의 유사도는 0 ~ 100%로 정규화하였다. 유사도가 100%이라는 것은 두 프로그램이 완전히 동일하게 작성된 것을 나타낸다. 즉, ICPC07-A 그룹에서  $T_{01}$ 과  $T_{02}$ 의 프로그램은 완전히 같음을 나타낸다.

표 5.3을 살펴보면 2007년도 인터넷 예선에서는 총 7개 팀의 프로그램에서 표절로 의심되는 코드들이 발견되었다. 표를 자세히 살펴보면, ICPC07-A그룹과 ICPC07-D 그룹에서  $T_{01}$ 과  $T_{02}$ ,  $T_{07}$ 들의 프로그램이 완전히 동일하게 작성된 것을 볼 수 있다. 여기서  $T_{01}$ 과  $T_{02}$ 는 두 그룹에서 프로그램이 거의 유사하게 작성되었다. 또한,  $T_{02}$ 는  $T_{01}$ 과  $T_{07}$ 의 서로 다른 두 팀과 프로그램이 유사하게 작성된 것을 볼 수 있다. 이들 세 팀을 조사한 결과, 이들은 모두

같은 대학교에서 출전한 팀으로 밝혀졌다. 이처럼 지역예선이 온라인 방식으로 대회가 치뤄진다는 점을 감안할 때, 세 팀은 감독관의 주의를 피해 네트워크나 다른 저장매체를 이용하여 프로그램을 주고 받은 것으로 사료된다.

다음은 오프라인으로 대회가 진행되는 2007년도 ICPC 동아시아대회 본선에 제출된 프로그램을 분석한 것이다. 표 5.4는 SoVAC을 이용한 표절 프로그램 탐색 결과를 보여준다. 표 5.4에서도 프로그램 A와 프로그램 B 항목의 팀번호는 임의로 재설정하였다.

표 5.4: 2007년도 ICPC 동아시아대회 본선 프로그램 집합에 대한 표절탐색 결과

| 번호 | 그룹       | 프로그램 A   | 프로그램 B   | 유사도    |
|----|----------|----------|----------|--------|
| 1  | ICPC07-E | $T_{01}$ | $T_{02}$ | 100.00 |
| 2  |          | $T_{03}$ | $T_{04}$ | 32.08  |
| 3  |          | $T_{05}$ | $T_{06}$ | 31.80  |
| 4  | ICPC07-F | $T_{01}$ | $T_{07}$ | 100.00 |
| 5  |          | $T_{08}$ | $T_{09}$ | 65.11  |
| 6  |          | $T_{02}$ | $T_{10}$ | 53.20  |
| 7  | ICPC07-G | $T_{01}$ | $T_{11}$ | 100.00 |
| 8  |          | $T_{07}$ | $T_{12}$ | 13.50  |
| 9  |          | $T_{01}$ | $T_{13}$ | 13.50  |
| 10 | ICPC07-H | $T_{01}$ | $T_{14}$ | 79.67  |
| 11 |          | $T_{15}$ | $T_{16}$ | 39.03  |
| 12 |          | $T_{16}$ | $T_{17}$ | 36.08  |

표 5.3의 실험 결과는 오프라인으로 진행되는 본선대회의 특징을 잘 보여준다. 표 5.3은 각 그룹당 유사도가 가장 높은 세 쌍의 프로그램만 기록하였다. 실험결과를 자세히 살펴보면, 유사도가 가장 높은 프로그램을 제외하면 대체로 유사도가 50% 이하로 낮게 측정되었다. 이는 오프라인에서 프로그램이 독립적으로 작성되었음을 알 수 있다.

표 5.3에서는 모든 그룹에서 유사도가 가장 높은 프로그램 쌍에서 표절이 되었음을 알 수 있다. ICPC07-E를 비롯한 세 그룹에서 유사도가 100%로 측정되었고, 나머지 한 그룹에서 약 80%가 유사하게 작성된 것을 볼 수 있다. 또 한가지 특징은 모든 표절 프로그램 쌍에  $T_{01}$ 이 연루되어 있음을 알 수 있다. 이 같은 경우는 매우 드문 것으로 보통의 표절은 두 팀

사이에 이루어 지는데  $T_{01}$ 의 경우 여러 팀의 프로그램과 동일하게 작성되었다.

$T_{01}$ 은  $\{T_{02}, T_{07}, T_{11}, T_{14}\}$ 의 프로그램들과 각각 유사하게 작성되었다. 이는 첫째로  $T_{01}$ 이 표절 행위의 주모자라는 것을 알 수 있다. 다음으로 불특정 다수로부터 프로그램을 표절했다는 것을 알 수 있다. 2007년도 ICPC 대회 조사결과  $T_{01}$ 은 네트워크 패킷을 훔치는 해킹 프로그램을 이용해 표절한 것으로 밝혀졌다. 해킹 프로그램을 이용해 내부 네트워크에 연결된 컴퓨터로부터 채점 서버로 전송되는 패킷을 해킹하여 프로그램 소스코드를 표절한 것이다. 이처럼 참가자가 많고 불특정 다수의 프로그램으로부터 표절을 할 경우 감독관에 의해 표절을 탐색하기가 매우 어렵다.

다음으로 실제 학생들이 프로그래밍 교과목을 수강하면서 제출한 프로그램들에 대한 표절 검사 결과에 대해 기술한다. C++ 프로그래밍 교과목은 학부 2학년과정으로 C++에 대해 배운다. 따라서 제출된 프로그램은 모두 C++로 작성된 프로그램들이며, 한 학기 동안 과제와 중간 및 기말 실기 평가를 통해 제출된 프로그램들이다. 프로그래밍 실습과제는 2~3일 정도의 시간을 주어 자율적으로 작성하도록 하였으며, 중간고사와 기말고사는 약3시간 정도의 시간을 주고 오프라인 방식으로 진행되었다. 표 5.5은 실습과제에 대하여 SoVAC을 이용한 표절 프로그램 탐색 결과를 보여준다.

표 5.5의 실험결과는 일반적인 프로그래밍 실습과제에서 발생할 수 있는 표절 발생에 대한 특징을 잘 보여준다. 표 5.5의 실험결과를 살펴보면, 표절로 의심되는 프로그램 쌍들의 경우 유사도가 매우 높게 나타나는 것을 알 수 있다. 그리고 복사-붙이기를 통해 코드의 수정 없이 다른 학생의 소스코드를 그대로 제출하는 경우도 빈번하게 볼 수 있다. 또한, CPP10-P05와 CPP10-P06 그룹의 검사결과를 보면, 한번 표절 관계가 있었던 학생 그룹은 다음에도 반복적으로 표절을 할 위험이 매우 높다는 것을 알 수 있다.

마지막으로 C++ 프로그래밍의 중간 및 기말고사의 표절 탐색 결과를 살펴보자. 표 5.2의 CPP08-P15 ~ CPP08-P20는 중간고사의 프로그램 집합이며, CPP08-P21 ~ CPP08-P25는 기말고사 프로그램 집합이다. 표 5.6은 표절 탐색 결과를 보여준다.

표 5.5: 2008년도 C++ 프로그래밍 실습 과제 프로그램 집합에 대한 표절탐색 결과

| 번호 | 그룹        | 프로그램 A   | 프로그램 B   | 유사도    |
|----|-----------|----------|----------|--------|
| 1  | CPP08-P08 | $T_{01}$ | $T_{02}$ | 96.65  |
| 2  |           | $T_{02}$ | $T_{03}$ | 96.61  |
| 3  |           | $T_{01}$ | $T_{03}$ | 95.23  |
| 4  |           | $T_{04}$ | $T_{05}$ | 70.77  |
| 5  | CPP08-P09 | $T_{06}$ | $T_{07}$ | 95.17  |
| 6  |           | $T_{08}$ | $T_{09}$ | 90.08  |
| 7  |           | $T_{10}$ | $T_{11}$ | 83.94  |
| 8  | CPP10-P05 | $T_{12}$ | $T_{13}$ | 100.00 |
| 9  |           | $T_{12}$ | $T_{14}$ | 100.00 |
| 10 |           | $T_{12}$ | $T_{15}$ | 100.00 |
| 11 |           | $T_{13}$ | $T_{14}$ | 100.00 |
| 12 |           | $T_{13}$ | $T_{15}$ | 100.00 |
| 13 | CPP08-P06 | $T_{12}$ | $T_{13}$ | 100.00 |
| 9  |           | $T_{12}$ | $T_{14}$ | 100.00 |
| 10 |           | $T_{12}$ | $T_{15}$ | 100.00 |
| 11 |           | $T_{13}$ | $T_{14}$ | 100.00 |
| 12 |           | $T_{13}$ | $T_{15}$ | 100.00 |

표 5.6: 2008년도 C++ 프로그래밍 중간 및 기말고사 프로그램 집합에 대한 표절탐색 결과

| 번호 | 그룹        | 프로그램 A   | 프로그램 B   | 유사도   |
|----|-----------|----------|----------|-------|
| 1  | CPP08-P16 | $T_{01}$ | $T_{02}$ | 72.19 |
| 2  |           | $T_{03}$ | $T_{04}$ | 68.61 |
| 3  |           | $T_{05}$ | $T_{06}$ | 64.43 |
| 4  | CPP08-P17 | $T_{01}$ | $T_{07}$ | 82.03 |
| 5  |           | $T_{01}$ | $T_{08}$ | 77.24 |
| 6  |           | $T_{01}$ | $T_{09}$ | 70.53 |
| 7  | CPP08-P20 | $T_{10}$ | $T_{11}$ | 74.43 |
| 8  |           | $T_{12}$ | $T_{13}$ | 63.25 |
| 9  |           | $T_{14}$ | $T_{15}$ | 55.92 |
| 10 | CPP08-P21 | $T_{04}$ | $T_{16}$ | 72.87 |
| 11 |           | $T_{17}$ | $T_{18}$ | 64.31 |
| 12 |           | $T_{16}$ | $T_{17}$ | 64.30 |

표 5.6의 실험 결과는 ICPC 지역본선과 유사하게 오프라인 방식으로 진행한 경우에 대한 프로그램 유사도 특징을 보여준다. 표 5.6은 중간고사와 기말고사에서 프로그램 수가 가장 많은 집합의 표절 탐색 결과를 보여준다. 표에서 프로그램 A와 프로그램 B 항목은 표 5.3에서와 동일한 방식으로 부여하였다. 표 5.6의 실험결과에서 유사도 항목을 살펴보면 실습과제에 비해 프로그램 쌍들의 유사도가 낮았으며 표절 프로그램 쌍이 발견되지 않았다.

지금까지의 실험결과 적응적 프레임워크를 이용하여 구현한 SoVAC은 프로그램 집합에 포함된 표절 프로그램 쌍들을 효율적으로 찾을 수 있었다. 그리고 프로그램 표절의 경우 프로그램 개발 환경에 많은 영향을 받는 것을 알 수 있다. 프로그램 개발이 온라인 방식으로 진행될 경우 프로그램 표절이 상대적으로 많았으며, 감독관의 입회하에 진행되는 오프라인 방식의 경우 상대적으로 표절 프로그램 발생 비율이 낮은 것을 볼 수 있었다.

### 5.3 유사구간 탐색

본 절에서는 유사구간의 차이를 구체적으로 살펴본다. 유사구간은 각 토큰별 유사도 값에 따라 결정되므로 유사도 행렬 값에 따라 달라질 수 있다. 유사 구간 탐색 실험에서는 본 논문에서 제안한 적응적 유사도 행렬과 지역정렬의 기본 점수 행렬인 고정적 유사도 행렬, 추가로 JPlag 시스템의 유사도 탐색 방법인 욕심쟁이 문자열 묶음(Greedy-String-Tiling) 알고리즘을 적용한 경우에 대해 표절로 의심되는 코드의 유사구간을 비교해 보았다.

그림 5.3 및 그림 5.4에 제시된 두 프로그램은 ICPC 대회에 제출된 프로그램 중에서 표절로 의심되는 프로그램 쌍이다. 두 프로그램을 표절로 보는 이유는 다음과 같다. 먼저, 변수  $t$ 와 변수  $n$ 을 바꾸어 사용했다. 이 문제의 테스트 데이터는 모두  $t$ 개의 경우가 있으며, 각각의 경우마다 세부 데이터가  $n$ 개가 있다고 문제에서 설명을 했기 때문에 적어도 문제에서 제시된 변수를 바로 사용한다면 외부에  $t$  번의 루프가 있어야 하고, 각 케이스마다  $n$  번의 `scanf`가 있어야 정상이다. 그런데  $P_b$ 에서는 이 의미가 거꾸로 사용되고 있다. 두 번째로  $P_a$ 의 19번째 줄의 `for` 루프에서  $j$ 의 제한 조건이  $P_b$ 에서는 그 역순( $j--$ )로 표시되었다.  $P_b$ 를

```

1 #include <stdio.h>
2 int main ()
3 {
4     int t, n;
5     int i, j;
6     int p[500];
7     int ans[500], max;
8     scanf("%d", &t);
9
10    while (t--)
11    {
12        scanf("%d", &n);
13        for (i=0; i<n; i++)
14        {
15            scanf("%d", &p[i]);
16            ans[i] = 0;
17        }
18        for (i=0; i<n; i++)
19            for (j=0; j<i; j++)
20                if (p[i] >= p[j] && ans[i] < ans[j] + 1)
21                    ans[i] = ans[j] + 1;
22        max = 0;
23        for (i=0; i<n; i++)
24            if (ans[i] > max) max = ans[i];
25        printf("%d n", max + 1);
26    }
27    return 0;
28 }

```

그림 5.3: ICPC05에서 표절로 의심되는 프로그램  $P_a$

```

1 #include<stdio.h>
2 int main()
3 {
4     int i, j;
5     int data[600], cnt[600];
6     int n, t;
7     int max;
8     scanf("%d", &n);
9     while(n--)
10    {
11        scanf("%d", &t);
12        for( i=0 ; i<t ; i++ )
13        {
14            scanf("%d", &data[i]);
15            cnt[i] = 0;
16        }
17
18        for( i=0 ; i<t ; i++ )
19        {
20            for( j=i ; j>=0 ; j-- )
21            {
22                if( data[i] >= data[j] && cnt[j]+1 > cnt[i] )
23                {
24                    cnt[i] = cnt[j]+1;
25                }
26            }
27        }
28        max = 0;
29        for( i=0 ; i<t ; i++ )
30        {
31            if( max < cnt[i] )
32                max = cnt[i];
33        }
34        printf("%d n", max);
35    }
36    return 0;
37 }

```

그림 5.4:  $P_a$ 의 표절로 의심되는 프로그램  $P_b$

살펴보면 다른 for 루프는 모두 일반적인 형식인 `for(x = 0; x < n; x++)`으로 사용했음에도 불구하고 실제 그 순서에 아무런 의미가 없는 j에 대한 for 루프만을 다르게 표시했다는 것은 표절의 의심을 사기에 충분하다고 할 것이다.

설명 편의를 위해 적응적 지역정렬을 LAA(Local Alignment with Adaptive matrix)로 나타내고, 고정적 지역정렬을 LAS(Local Alignment with Static matrix)로 나타낸다. 그리고 JPlag의 욕심쟁이 문자열 묶음 알고리즘을 GST로 표시한다. 다음으로 두 프로그램 사이의 유사구간은 정의 2와 같이  $C = \langle C(P_a, L_i, L_j), C(P_b, L_m, L_n) \rangle$ 의 코드 클론으로 나타낸다( $i < j, m < n$ ). 그림 5.3와 5.4의 두 프로그램을 비교했을 때, GST는 코드 클론  $C_{GST}$ 를  $\langle C(P_a, L_1, L_{18}), C(P_b, L_1, L_{18}) \rangle$ 로 보고했고, LAS는  $\langle C(P_a, L_1, L_{19}), C(P_b, L_1, L_{20}) \rangle$  구간을 표절 구간으로 보고했다. 마지막으로 LAA로 비교했을 때에는 표절로 의심되는 코드 클론  $C_{LAA}$ 가  $\langle C(P_a, L_1, L_{28}), C(P_b, L_1, L_{37}) \rangle$ 로 더 넓은 구간이 일치한다고 보고했다.

위의 유사구간 탐색 실험은 LAA 방법이 키워드의 차이를 잘 극복해 주었다는 것을 보여준다. 즉 for 루프 조건을 바꾸는 구간이 다소 길긴 하지만 LAA 방법은 토큰의 불일치 값이 LAS의 불일치 값인 -1보다 작으므로 그 뒤의 일치한 부분을 포함시켜 정렬을 구성하였다. 통상 사용하지 않는 코드를 강제로 넣어서 표절을 하는 경우에 표절자는 채점자의 눈에 크게 띄지 않는 코드를 넣는다. 왜냐하면 아주 특별한 기능이나 특이한 구문을 삽입할 경우 눈에 잘 띄기 때문에 일반적인 지정문(assignment)나 간단한 if-else등을 많이 사용한다. 이 경우 LAA는 LAS나 GST에 비해 불일치에 의한 감점이 작기 때문에, 불일치 구간 밖에 다른 일치 구간이 있을 경우 그것을 포함하여 표절구간으로 계산한다. 위의 경우는 LAA가 다른 방법들에 비해 유연하다는 것을 잘 보여주는 경우이다.

## 5.4 JPlag 시스템과의 비교

본 절에서는 적응적 프레임워크에 기반한 SoVAC과 JPlag의 효율성을 비교한다. SoVAC과 JPlag 시스템은 모두 중간표현으로 토큰 서열을 사용한다는 공통점이 있다. 하지만 후단부의 유사도는 서로 다른 방법으로 계산된다. JPlag는 GST 알고리즘을 이용하고 SoVAC은 LAA 알고리즘을 사용한다. 두 방법의 가장 큰 차이점은 LAA는 프로그램 집합에 따라 유사도 계산을 위한 매트릭스가 달라진다는 것과 GST는 서열이 일치한 경우만을 연결하는데 반해, LAA는 약간의 불일치에 대해서 감점을 부여하고 연결한다는 것이다.

표절 검사의 경우 시스템의 성능을 정량적으로 측정하는 것은 매우 중요하다. 표절 탐색 시스템의 경우 잘못된 판정을 하면 안되기 때문에 판정의 정확성이 무엇보다 중요하다. 표절 검사 시스템의 유효성(effectiveness)과 정확성(accuracy)을 정량적으로 측정하기 위해 본 논문에서는 “민감도(sensitivity)”와 “적확도(specificity)”를 계산한다. 이들 두 변수는 정보검색 및 실험을 통한 성능 측정 연구에서 일반적으로 사용되는 방법이다. 설명을 위해 지금부터 민감도를  $SN$ 으로 적확도를  $SP$ 로 표기한다.

본 논문에서는  $n$ 개의 프로그램이 주어졌을 때,  $n(n-1)/2$ 개의 프로그램 쌍으로부터  $SN$ 과  $SP$ 의 계산을 위해 먼저 다음의 4가지 경우를 판별한다.

- 올바른 양성판정(True Positive( $TP$ )): 실제 표절된 코드를 올바르게 찾음
- 올바른 음성판정(True Negative( $TN$ )): 표절이 아닌 코드를 검출하지 않음
- 잘못된 양성판정(False Positive( $FP$ )): 표절이 아닌 코드를 표절로 판정
- 잘못된 음성판정(False Negative( $FN$ )): 실제 표절된 코드를 표절이 아닌 것으로 판정

위의 정의처럼  $TP$ 와  $TN$ 은 올바르게 찾은 경우이고 나머지 두 경우는 판정이 틀린 경우이다.  $FP$ 는 검사 결과 두 프로그램이 표절된 것으로 판정했지만 실제로는 독립적으로 작성된 경우이다. 실제로 이 경우에 대해서는 다른 외부 요인에 의해 같게 작성된 경우도 있다. 마지막으로  $FN$ 의 경우는 표절이 아닌 프로그램을 표절로 판정한 경우이다. 본 절의 실험

에서는 실험자가 직접 표절하지 않은 프로그램에 대해서는 독립적으로 작성된 것으로 가정한다. 표 5.7는 4가지 다른 판정 결과를 요약하여 보여준다.

표 5.7: 표절 상황 및 표절 판정에 대한 4가지 결과

|      |      | 실제 조건                  |                        |
|------|------|------------------------|------------------------|
|      |      | 표절                     | 표절아님                   |
| 표절검사 | 표절   | 올바른 양성판정 ( <i>TP</i> ) | 잘못된 양성판정 ( <i>FP</i> ) |
|      | 표절아님 | 잘못된 음성판정 ( <i>FN</i> ) | 올바른 음성판정 ( <i>TN</i> ) |

표 5.7의 4가지 조건을 이용해 민감도(*SN*)와 정확도(*SP*)은 다음과 같이 정의된다.

$$SN(k) = \frac{|TP|}{|TP| + |FN|} = \frac{P(k) \text{ 집합에서 표절 프로그램 쌍의 수}}{RP}$$

$$SP(k) = \frac{|TN|}{|TN| + |FP|} = \frac{\overline{P(k)} \text{ 집합에서 표절아닌 프로그램 쌍의 수}}{NP}$$

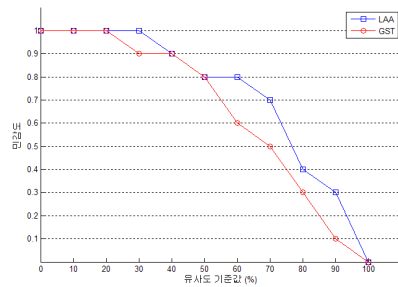
여기서 *k*는 표절에 대한 유사도 기준 값(user-defined cutoff value)이다. 유사도가 *k*일 때, *P(k)*는 유사도가 *k*%와 같거나 그 이상인 프로그램 쌍들의 집합을 나타낸다. 그리고  $\overline{P(k)}$ 는 반대로 유사도가 *k*% 이하인 프로그램 쌍들의 집합이다. *RP*는 프로그램 집합에 포함되어 있는 실제로 표절된 프로그램 쌍의 개수를 나타내고, *NP*는 실제로 표절과 연관이 없는 프로그램 쌍의 개수를 나타낸다.

민감도와 정확도는 시스템의 성능에 대하여 다음과 같은 의미를 나타낸다. 예를 들어 시스템이 민감도가 높다는 것은 그 시스템이 실제 표절된 프로그램 쌍을 놓지치 않음을 나타낸다. 그리고 시스템의 정확도가 높다는 것은 독립적으로 작성된 것을 표절된 것으로 잘못 판정하지 않는 것을 의미한다. 가장 바람직한 경우는 민감도와 정확도가 모두 높은 시스템이다. 하지만 일반적으로 민감도와 정확도는 서로 모순되는 특성들이다.

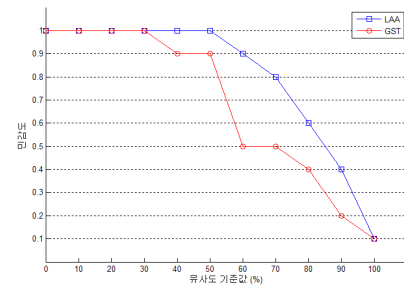
*SN*과 *SP*를 측정하기 위해 실험에서는 프로그램 그룹에 대해 표절 프로그램을 포함시켰다. 표절 프로그램을 추가한 프로그램 집합에 추가한 이유는 *SN*과 *SP*는 실제로 표절된

프로그램들을 사전에 알고 있어야 계산 가능하기 때문이다. 민감도와 정확도 측정을 위해 실험에서는 ICPC06의 4그룹(ICPC06-C~ICPC06-F)에서 각각 10개의 표절 프로그램을 생성하여 추가하였다. 표절 프로그램 생성은 각 그룹당 30분씩의 시간을 주고 원본과 동일한 기능을 수행하는 프로그램을 생성하도록 하였다. 시간 제약 이외에는 자율적으로 표절을 하도록 하였다.

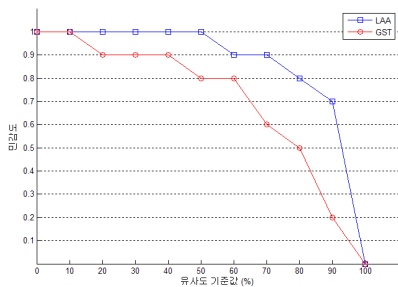
실험에서는  $x$ -축의 유사도 기준 값을 변경하면서 SoVAC과 JPlag의  $SN$ 과  $SP$ 를 비교하였다. 실험에서 표절 프로그램은 두 시스템을 통해 비교한 다음 유사도가  $x_c$  보다 높은 경우 두 프로그램은 표절된 것으로 판정하였다. 그림 5.5는 유사도 변화에 따른 두 시스템의 민감도 변화곡선을 보여준다.



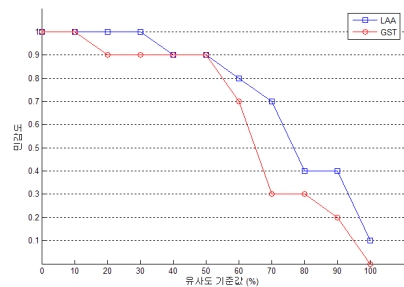
(a) ICPC06-C



(b) ICPC06-D



(c) ICPC06-E



(d) ICPC06-F

그림 5.5: 유사도 기준 값( $x$ -축)의 변화에 따른 SoVAC과 JPlag의 민감도 실험결과

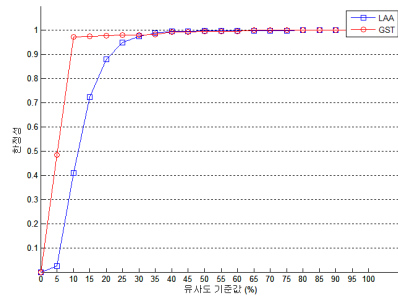
민감도와 정확도 실험에서 SoVAC의 유사도 행렬은 3.3.4절의  $Score_{AM}$ 을 사용하였으며,

적응적 유사도 행렬의 일치와 불일치를 위한 가중치 값은  $\alpha = 0.65$ 와  $\beta = 0.35$ 로 설정하였다. 이는 토큰의 일치에 상대적으로 더 많은 가중치를 준다. 그리고 JPlag의 경우에는 시스템에서 권장하는 기본 설정을 사용하여 실험을 진행했다.

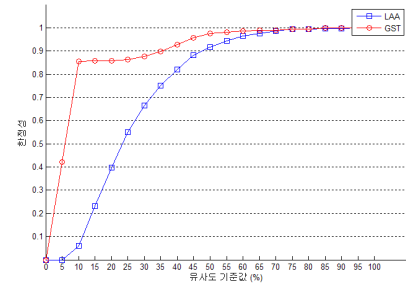
그림 5.5를 살펴보면, 네 그룹에서 모두 SoVAC의 민감도가 JPlag에 비해 높게 측정된 것을 볼 수 있다. 특히, 표절의심구간에 해당하는 유사도 80% 이상에서 SoVAC이 JPlag보다 표절 프로그램 쌍을 더욱 많이 발견한 것을 알 수 있다. 그리고 그림 5.5-(c)를 보면 JPlag의 유사도가 70%에서 민감도가 0.6인데 비해 SoVAC은 유사도가 70%일 때 민감도가 0.9로 측정되었다. 이것은 JPlag의 검사 결과가 SoVAC에 비해 잘못된 음성 판정(*FN*)이 많다는 것을 의미한다. 잘못된 음성 판정은 표절된 프로그램을 탐색하지 못하는 것으로 표절검사에서 매우 좋지 않은 경우에 속한다.

그리고 그림 5.5에서 유사도가 100%라는 것은 두 프로그램이 복사-붙이기 방법으로 표절했다는 것을 의미한다. 즉 두 프로그램은 기능적으로 완전히 동일하다는 것을 의미한다. 유사도가 100%인 프로그램은 그림 5.5-(b)와 그림 5.5-(d)에서 발견되었다. 그림 5.5-(b)의 경우, SoVAC과 JPlag이 동일하게 두 프로그램이 똑같다고 보고하였다. 하지만, 그림 5.5-(d)에서는 SoVAC의 경우에는 유사도가 100%인 프로그램 쌍을 발견하였지만, JPlag는 발견하지 못하였다. 두 프로그램 쌍을 직접 살펴본 결과, 표절 프로그램에서 실제 프로그램에서 사용하지 않는 함수를 소스코드에 추가하였다. 이에 대해, SoVAC은 프로그램 선형화 과정에서 필터링을 하기 때문에 두 프로그램의 유사도를 100%(완전일치)로 계산할 수 있었다.

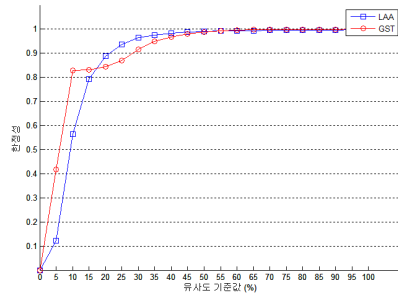
다음으로 그림 5.6은 각 그룹에 대한 *SP* 곡선을 보여준다. *SP* 곡선 또한 유사도 기준 값을 바뀌가면서 측정했다. *SP* 실험결과를 살펴보면 SoVAC의 *SP* 곡선이 JPlag 보다 부드럽게 변한다는 것을 확인할 수 있다. 이것은 유사도 변화에 대해 적응적 프레임워크의 알고리즘이 안정적이라는 것을 나타낸다고 볼 수 있다. 그림 5.6를 자세히 살펴보면 유사도가 10%에서 JPlag의 *SP*가 특이하게 변화는 것을 볼 수 있다. 이 부분은 JPlag의 유사도 계산이 토큰 서열의 완벽한 일치(perfect matching)에 의해서만 계산됨으로 잡음(noise)이 섞일 경우 상대적으로 일치하는 코드 클론이 짧기 때문이다. 그 결과 공백과 부분 불일치를 효과적으



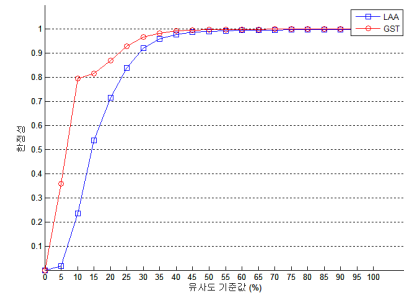
(a) ICPC06-C



(b) ICPC06-D



(c) ICPC06-E

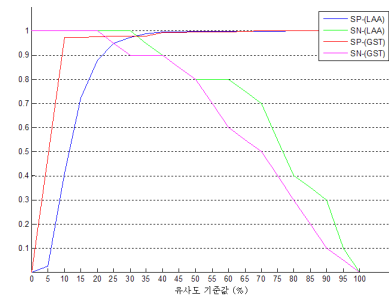


(d) ICPC06-F

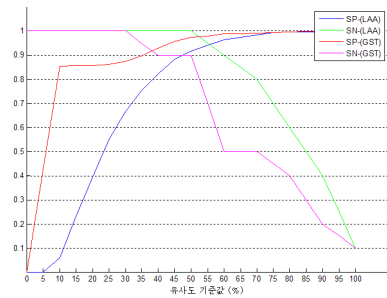
그림 5.6: 유사도 변화에 따른 SoVAC과 JPlag의 정확도(SP) 실험결과

로 고려하는 적응적 탐색 방법이 더욱 유연하므로 *SP* 곡선이 부드럽게 변하고 있다.

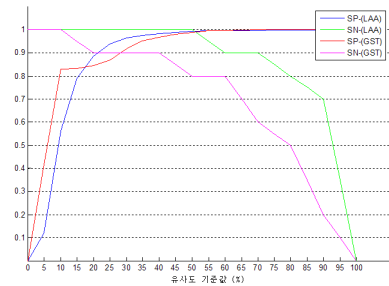
실험결과를 보면 *SN* 곡선은 단조감소를 보이고 *SP*은 단조증가를 한다. 여기서 실험 결과를 통해 시스템의 성능이 우수하다는 것을 보이는 방법은 *SN*과 *SP*가 교차하는 *x*-축과 *y*-축의 위치를 비교하는 것이다. 두 특성의 교차점에 대한 *x*-축은 민감도와 정확도 특성이 균형을 이루는 지점이다. 그림 5.7은 *SN*과 *SP*의 교차점 비교 결과를 보여준다.



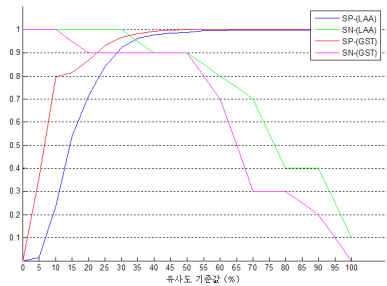
(a) ICPC06-C



(b) ICPC06-D



(c) ICPC06-E



(d) ICPC06-F

그림 5.7: ICPC 프로그램 집합에 대한 *SN*과 *SP* 교차 그래프

그림 5.7는 위의 4 그룹에 대해 *SN*과 *SP*가 교차하는 것을 비교한 것이다. 4가지 경우에 대해 SoVAC이 JPlag 보다 교차점의 *x*-축과 *y*-축 위치가 높다는 것을 볼 수 있다. 그림 5.7의 (a) 그룹에 대해서는 두 시스템이 비슷하지만, (b), (c), (d) 그룹에 대해서는 SoVAC의 교차점의 *y*-축 값이 JPlag보다 높다. *SN*과 *SP*의 실험결과는 적응적 프레임워크의 방법이 JPlag의 GST 방법보다 효과적이고 안정적이라는 것을 보여준다.

## 5.5 기존 지역정렬 방법과의 비교

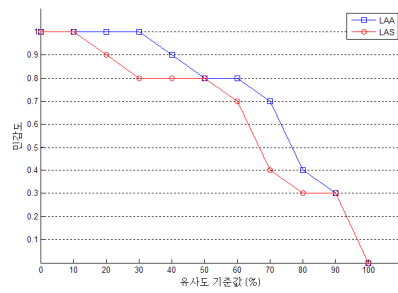
본 절에서는 적응적 지역정렬(LAA)과 기존의 고정적 지역정렬(LAS)의 성능을 실험을 통하여 평가한다. 두 알고리즘 간의 성능 비교는 5.4절에서 진행한 민감도와 정확도를 통해 비교한다. 비교 실험에 사용한 실험데이터도 동일한 프로그램 집합을 이용하였다. 그리고 적응적 지역정렬과 고정적 지역정렬의 비교를 위해 프로그램 유사도 분석의 진단부는 동일한 방법을 사용하였다. 즉, 프로그램 집합으로부터 생성된 토큰 서열은 동일하며 유사도 계산을 위한 행렬만 다르게 설정하였다.

적응적 지역정렬은  $Score_{AM}$  유사도 행렬을 사용하였으며, 고정적 지역정렬은 식 3.3의  $Score_L$  유사도 행렬을 사용하였다. 그리고 추가로 고정적 지역정렬에는 일치에 대한 가중치를 주지 않지만 성능비교를 위해 두 유사도 행렬 모두에  $\alpha = 0.65$ 와  $\beta = 0.35$ 의 가중치를 부여하였다. 그림 5.8와 그림 5.9는 유사도 변화에 따른 민감도와 정확도 변화곡선을 보여준다.

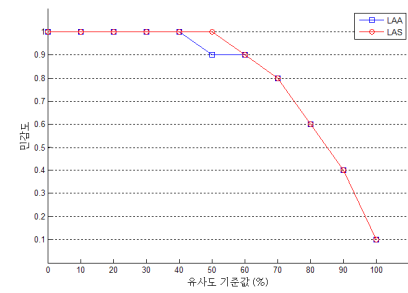
그림 5.8을 살펴보면, 네 개의 프로그램 집합에서 적응적 지역정렬과 고정적 지역정렬의 민감도가 유사한 것을 알 수 있다. 그림 5.8-(b)의 ICPC06-D 그룹에서는 두 방법의 민감도가 거의 동일하다는 것을 알 수 있다. 그러나 5.8-(c)와 5.8-(d)의 경우 적응적 방법이 민감도가 약간 더 높은 것을 알 수 있다. 특히, 유사도 기준 값을 90%로 했을 경우, 적응적 지역정렬을 사용하였을 때 표절 프로그램을 더 많이 찾아내는 것을 볼 수 있다. 이는 적응적 방법이 두 프로그램 간의 차이에 대한 감점이 작아 표절 프로그램에 대한 민감도가 높게 측정된 것이다.

다음은 적응적 지역정렬과 고정적 지역정렬을 사용하였을 때의 정확도를 비교하였다. 그림 5.9는 정확도 비교 결과를 보여준다.

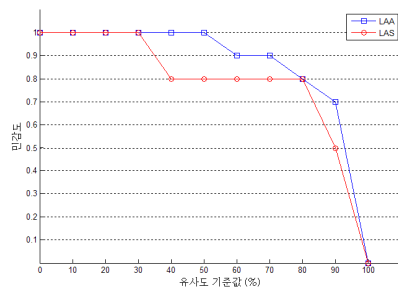
그림 5.9를 살펴보면, 적응적 지역정렬과 고정적 지역정렬을 사용했을 때,  $SP$  곡선은 두 방법이 거의 유사한 패턴으로 변화하는 것을 알 수 있다. 그리고  $SP$  곡선의 차이는 적응적 지역정렬이 고정적 지역정렬에 비해 전체적인 유사도가 높기 때문에 상승 곡선이 오른쪽



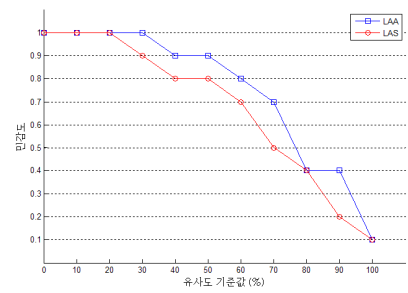
(a) ICPC06-C



(b) ICPC06-D

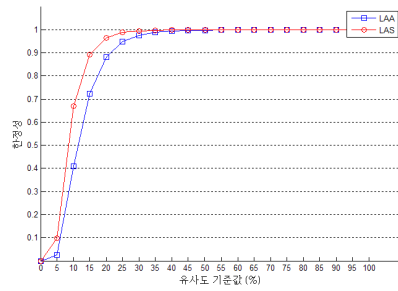


(c) ICPC06-E

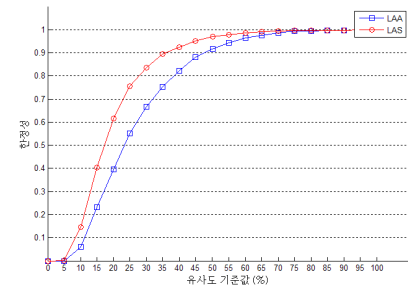


(d) ICPC06-F

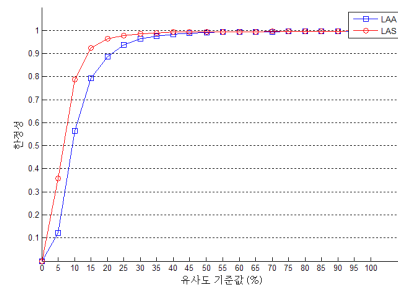
그림 5.8: 유사도 기준 값(x-축)의 변화에 따른 적응적 방법(LAA)과 고정적 방법(LAS)의 민감도 비교



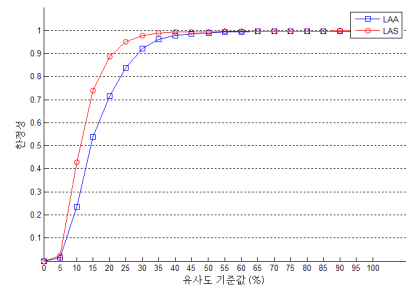
(a) ICPC06-C



(b) ICPC06-D



(c) ICPC06-E

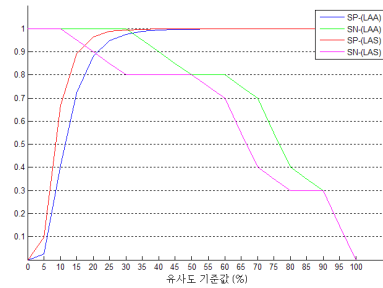


(d) ICPC06-F

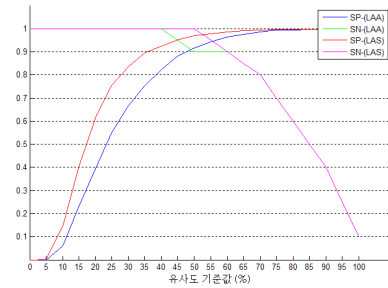
그림 5.9: 유사도 기준 값의 변화에 따른 적응적 방법(LAA)과 고정적 방법(LAS)의 민감도 비교

에 위치해 있다.

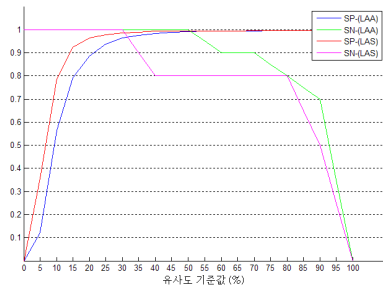
마지막으로 두 방법의 전체적인 성능을 평가하기 위해 민감도와 정확도의 교차점을 비교해 보았다. 두 방법의 성능평가는 JPlag와의 비교에서와 동일한 방법을 사용하였다. 그림 5.10은 *SN*과 *SP*의 교차점 비교 결과를 보여준다.



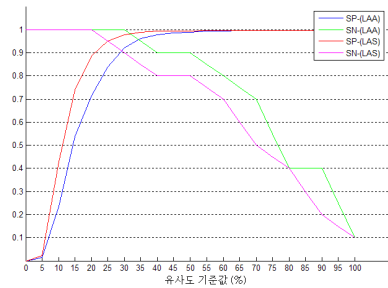
(a) ICPC06-C



(b) ICPC06-D



(c) ICPC06-E



(d) ICPC06-F

그림 5.10: 적응적 지역정렬과 고정적 지역정렬의 *SN*과 *SP* 교차 그래프

그림 5.10은 적응적 지역정렬과 고정적 지역정렬을 이용한 프로그램 표절 탐색 성능을 *SN*과 *SP*로 비교한 결과이다. 두 지역정렬의 성능은 정확도보다 민감도 차이에 의해 영향을 받는 것을 알 수 있다. 그림 5.10-(b)의 경우 민감도가 고정적 지역정렬(LAS)이 높기 때문에 표절 탐색 성능이 더 좋은 것으로 측정되었다. 하지만 (a)와 (c), (d) 그룹에서는 적응적 지역정렬(LAA)의 민감도가 우수하기 때문에 교차점을 비교에서 고정적 지역정렬에 비해 성능이 우수한 것을 알 수 있다. 이를 종합해 보면 적응적 지역정렬 방법은 고정적 지역정렬에

비해 민감도에서는 우수한 성능을 보였으며, 정확도는 비슷한 성능을 보였다.

## 5.6 프로그램 진화 분석

본 절에서는 비대칭 지역정렬에 기반한 프로그램 진화과정 분석의 효율성에 대해 평가하는 실험을 진행한다. 실험을 위해 인위적으로 표절한 프로그램 집합을 만들었다. 프로그램 집합은 4개의 그룹에 대해 20명의 대학원생들이 원본을 참조하여 유사하게 재작성하도록 하였다. 실험 데이터는 다음의 방식으로 생성하였다. 먼저 각 그룹별로 5개의 원본 프로그램을 선정하였다. 다음으로 10명의 학생에게 원본에서 2개의 프로그램을 유도하도록 하였다. 마지막으로 유도된 두 프로그램에서 하나를 선정하여 나머지 학생들에게 다시 유도하도록 하였다. 각 그룹당  $P_0 \mapsto \{P_1, P_2\}$ ,  $P_2 \mapsto \{P_3, P_4\}$ 와 같이 5개의 프로그램 집합이 생성된다. 표 5.8은 실험데이터의 통계적 정보를 보여준다.

표 5.8: 프로그램 진화분석 알고리즘 실험데이터

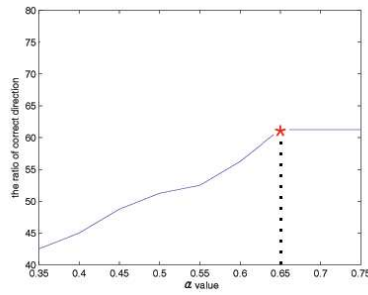
| 그룹  | 프로그램 | 트리 | 소스코드 |    |        |       |
|-----|------|----|------|----|--------|-------|
|     |      |    | 최고   | 최소 | 평균     | 표준편차  |
| P01 | 25   | 5  | 234  | 91 | 120.40 | 27.95 |
| P02 | 25   | 5  | 110  | 39 | 86.20  | 17.81 |
| P03 | 25   | 5  | 213  | 65 | 103.44 | 29.08 |
| P04 | 25   | 5  | 227  | 72 | 111.20 | 30.36 |

표 5.8의 유도된 프로그램 집합은 진화 트리로 표현 가능하다. 실험에서는 유사도 분석을 통해 구축한 프로그램 진화트리와 학생들로부터 인위적으로 유도한 프로그램들이 얼마나 동일한지를 측정한다. 알고리즘의 정확성은 트리를 생성했을 때 역방향 에지(inversion tree edge)의 발생 비율로 계산한다. 역방향 에지는 프로그램이 작성된 시간을 기준으로 잘못된 방향으로 가리키는 에지를 말한다. 하나의 프로그램 집합은 5개의 노드와 4개의 방향성 에지가 있다. 이를 종합하면 모든 그룹에서는 총 80개의 진화 에지가 만들어진다.

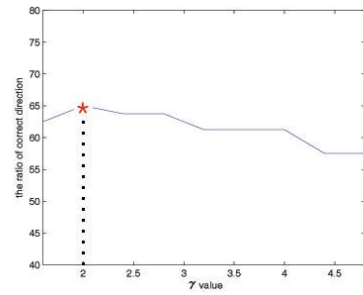
프로그램이 만들어진 시간을 기준으로하면  $P_t$ 는  $t$  시각에 만들어진 프로그램을 나타낸

다. 이를 간단히 표현하면,  $P_0$ 는 가장 먼저 생성된 프로그램이고,  $P_0$ 에서  $P_1, P_2, P_3, P_4$ 가 생성된다. 알고리즘에서 방향성 에지를  $\overrightarrow{(P_i, P_j)}$  같이 유추를 하면 프로그램 진화과정을 정확하게 분석한 것이다( $i < j$ ). 반대로  $\overrightarrow{(P_i, P_j)}(i > j)$ 일 경우 역방향 에지로 잘못된 분석이다. 그러므로 프로그램 진화트리에서는 역방향 에지를 최소한으로 줄여야 한다.

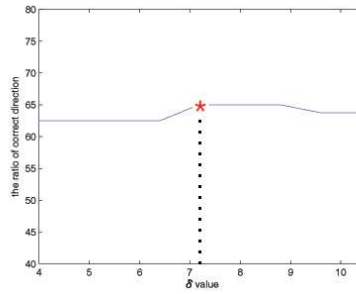
프로그램 진화 트리를 생성하기에 앞서 본 실험에서는 비대칭 유사도를 계산하기 위해 정의 9의  $Score_{Asym}$  매트릭스의 4가지 파라미터 변수에 대한 최적 값을 찾았다. 비대칭 지역 정렬의 성능은 4가지 파라미터에 따라 많은 영향을 받는다. 이 같은 문제는 다변량(multi-variable), 비선형 최적화(non-linear optimization) 문제이기 때문에 지역정렬에 대한 최적 함수는 없다. 실험에서는 순차적 파라미터 최적화(sequential parameter optimization) 방법으로 파라미터를 조정하였다. 그림 5.11은 파라미터 조정 결과를 보여준다.



(a)  $\alpha$  조정



(b)  $\gamma$  조정( $\alpha$  고정)



(c)  $\delta$  조정( $\alpha$ 와  $\gamma$  고정)

그림 5.11: 파라미터 조정 결과 :  $\alpha = 0.65$ ,  $\gamma = 2.0$ , and  $\delta = 7.2$ .

파라미터 조정은 일치를 위한  $\alpha$  변수와 공백 삽입에 대한 감점  $\delta$ , 공백 삭제에 대한 감점  $\gamma$  변수를 최적화 하였으며, 조정 변수를 제외한 기타 변수들을 고정시키는 방법으로 최상의 성능을 나타내는 값을 찾았다. 순차적 파라미터 조정 결과,  $\alpha = 0.65$ ,  $\beta = 0.35$ ,  $\gamma = 2.0$ ,  $\delta = 7.2$  일 때 정확성이 가장 높게 측정되었다. 이와 같이 조정된 파라미터를 알고리즘 1에 적용한 결과, 총 80개의 방향성 에지 중에 65%의 에지 방향을 올바르게 유추하였다. 4개의 프로그램 그룹(P01 ~ P04)에서 정확성은 각각 80%, 55%, 70%, 55%로 측정되었다. 그림 5.12는 그래프를 이용하여 진화트리를 구축한 결과를 보여준다.

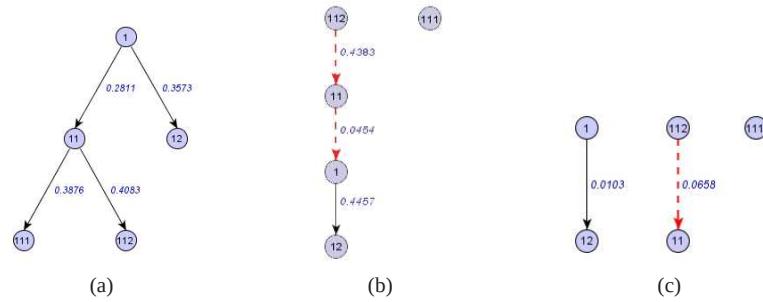


그림 5.12: 프로그램 진화 트리: 올바른 진화 트리 (a)와 잘못된 진화 트리 (b), (c)

그림 5.12-(a)는 4개의 에지에 대해 정확하게 유추를 한 경우를 나타내고, 그림 5.12-(b)와 (c)는 잘못된 결과이다. 그림에서 노드는 소스코드를 나타내고, 에지의 방향은 유도 방향을 나타낸다. 그리고 그림 5.12-(a)의 붉은색(점선) 에지는 역방향 에지로 잘못된 분석 결과를 나타낸다. 노드의 라벨에 표시된 숫자는 소스코드의 시간 인덱스로 자신이 어디로부터 유도되었는가를 나타낸다. 예를 들어, 1은 원본코드를 나타내고, 12는 1번 소스코드로부터 유도된 것을 나타낸다. 112는  $1 \mapsto 1 \mapsto 2$ 의 순으로 진화한 것을 의미한다.

다음으로 실제 개발된 프로그램을 대상으로 진화분석 알고리즘의 성능을 평가하기 위해 공개 소스 프로젝트를 대상으로 진화 과정을 분석하는 실험을 진행하였다. 이를 위해 실제로 많이 사용되고 있는 공개 소스 프로젝트 5종을 실험데이터로 선정하였다. 표 5.9는 실험데이터에 대한 정보를 보여준다.

표 5.9: 진화분석 실험에 사용된 공개 소스 프로젝트 실험데이터

| 번호 | 그룹                  | 소스코드  | 쌍(pair) | 소스코드   |    |         |         |
|----|---------------------|-------|---------|--------|----|---------|---------|
|    |                     |       |         | 최고     | 최소 | 평균      | 표준편차    |
| 1  | Apache-httpd-2.0.63 | 378   | 333     | 5,185  | 82 | 536.56  | 771.05  |
| 2  | Apache-httpd-2.2.15 | 450   |         | 9,207  | 82 | 587.90  | 1065.22 |
| 3  | CVS-1.7             | 135   | 131     | 8,622  | 25 | 691.94  | 1207.46 |
| 4  | CVS-2.3             | 135   |         | 8,945  | 25 | 700.57  | 1228.25 |
| 5  | GDB-6.4             | 1,229 | 1,017   | 28,949 | 25 | 1195.97 | 1732.19 |
| 6  | GDB-7.1             | 1,454 |         | 28,945 | 25 | 1226.84 | 1922.16 |
| 7  | GIT-1.61            | 237   | 159     | 4,927  | 13 | 442.99  | 608.23  |
| 8  | GIT-1.71            | 245   |         | 4,924  | 13 | 474.22  | 649.63  |
| 9  | Linux-2.6.30        | 165   | 156     | 10,242 | 37 | 762.55  | 1071.64 |
| 10 | Linux-2.6.34        | 179   |         | 9,208  | 37 | 815.24  | 1132.70 |

표 5.9와 같이 공개 소스 프로젝트 실험데이터로 아파치 웹서버, 버전 관리 시스템인 CVS와 GIT, 소스코드 디버거인 GDB, 리눅스 커널을 사용하였다. 여기서 리눅스 커널의 경우, 리눅스 프로젝트에서 “kernel” 폴더에 포함된 소스코드만 사용하였다. 그리고 실험은 다음과 같이 진행하였다. 먼저 각 프로젝트에서 두 가지 버전의 소스코드를 수집하였다. 그리고 프로젝트에서 C 소스코드만 추출하였다. 실험데이터에서 소스코드 크기가 매우 작거나 토큰 생성이 불가능한 소스코드는 제거하였다. 다음으로 소스코드로부터 순차적 방법으로 토큰 서열을 추출하고 프로그램 쌍에 대해 진화거리를 측정하였다. 마지막으로 프로그램 쌍에 대한 진화 방향을 결정하고, 진화 방향의 정확성을 평가하였다.

진화분석 평가에 사용한 실험데이터는 두 버전의 프로젝트에 같은 이름으로 포함되어 있는 소스코드만을 대상으로 하였다. 표 5.9의 프로그램 쌍 항목이 실제 진화분석 평가에 사용된 프로그램 쌍의 개수를 나타낸다. 프로그램 쌍은 두 버전의 프로젝트에서 같은 이름으로 작성된 소스코드의 개수이다. 예를 들어, GDB의 경우, GDB-6.4 버전의 1,236개 소스코드와 GDB-7.1 버전의 1,461개 소스코드 중에 1,017개의 소스코드 쌍을 실험데이터로 사용하였다. 표 5.10은 공개 소스 프로젝트를 이용한 진화분석 실험결과를 보여준다.

표 5.10: 공개 소스 프로젝트 진화분석 실험 결과

| 번호 | 그룹           | 쌍(pair) | 소스코드 수 |     |     |       |
|----|--------------|---------|--------|-----|-----|-------|
|    |              |         | 완전일치   | 정방향 | 역방향 | 비율    |
| 1  | Apache-httpd | 333     | 104    | 181 | 48  | 79.03 |
| 2  | CVS          | 131     | 87     | 33  | 11  | 75.00 |
| 3  | GDB          | 1,017   | 370    | 523 | 124 | 80.83 |
| 4  | GIT          | 159     | 33     | 101 | 25  | 80.16 |
| 5  | Linux        | 156     | 23     | 91  | 42  | 68.42 |

표 5.10에서 완전일치는 양방향으로 진화거리가 같은 것으로 높은 버전의 소스코드에 수정이 없었다는 것을 나타낸다. 즉, 높은 버전의 소스코드와 낮은 버전의 소스코드가 완전히 같다는 것을 나타낸다. 다음으로 정방향은 진화방향이 낮은 버전에서 높은 버전으로 향하는 소스코드 쌍의 개수를 나타내고, 역방향은 높은 버전에서 낮은 버전으로 진화방향이 결정된 소스코드 쌍의 개수를 나타낸다. 마지막으로 비율은 정방향 비율을 나타낸다.

표 5.10의 실험결과를 살펴보면, 5종의 프로젝트에서 모두 정방향 쌍의 비율이 역방향 비율보다 높은 것을 알 수 있다. 아파치 웹서버와 GDB, GIT의 경우 약 80% 정도의 소스코드 쌍의 진화방향이 낮은 버전에서 높은 버전으로 향하는 것을 알 수 있다. 그리고 버전 차이가 크지 않은 리눅스의 경우 다른 프로젝트들 보다 정방향 비율이 낮은 것을 알 수 있다. 표 5.10에서 볼 수 있는 것과 같이, 정방향 비율이 역방향 비율보다 높다는 것은 비대칭 지역정렬 기반의 진화분석이 프로그램의 진화과정을 올바르게 추정하고 있다는 것을 알 수 있다.

본 논문에서는 표절 프로그램과 공개 소스 프로젝트 소스코드를 이용한 진화분석 실험을 통해 프로그램 진화트리 생성알고리즘의 성능을 평가하였다. 실험결과 비대칭 지역정렬을 이용한 프로그램 진화분석이 가능함을 보였다. 본 논문의 진화트리 생성 알고리즘은 표절 관계 분석과 프로그램 진화과정 분석에 활용될 수 있다.

## 제 6장

# 결론

본 연구에서는 프로그램 유사도 분석 연구의 문제점들을 해결하기 위한 새로운 방법들을 제시하였다. 지금까지의 프로그램 유사도 분석에 기반한 표절 탐색 연구는 두 프로그램의 유사도를 정량적으로 측정하여 표절된 프로그램을 찾는 데에만 초점을 맞추어 왔다. 본 연구에서는 정량적 유사도 계산에서 한단계 더 나아가 다음 문제들을 해결하기 위한 방법을 제시하였다.

먼저, 두 프로그램의 유사도가 높게 측정되었을 때, 이 프로그램 쌍을 표절로 판정할 것인지 아닌지에 대한 판정 기준이 없다는 것이다. 같은 목적으로 작성된 프로그램들은 여러 가지 제약사항들로 인해 우연히 같게 작성될 수도 있기 때문이다. 이에 대해 본 연구에서는 길이가 다른  $M$ 과  $N$ 의 독립적인 프로그램에서 유사하게 작성되었을 때, 그 정도의 분포를 확률모델로 구성하여 그것이 우연히 나타날 가능성을 계산하고 그 값을 기준으로 두 개체의 표절 여부를 판정한다.

다음으로 두 프로그램이 표절로 판정되었을 때, 프로그램 표절 관계 분석이 어렵다는 것이다. 다시 말해서, 원본 프로그램과 표절 프로그램을 기계적으로 판정하기 어렵다는 것이다. 이에 대해 본 연구에서는 프로그램을 개체로 보고 생물정보학에서 연구되고 있는 개

체의 진화적 분석 방법을 통해 문제 해결에 접근하였다. 본 연구에서는 적응적 지역정렬에 기반한 비대칭 유사도 함수를 이용해 프로그램간 진화거리를 측정하고 소스코드의 추가, 삭제, 수정 등의 작업에 대한 소스코드 진화과정을 추정하는 알고리즘을 제안하였다.

마지막으로 실제 표절 데이터를 확보하기가 어렵다는 것이다. 지금까지의 연구에서는 가상의 표절데이터를 사용해 표절 탐색 시스템의 성능을 평가하였다. 가상의 데이터로는 표절시스템의 성능을 평가하기에 어려움이 있다. 이에 본 연구에서는 ACM에서 주최하는 ICPC 대회에 제출된 프로그램과 실제 대학의 프로그래밍 교과목에서 학생들이 제출한 프로그램을 이용하여 성능을 평가하였다. 또한, 실제 사용되고 있는 공개 소스 프로젝트를 이용하여 프로그램 진화분석 실험을 진행하였다. 그리고, 본 연구의 민감도와 정확도 실험에서는 시스템의 성능 평가를 위해 표절 프로그램을 생성하였다.

위와 같은 문제 해결을 위해 본 논문에서는 프로그램 유사도 분석을 위한 적응적 유사도 프레임워크를 제시하고 SoVAC 시스템을 구현하였다. 적응적 유사도 분석은 프로그램 집합에 사용된 키워드 빈도에 따라 매번 다른 유사도 점수 행렬을 만들어 낸다. 본 논문에서는 실험을 통해 적응적 유사도 분석 방법은 고정적 방법에 비해 민감도가 높게 측정되었으며, 프로그램 유사도 분석에 적합함을 보였다.

적응적 유사도 분석 프레임워크는 프로그램 선형화를 위한 토큰 서열 추출기만 구현하면 쉽게 다른 언어로 확장할 수 있다. 또한 적응적 프레임워크를 이루는 개별 모듈들은 각각 선택적으로 프로그램 유사도 분석 시스템 구현에 사용될 수 있다. 본 논문에서 구현한 적응적 프레임워크의 주요 특징을 요약하면 다음과 같다.

- **프로그램 정적 추적(PST):** 프로그램 선형화는 프로그램 소스코드로부터 중간 표현인 토큰서열을 생성한다. 여기서 프로그램 정적 추적은 파서에 의해 생성된 파스트리를 이용해 소스코드 수준에서 프로그램 시작에서부터 끝까지 정적으로 실행한다. 특히, 정적 추적에서는 함수 호출 그래프를 고려함으로써 프로그램 유사도 측정의 정확성을 향상한다. 또한, 순환호출 및 재귀호출 등을 함께 고려하고 있어 프로그램 유사도 분석의 안전성을 높이고

있다.

- **직병렬(SP) 그래프 모델:** C나 C++, 자바와 같은 프로그램은 함수나 메소드를 기본단위로 나뉜다. 그리고 함수들은 제어문이나 반복문으로 나뉠 수 있다. 제어문이나 반복문 같은 경우 조건식의 참, 거짓에 따라 실행 패스가 달라지는데, 적응적 프레임워크에서는 Serial-Parallel 그래프 모델을 이용해 정규화된 형식으로 변환한다. 이 모델은 PST와 함께 적응적 프레임워크의 전단부를 구성하며 안정적이고 유연한 중간표현을 생성함으로써 분석 결과의 정확성을 향상 시킨다.
- **적응적 지역정렬:** 지역정렬은 생물정보학에서 DNA나 RNA, 단백질 서열들의 유사도 분석에서 범용적으로 이용하는 방법으로 일치, 불일치, 길이 차이에 의한 공백 삽입에 의한 가점과 감점을 통해 두 서열의 유사도를 계산한다. 일반적인 지역정렬은 서열의 특성에 상관없이 고정적인 점수(일치:+1, 불일치:-1, 공백에 의한 일치:-2)를 할당한다. 적응적 지역정렬은 적응적 유사도 분석 프레임워크의 핵심으로 프로그램 집합의 키워드 빈도 벡터를 이용해 프로그램에 따라 동적으로 다른 점수를 부여한다. 이 방법에서는 중요한 키워드의 일치에 대해서는 높은 점수를 부여하고, 지정문(‘=’)과 같이 중요하지 않은 키워드에 대해서는 낮은 점수를 부여하여 자연스럽게 유연하게 유사도를 계산한다. 본 논문의 유사도 계산 방법은 정보이론에 입각한 새로운 방법이다.
- **비대칭 유사도 행렬:** 유사도 계산을 위한 함수는 일반적으로 대칭적(symmetric)으로 계산된다. 하지만 표절연구에서는 유사도 계산뿐만 아니라 원본과 표절을 가려내는 것이 중요하다. 적응적 프레임워크에서는 적응적 지역정렬을 확장해 비대칭 유사도 행렬로 유사도를 계산하는 *Asym()* 함수를 제안했다. *Asym()* 함수는 프로그램 수정 시 구문을 새로 삽입하는 것보다 삭제하는 것이 더욱 어렵다는 것을 가정으로 두 프로그램 사이의 유사도를 비대칭적으로 계산한다( $Asym(P_a, P_b) \neq Asym(P_b, P_a)$ ). 비대칭 유사도 행렬은 소스코드 클러스터링과 진화분석의 기본 유사도 계산 함수로 사용된다.
- **확률 분포 모델 기반 클러스터링:** 본 논문에서는 기존의 표절 탐색 연구에서보다 한 단계

발전한 확률 분포 모델에 기반한 클러스터링 알고리즘을 제시했다. 기존 표절 탐색 알고리즘에서는 두 개체의 유사도가  $p\%$  일 때, 이것을 어떻게 해석하는가에 대한 체계적인 연구가 없었다. 이에 대해 본 논문에서는 길이가 다른  $M$ 과  $N$ 의 독립적인 프로그램에서 한 부분이 같을 때, 그 정도의 분포를 굼벨 확률 분포 모델로 구성하여 그것이 우연히 나타날 가능성을 계산하고 실험적인 방법으로 소스코드를 클러스터링하는 알고리즘을 제시했다.

- **프로그램 진화과정 분석:** 본 논문에서는 비대칭 지역정렬에 기반한 프로그램 진화분석 알고리즘을 제시했다. 프로그램 진화분석은 생물 정보학 등에서 연구되고 있는 개체의 진화론적 분석 방법과 유사하게 프로그램 개체의 진화과정을 분석한 것이다. 본 연구에서는 두 프로그램 사이의 진화거리를 측정하고 진화방향을 결정한다. 표절 프로그램과 공개 소스 프로젝트를 이용한 진화분석 성능 평가 실험을 진행했으며, 실험결과 비대칭 지역정렬을 이용한 프로그램 진화분석이 가능함을 보였다.

본 논문에서 적응적 프레임워크를 기반으로 구현한 SoVAC 시스템은 프로그램 소스코드 집합을 입력으로 하여 유사도를 분석한 후, 프로그램 유사도 테이블, 소스코드 뷰, 시각화 등으로 분석 결과를 출력한다. 프로그램 유사도 테이블은 프로그램 쌍들의 유사도를 테이블 형식으로 보여주고, 소스코드 뷰는 개별적인 프로그램 쌍에서 실제 유사하게 작성된 부분을 소스코드와 직접 연결하여 보여준다. 그리고 시각화 도구인 PyView를 통해 그래프로 시각화하여 소스코드 클러스터링과 진화분석 결과를 보여준다.

마지막으로 본 논문에서는 실험을 통해 적응적 프레임워크의 성능을 평가하였다. 실험결과 적응적 지역정렬 알고리즘은 기존의 고정적 방법들 보다 견고하고 유연하게 표절 프로그램을 탐색할 수 있었다. 그리고 시스템의 성능을 평가하기 위한 민감도와 정확도 실험결과 JPlag 시스템의 GST 알고리즘과 고정적 지역정렬 알고리즘보다 우수한 성능을 나타낼 수 있었다. 또한, 비대칭 지역정렬을 이용한 프로그램 진화분석이 가능함을 보였다.

본 연구의 향후 과제로 본문에서도 언급했지만 표절에 대한 판단기준과 실제 표절과의

사표절 구분을 위한 이론적 모델 연구가 필요하다. 본 논문에서는 실험적 방법으로 극단치 분포 모델인 굼벨 분포 모델을 이용해 의사표절을 판정하는 방법을 제시했다. 본 논문에서 제시한 확률적 분포 모델이 생물정보학 등에서 DNA 서열에 대해 실험적으로 증명된 모델 이긴 하지만 프로그램에 적용했을 때 적합하지 않은 부분도 있었다. 향후에는 이를 개선한 이론적 모델에 대한 연구가 필요하다.

그리고 적응적 유사도 분석 프레임워크는 다양한 분야에 응용이 가능하다. 본 연구의 프로그램 소스코드 기반 유사도 분석뿐만 아니라 소스코드 없이 바이트코드(bytecode)를 이용해 두 프로그램의 유사도를 분석하는 연구를 진행했었다. 이를 통해, 적응적 프레임워크는 유사도 분석 연구에 다양하게 활용될 수 있음을 확인했다. 향후에는 적응적 프레임워크를 프로그램 뿐만 아니라 논문이나 웹문서를 대상으로 텍스트 마이닝이나 사람이 비교하기 어려운 Verilog나 VHDL과 같은 하드웨어 코드에 적용할 것이다. 또한 역으로 적응적 유사도 행렬은 DNA 서열 분석에도 적용 가능할 것으로 기대된다.

마지막으로 적응적 프레임워크를 이클립스 플러그인으로 통합하는 것이 필요하다. 이클립스는 많이 이용되고 있는 플랫폼으로 다양한 서비스를 구현한 플러그인들이 제공되고 있다. 적응적 프레임워크 기반 소프트웨어 클론 탐색 플러그인을 구현하여 CDT(C/C++ Development Toolkit)이나 JDT(Java Development Toolkit)과 연동한다면 실제 프로그램 개발에서도 활용 가능할 것이다.

## 참고문헌

- [1] Charlie Daly and Jane Horgan. A technique for detecting plagiarism in computer code. *Comput. J.*, 48(6):662–666, 2005.
- [2] Brenda Cheang, Andy Kurnia, Andrew Lim, and Wee-Chong Oon. On automated grading of programming assignments in an academic institution. *Comput. Educ.*, 41(2):121–131, 2003.
- [3] B. S. Baker. On finding duplication and near-duplication in large software systems. In *WCRE '95: Proceedings of the Second Working Conference on Reverse Engineering*, page 86, Washington, DC, USA, 1995. IEEE Computer Society.
- [4] Stefan Bellon, Rainer Koschke, Giulio Antoniol, Jens Krinke, and Ettore Merlo. Comparison and evaluation of clone detection tools. *IEEE Transactions on Software Engineering*, 33:577–591, 2007.
- [5] Chanchal K. Roy, James R. Cordy, and Rainer Koschke. Comparison and evaluation of code clone detection techniques and tools: A qualitative approach. *Sci. Comput. Program.*, 74(7):470–495, 2009.
- [6] Chanchal Kumar Roy and James R. Cordy. A survey on software clone detection research. *SCHOOL OF COMPUTING TR 2007-541, QUEEN'S UNIVERSITY*, 115, 2007.

- [7] J. W. Hunt and M. D. McIlroy. An algorithm for differential file comparison. Technical Report CSTR 41, Bell Laboratories, Murray Hill, NJ, 1976.
- [8] Maurice H. Halstead. *Elements of Software Science (Operating and programming systems series)*. Elsevier Science Inc., New York, NY, USA, 1977.
- [9] K. J. Ottenstein. An algorithmic approach to the detection and prevention of plagiarism. *SIGCSE Bull.*, 8(4):30–41, 1976.
- [10] H. L. Berghel and D. L. Sallach. Measurements of program similarity in identical task environments. *SIGPLAN Not.*, 19(8):65–76, 1984.
- [11] Sam Grier. A tool that detects plagiarism in pascal programs. In *SIGCSE '81: Proceedings of the twelfth SIGCSE technical symposium on Computer science education*, pages 15–20, New York, NY, USA, 1981. ACM.
- [12] J. Howard Johnson. Identifying redundancy in source code using fingerprints. In *CASCON '93: Proceedings of the 1993 conference of the Centre for Advanced Studies on Collaborative research*, pages 171–183. IBM Press, 1993.
- [13] Shauna D. Stephens. Using metrics to detect plagiarism (student paper). In *Proceedings of the seventh annual consortium for computing in small colleges central plains conference on The journal of computing in small colleges*, pages 191–196, , USA, 2001. Consortium for Computing Sciences in Colleges.
- [14] John L. Donaldson, Ann-Marie Lancaster, and Paula H. Sposato. A plagiarism detection system. In *SIGCSE '81: Proceedings of the twelfth SIGCSE technical symposium on Computer science education*, pages 21–25, New York, NY, USA, 1981. ACM.

- [15] Jean Mayrand, Claude Leblanc, and Ettore Merlo. Experiment on the automatic detection of function clones in a software system using metrics. In *ICSM '96: Proceedings of the 1996 International Conference on Software Maintenance*, page 244, Washington, DC, USA, 1996. IEEE Computer Society.
- [16] Jean-Francois Patenaude, Ettore Merlo, Michel Dagenais, and Bruno Laguë. Extending software quality assessment techniques to java systems. In *IWPC '99: Proceedings of the 7th International Workshop on Program Comprehension*, page 49, Washington, DC, USA, 1999. IEEE Computer Society.
- [17] T. Yamamoto, M. Matsusita, T. Kamiya, and K. Inoue. Measuring similarity of large software systems based on source code correspondence, 2002.
- [18] Kristina L. Verco and Michael J. Wise. Software for detecting suspected plagiarism: Comparing structure and attribute-counting systems. In *Proc. of 1st Australian Conference on Computer Science Education*, pages 86–95. ACM, 1996.
- [19] Stephane Ducasse, Oscar Nierstrasz, and Matthias Rieger. Lightweight detection of duplicated code - a language-independent approach. Technical report, University of Bern, Institute of Computer Science and Applied Mathematics, Bern, Switzerland, 2004.
- [20] Stéphane Ducasse, Oscar Nierstrasz, and Matthias Rieger. On the effectiveness of clone detection by string matching: Research articles. *J. Softw. Maint. Evol.*, 18(1):37–58, 2006.
- [21] Richard M. Karp and Michael O. Rabin. Efficient randomized pattern-matching algorithms. *IBM J. Res. Dev.*, 31(2):249–260, 1987.
- [22] Udi Manber. Finding similar files in a large file system. In *WTEC'94: Proceedings of the USENIX Winter 1994 Technical Conference on USENIX Winter 1994 Technical Conference*, pages 2–2, Berkeley, CA, USA, 1994. USENIX Association.

- [23] Seunghak Lee and Iryoung Jeong. Sdd: high performance code clone detection system for large scale source code. In *OOPSLA '05: Companion to the 20th annual ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications*, pages 140–141, New York, NY, USA, 2005. ACM.
- [24] Richard Wettel and Radu Marinescu. Archeology of code duplication: Recovering duplication chains from small duplication fragments. In *SYNASC '05: Proceedings of the Seventh International Symposium on Symbolic and Numeric Algorithms for Scientific Computing*, page 63, Washington, DC, USA, 2005. IEEE Computer Society.
- [25] Stéphane Ducasse, Matthias Rieger, and Serge Demeyer. A language independent approach for detecting duplicated code. In *ICSM '99: Proceedings of the IEEE International Conference on Software Maintenance*, page 109, Washington, DC, USA, 1999. IEEE Computer Society.
- [26] Yaron Caspi and Michal Irani. Spatio-temporal alignment of sequences. *IEEE Trans. Pattern Anal. Mach. Intell.*, 24(11):1409–1424, 2002.
- [27] Michael S. Waterman. *Introduction to Computational Biology: Sequences, Maps and Genomes*. CRC Press, 1995.
- [28] David Gitchell and Nicholas Tran. Sim: a utility for detecting similarity in computer programs. *SIGCSE Bull.*, 31(1):266–270, 1999.
- [29] Brenda S. Baker. Parameterized diff. In *SODA '99: Proceedings of the tenth annual ACM-SIAM symposium on Discrete algorithms*, pages 854–855, Philadelphia, PA, USA, 1999. Society for Industrial and Applied Mathematics.
- [30] Toshihiro Kamiya, Shinji Kusumoto, and Katsuro Inoue. Ccfinder: A multilinguistic token-

based code clone detection system for large scale source code. *IEEE Transactions on Software Engineering*, 28:654–670, 2002.

- [31] Hamid Abdul Basit, Simon J. Puglisi, William F. Smyth, Andrew Turpin, and Stan Jarzabek. Efficient token based clone detection with flexible tokenization. In *ESEC-FSE companion '07: The 6th Joint Meeting on European software engineering conference and the ACM SIGSOFT symposium on the foundations of software engineering*, pages 513–516, New York, NY, USA, 2007. ACM.
- [32] Zhenmin Li, Shan Lu, Suvda Myagmar, and Yuanyuan Zhou. Cp-miner: a tool for finding copy-paste and related bugs in operating system code. In *OSDI'04: Proceedings of the 6th conference on Symposium on Operating Systems Design & Implementation*, pages 20–20, Berkeley, CA, USA, 2004. USENIX Association.
- [33] Zhenmin Li, Shan Lu, Suvda Myagmar, and Yuanyuan Zhou. Cp-miner: Finding copy-paste and related bugs in large-scale software code. *IEEE Transactions on Software Engineering*, 32:176–192, 2006.
- [34] Xifeng Yan, Jiawei Han, and Ramin Afshar. Clospan: Mining closed sequential patterns in large datasets. In *In SDM*, pages 166–177, 2003.
- [35] Alex Aiken. Moss(measure of software similarity) plagiarism detection system <http://theory.stanford.edu/aiken/moss/>, 1998.
- [36] Brenda S. Baker and Udi Manber. Deducing similarities in java sources from bytecodes. In *ATEC '98: Proceedings of the annual conference on USENIX Annual Technical Conference*, pages 15–15, Berkeley, CA, USA, 1998. USENIX Association.
- [37] Lutz Prechelt, Guido Malpohl, and Michael Philippsen. Finding plagiarisms among a set of programs with JPlag. *Journal of Universal Computer Science*, 8(11):1016–1038, 2002.

- [38] Michael J. Wise. YAP3: improved detection of similarities in computer program and other texts. In *Proc. of SIGCSE '96*, pages 130–134, 1996.
- [39] Tetsuo Yamamoto, Makoto Matsushita, Toshihiro Kamiya, and Katsuro Inoue. Measuring similarity of large software systems based on source code correspondence. Technical report, In: *Product Focused Software Process Improvement*. Volume 3547 of *Lecture Notes in Computer Studies*. Springer Berlin / Heidelberg (2005) 530–544 <http://www.springerlink.com/content/h6m2vg5c3ejk38l4>, 2002.
- [40] Jeong-Woo Son, Seong-Bae Park, and Se-Young Park. Program plagiarism detection using parse tree kernels. In *Proceedings of the 9th Pacific Rim International Conference on Artificial Intelligence (PRICAI 2006)*, volume 4099 of *Lecture Notes in Computer Science*, pages 1000–1004. Springer, August 2006.
- [41] Lingxiao Jiang, Ghassan Misherghi, Zhendong Su, and Stephane Glondu. Deckard: Scalable and accurate tree-based detection of code clones. In *ICSE '07: Proceedings of the 29th international conference on Software Engineering*, pages 96–105, Washington, DC, USA, 2007. IEEE Computer Society.
- [42] Ira Baxter, Andrew Yahin, Leonardo Moura, Marcelo Sant’Anna, and Lorraine Bier. Clone detection using abstract syntax trees. In *Proceedings of the International Conference on Software Maintenance*, pages 368–377. IEEE Computer Society Press, 1998.
- [43] William S. Evans, Christopher W. Fraser, and Fei Ma. Clone detection via structural abstraction. *Reverse Engineering, Working Conference on*, 0:150–159, 2007.
- [44] Raimar Falke, Pierre Frenzel, and Rainer Koschke. Empirical evaluation of clone detection using syntax suffix trees. *Empirical Softw. Engg.*, 13(6):601–643, 2008.

- [45] Rainer Koschke, Raimar Falke, and Pierre Frenzel. Clone detection using abstract syntax suffix trees. In *WCRE '06: Proceedings of the 13th Working Conference on Reverse Engineering*, pages 253–262, Washington, DC, USA, 2006. IEEE Computer Society.
- [46] Robert Tairas and Jeff Gray. Phoenix-based clone detection using suffix trees. In *ACM-SE 44: Proceedings of the 44th annual Southeast regional conference*, pages 679–684, New York, NY, USA, 2006. ACM.
- [47] C.M.X. Benjamin, W.L. Woon, and K.S.D. Wong. A graphical and convenient tool for document comparison and visualization. pages 362 –367, may 2008.
- [48] Manuel Freire. Visualizing program similarity in the ac plagiarism detection system. In *AVI '08: Proceedings of the working conference on Advanced visual interfaces*, pages 404–407, New York, NY, USA, 2008. ACM.
- [49] J. Howard Johnson. Visualizing textual redundancy in legacy source. In *CASCON '94: Proceedings of the 1994 conference of the Centre for Advanced Studies on Collaborative research*, page 32. IBM Press, 1994.
- [50] T. Mala and T. V. Geetha. Visualization of plagiarism detected in documents. In *ICCIMA '07: Proceedings of the International Conference on Computational Intelligence and Multimedia Applications (ICCIMA 2007)*, pages 92–96, Washington, DC, USA, 2007. IEEE Computer Society.
- [51] Randy L. Ribler and Marc Abrams. Using visualization to detect plagiarism in computer science classes. In *INFOVIS*, pages 173–178, 2000.
- [52] Filip Van Rysselberghe and Serge Demeyer. Studying software evolution information by visualizing the change history. In *ICSM*, pages 328–337, 2004.

- [53] Teuvo Kohonen. Self-organized formation of topologically correct feature maps. *Biological Cybernetics*, 43(1):59–69, January 1982.
- [54] Mark Gabel, Lingxiao Jiang, and Zhendong Su. Scalable detection of semantic clones. In *ICSE '08: Proceedings of the 30th international conference on Software engineering*, pages 321–330, New York, NY, USA, 2008. ACM.
- [55] Raghavan Komondoor and Susan Horwitz. Using slicing to identify duplication in source code. In *SAS '01: Proceedings of the 8th International Symposium on Static Analysis*, pages 40–56, London, UK, 2001. Springer-Verlag.
- [56] Chao Liu, Chen Chen, Jiawei Han, and Philip S. Yu. Gplag: detection of software plagiarism by program dependence graph analysis. In *KDD '06: Proceedings of the 12th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 872–881, New York, NY, USA, 2006. ACM.
- [57] Nam H. Pham, Hoan Anh Nguyen, Tung Thanh Nguyen, Jafar M. Al-Kofahi, and Tien N. Nguyen. Complete and accurate clone detection in graph-based models. In *ICSE '09: Proceedings of the 2009 IEEE 31st International Conference on Software Engineering*, pages 276–286, Washington, DC, USA, 2009. IEEE Computer Society.
- [58] J. R. Ullmann. An algorithm for subgraph isomorphism. *J. ACM*, 23(1):31–42, 1976.
- [59] Raghavan Komondoor and Susan Horwitz. Effective, automatic procedure extraction. In *IWPC '03: Proceedings of the 11th IEEE International Workshop on Program Comprehension*, page 33, Washington, DC, USA, 2003. IEEE Computer Society.
- [60] Xin Chen, Brent Francia, Ming Li, Brian McKinnon, and Amit Seker. Shared information and program plagiarism detection. *IEEE Transactions on Information Theory*, 50(7):1545–1551, 2004.

- [61] Christian Arwin and S. M. M. Tahaghoghi. Plagiarism detection across programming languages. In *ACSC '06: Proceedings of the 29th Australasian Computer Science Conference*, pages 277–286, Darlinghurst, Australia, Australia, 2006. Australian Computer Society, Inc.
- [62] Jeong-Hoon Ji, Gyun Woo, and Hwan-Gue Cho. A plagiarism detection technique for java program using bytecode analysis. volume 1, pages 1092 –1098, nov. 2008.
- [63] Hyun-il Lim, Heewan Park, Seokwoo Choi, and Taisook Han. A method for detecting the theft of java programs through analysis of the control flow information. *Inf. Softw. Technol.*, 51(9):1338–1350, 2009.
- [64] Eugene W. Myers. An o(nd) difference algorithm and its variations. *Algorithmica*, 1:251–266, 1986.
- [65] Narayanan Shivakumar and Hector Garcia-molina. Scam: A copy detection mechanism for digital documents. In *In Proceedings of the Second Annual Conference on the Theory and Practice of Digital Libraries*, 1995.
- [66] Sergey Brin, James Davis, and Hector Garcia-molina. Copy detection mechanisms for digital documents. In *In Proceedings of the ACM SIGMOD Annual Conference*, pages 398–409, 1995.
- [67] Chang-Keon Ryu, Hyong-Jun Kim, and Hwan-Gue Cho. A detecting and tracing algorithm for unauthorized internet-news plagiarism using spatio-temporal document evolution model. In *SAC '09: Proceedings of the 2009 ACM symposium on Applied Computing*, pages 863–868, New York, NY, USA, 2009. ACM.
- [68] Chang-Keon Ryu, Hyong-Jun Kim, Seung-Hyun Ji, Gyun Woo, and Hwan-Gue Cho. Detecting and tracing plagiarized documents by reconstruction plagiarism-evolution tree. pages 119 –124, july 2008.

- [69] M. Vingron and M. S. Waterman. Sequence alignment and penalty choice. review of concepts, case studies and implications. *Journal of molecular biology*, 235(1):1–12, January 1994.
- [70] Michael J. Wise. Neweyes: A system for comparing biological sequences using the running karp-rabin greedy string-tiling algorithm. In *Proceedings of the 3rd International Conference on Intelligent Systems for Molecular Biology*, pages 393–401, 1995.
- [71] Jeong-Hoon Ji, Gyun Woo, and Hwan-Gue Cho. A source code linearization technique for detecting plagiarized programs. *ACM SIGCSE Bulletin*, 39(3):73–77, 2007.
- [72] M. S. Waterman and T. F. Smith. Identification of common molecular subsequences. *J. Mol. Biol.*, 147:195–197, 1981.
- [73] Karen Sparck Jones. A statistical interpretation of term specificity and its application in retrieval. pages 132–142, 1988.
- [74] Sandeep Tata and Jignesh M. Patel. Estimating the selectivity of tf-idf based cosine similarity predicates. *SIGMOD Rec.*, 36(4):75–80, 2007.
- [75] Ho Chung Wu, Robert Wing Pong Luk, Kam Fai Wong, and Kui Lam Kwok. Interpreting tf-idf term weights as making relevance decisions. *ACM Trans. Inf. Syst.*, 26(3):1–37, 2008.
- [76] Stephen Robertson. Understanding inverse document frequency: On theoretical arguments for idf. *Journal of Documentation*, 60:2004, 2004.
- [77] Stephen F. Altschul, Ralf Bundschuh, Rolf Olsen, and Terence Hwa. The estimation of statistical parameters for local alignment score distributions. *Nucleic Acids Research*, 29(2):351–361, 2001.

- [78] Samuel Karlin and Stephen F. Altschul. Methods for assessing the statistical significance of molecular sequence features by using general scoring schemes. In *Proc. Natl. Acad. Sci. USA*, volume 87, pages 2264–2268, March 1990.
- [79] Maik Kschischo, Michael Lässig, and Yi-Kuo Yu. Toward an accurate statistics of gapped alignment. *Bulletin of Mathematical Biology*, 67:169–191, 2005.
- [80] Jeong-Hoon Ji, Gyun Woo, Sang-Hyun Park, and Hwan-Gue Cho. An intelligent system for detecting source code plagiarism using a probabilistic graph model. In *MLDM Posters*, pages 55–69, 2007.
- [81] Jeong-Hoon Ji, Gyun Woo, Su-Hyun Park, and Hwan-Gue Cho. Understanding evolution process of program source for investigating software authorship and plagiarism. In *Proceedings of the 2nd International Conference on Digital Information Management*, 2007. *To appear*.

# 제 A장

## 부록

### A.1 프로그램 집합에 대한 키워드 빈도 조사

1. 키워드 빈도: ICPC(표 5.1), C++ 프로그래밍(표 5.2), GDB 프로젝트(표 5.9)

표 A.1: ICPC 실험데이터에 대한 전체 키워드 빈도: 총 247,798개의 키워드가 사용됨

| 높은 빈도 키워드             |       |                | 낮은 빈도 키워드 |           |           |
|-----------------------|-------|----------------|-----------|-----------|-----------|
|                       |       | 빈도% (개수)       |           |           | 빈도% (개수)  |
| ASSIGN_EQ             | "="   | 15.43 (38,237) | CATCH     | "catch"   | 0.00 (1)  |
| BLOCK_START           | "{"   | 9.67 (23,963)  | TRY       | "try"     | 0.00 (1)  |
| BLOCK_END             | "}"   | 9.67 (23,963)  | SHL_EQ    | "<="      | 0.00 (1)  |
| UNTRACKABLE_FUNC_CALL |       | 5.78 (14,319)  | TYPEDDEF  | "typedef" | 0.00 (2)  |
| INT                   | "int" | 5.75 (14,241)  | BITOR_EQ  | " ="      | 0.00 (6)  |
| INCR                  | "if"  | 5.58 (13,820)  | SIGNED    | "signed"  | 0.00 (6)  |
| IF                    | "=="  | 5.44 (13,470)  | CONST     | "const"   | 0.00 (6)  |
| LT                    | "<"   | 4.38 (10,844)  | BIT_XOR   | "^"       | 0.01 (14) |
| FOR                   | "for" | 4.06 (10,069)  | BITXOR_EQ | "^="      | 0.01 (15) |
| EQ                    | "=="  | 3.36 (8,330)   | STRUCT    | "struct"  | 0.01 (15) |

표 A.2: C++ 프로그래밍 실험데이터에 대한 키워드 빈도: 총 97,731개의 키워드가 사용됨

| 높은 빈도 키워드             |       |                | 낮은 빈도 키워드 |            |           |
|-----------------------|-------|----------------|-----------|------------|-----------|
|                       |       | 빈도% (개수)       |           |            | 빈도% (개수)  |
| ASSIGN_EQ             | "="   | 13.26 (12,961) | DO        | "do"       | 0.01 (1)  |
| BLOCK_START           | "{"   | 10.92 (10,670) | STRUCT    | "struct"   | 0.01 (5)  |
| BLOCK_END             | "}"   | 10.92 (10,670) | BIT_XOR   | "^"        | 0.01 (7)  |
| UNTRACKABLE_FUNC_CALL |       | 7.71 (7,531)   | DELETE    | "delete"   | 0.01 (7)  |
| INT                   | "int" | 5.02 (4,910)   | GOTO      | "goto"     | 0.01 (9)  |
| IF                    | "if"  | 4.69 (4,586)   | DEFAULT   | "default"  | 0.01 (14) |
| EQ                    | "=="  | 4.37 (4,272)   | BOOL      | "bool"     | 0.02 (15) |
| FUNC_CALL             |       | 3.71 (3,622)   | TRUE      | "true"     | 0.02 (16) |
| INCR                  | "++"  | 3.29 (3,215)   | UNSIGNED  | "unsigned" | 0.02 (18) |
| LT                    | "<"   | 3.04 (2,974)   | CATCH     | "catch"    | 0.02 (18) |

표 A.3: GDB 프로젝트에 대한 키워드 빈도: 총 2,862,665개의 키워드가 사용됨

| 높은 빈도 키워드   |          |                 | 낮은 빈도 키워드 |           |            |
|-------------|----------|-----------------|-----------|-----------|------------|
|             |          | 빈도% (개수)        |           |           | 빈도% (개수)   |
| BLOCK_START | "{"      | 15.89 (454,906) | MOD_EQ    | 0.00 (21) |            |
| BLOCK_END   | "}"      | 15.89 (454,906) | BOOL      | "bool"    | 0.00 (36)  |
| ASSIGN_EQ   | "="      | 12.26 (351,008) | TRY       | "try"     | 0.00 (59)  |
| PTR         | ""       | 8.54 (244,561)  | DIV_EQ    | "!="      | 0.00 (68)  |
| IF          | "if"     | 5.01 (143,432)  | VIRTUAL   | virtual   | 0.00 (86)  |
| BIT_AND     | "&&"     | 3.04 (87,098)   | FLOAT     | "float"   | 0.00 (142) |
| INT         | "int"    | 2.99 (85,502)   | BITXOR_EQ | "^="      | 0.00 (143) |
| RETURN      | "return" | 2.74 (78,563)   | SIGNED    | "signed"  | 0.00 (307) |
| CASE        | "case"   | 2.72 (77,993)   | DOUBLE    | "double"  | 0.01 (400) |
| EQ          | "=="     | 2.63 (75,228)   | SHL_EQ    | "<="      | 0.01 (491) |

# Program Similarity Analysis Framework Using Adaptive Sequence Alignment Technique

Jeong-Hoon Ji

*Department of Computer Engineering, Pusan National University*

## Abstract

Many software systems contain identical or very similar code fragments called *code clones* that are produced by copying and pasting with or without minor modifications. Cloning code is one of main reasons for higher maintenance cost. Similarly, illegal code theft/copying and program plagiarism have become widespread and serious problems in universities. It is nearly impossible for a human reader to detect plagiarism manually by comparing all possible pairs of programs. Program similarity analysis is available solution these problems.

Two methodologies are widely used for measuring the similarity of source codes. One is based on software metric comparison, called *fingerprint*, and the other is based on the program structure comparison. Fingerprint is a popular methodology in the early stages of a similarity measuring system but the detection capability of fingerprints is too dependent to the minor modifications of the source codes. In this respect, structure-based comparison approach is more robust than fingerprints. Structured-based comparison approaches typically include metric-based algorithm, string matching, token sequence alignment, parse tree comparison, visualization, and hybrid approaches.

This thesis proposes an adaptive framework for analyzing program similarities to support plagiarism detection and software maintenance. The framework addresses of several phases: program

linearization (tokenization), similarity measure, source code clustering, and evolutionary analysis. First, if a set of programs is given as input, each program in the set is linearized to a sequence of tokens (a program DNA) by a keyword extractor with Program Static Tracing (PST) method. The PST statically executes a program at the syntax-level and then extracts predefined keywords according to the order of the executed functions. Second, two enhanced local alignments are performed, namely the adaptive local alignment (*LAA*) and asymmetric local alignment (*Asym*). *LAA* exploits an adaptive similarity matrix. Each entry of the adaptive similarity matrix is the logarithm of the probability of each keyword computed from the frequencies in the input set of programs. The asymmetric similarity  $Asym(P_a, P_b)$  estimates the work needed to plagiarize the program  $P_a$  based on the program  $P_b$ . Finally, the framework provides clustering and phylogenetic forest construction algorithm in order to provide a clear-cut model based on a probability graph model on extreme distribution, for distinguishing real plagiarism from pseudo plagiarism for reconstructing a tree depicting the evolution process of the source codes.

To show the effectiveness and the efficiency of the proposed framework, some experiments have been conducted on a set of programs by more than 32 real programming contests and assignments. According to the experimental results, the adaptive algorithms are more sensitive than those of the previous local alignments based on fixed similarity matrices. In addition, the performance of the adaptive local alignment is superior to Greedy-String Tiling for detecting various plagiarism cases.

# 감사의 글

우선 학위 논문을 완성하기까지 도움을 주신 모든 분들께 감사의 마음을 전하고 싶습니다. 1998년 대학 입학식 날 대학원 과정까지 공부를 해야겠다는 막연한 목표를 세웠지만 솔직히 10년 넘게 학교를 다니게 될 줄은 몰랐습니다. 제 인생의 20대를 학교에서만 보낸 듯 하여 조금 아쉽지만 좋은 사람들과 연구를 하면서 즐겁게 보냈다는 생각이 듭니다. 학교에서 오랜 시간을 보낸 만큼 많은 기억들이 남아 있습니다. 물론 좋지 못한 기억들도 있지만 좋은 추억이 더 많으며 좋은 추억들만 간직하고자 합니다. 학위 과정을 마친다는 것이 마침표가 아닌 새로운 시작이라고 생각합니다. 다시 마음을 다잡고 더 큰 목표를 위해 열심히 노력하겠습니다.

많이 부족한 저에게 언제나 웃는 얼굴로 좋은 말씀과 가르침을 주신 우균 교수님께 존경과 감사의 마음을 표합니다. 교수님과 함께 할 수 있었기에 오늘의 결실을 맺을 수 있다고 생각합니다. 그리고 세상을 살아가는 방법과 야구, 맥주의 맛을 알게 해주신 조환규 교수님께도 감사의 마음을 전합니다. 모두 말씀 드릴 수 없지만 지금까지 함께 생활했던 실험실 후배님들께도 고마움을 표합니다. 아무것도 없었던 빈 실험실을 신희집처럼 하나씩 채워나갔던 동주와 한일군, 저 때문에 아직 부산에 남아 있다는 건우, 지금도 함께하고 있는 동생같은 정민, 주원, 진수, 똑똑이(은경)에게 고마움을 전합니다. 2년 넘는 시간 동안 야구를 함께 보며 소주잔을 기울여 준 배민정양에게는 특별히 감사의 마음과 술을 사겠습니다. 마지막으로 힘들었던 시간에 제 이야기를 잘 들어주시고 상담해주신 이윤정 박사님께도 고마움을 전합니다.

끝으로 지금 우리 집에서 무럭무럭 자라고 있는 이쁜 혜원이와 먹는 것만 보면 달려드는 묵돌이 지우에게 삼촌이 사랑한다는 말을 전합니다. 그리고 오늘의 제가 있기까지 헌신적으로 사랑을 주신 부모님과 곁에서 응원해 준 가족들에게 이 논문을 바칩니다.